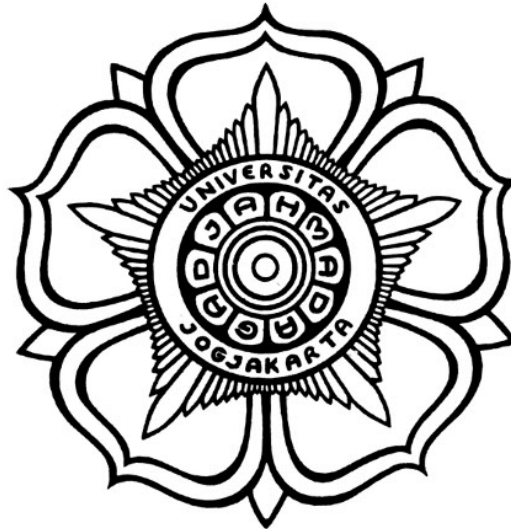


RANCANG BANGUN RUPIAH DIGITAL RITEL BERBASIS HYPERLEDGER FABRIC

SKRIPSI



***THE SUSTAINABLE DEVELOPMENT GOALS
Decent Work and Economic Growth
Industry, Innovation and Infrastructure***

Disusun oleh:

**Muhammad Fachrizal Giffari
22/504570/TK/55192**

**PROGRAM SARJANA PROGRAM STUDI TEKNOLOGI INFORMASI
DEPARTEMEN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI
FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

RANCANG BANGUN RUPIAH DIGITAL RITEL BERBASIS HYPERLEDGER FABRIC

SKRIPSI

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik
pada Departemen Teknik Elektro dan Teknologi Informasi
Fakultas Teknik
Universitas Gadjah Mada

Disusun oleh:

Muhammad Fachrizal Giffari
22/504570/TK/55192

Telah disetujui dan disahkan

Pada tanggal

Dosen Pembimbing I

Dosen Pembimbing II

Teguh Bharata Adji, S.T., M.T., M.Eng., Ph.D.
NIP. 196909201995121001

Dr. Ir. Guntur Dharma Putra, S.T., M.Sc.
NIP. 111199104201802102

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini :

Nama : Muhammad Fachrizal Giffari
NIM : 22/504570/TK/55192
Tahun terdaftar : 2022
Program : Sarjana
Program Studi : Teknologi Infomrasi
Fakultas : Teknik Universitas Gadjah Mada

Menyatakan bahwa dalam dokumen ilmiah Skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, 01 Juli 2026

Materai Rp10.000

(Tanda tangan)

Muhammad Fachrizal Giffari
22/504570/TK/55192

HALAMAN PERSEMBAHAN

Bismillaahirrahmaanirrahiim.

Skripsi ini kupersembahkan kepada kedua orang tuaku, saudaraku, dan juga
teman-temanku

Skripsi ini juga kupersembahkan kepada sahabatku yaitu almarhum Bagas Amar Hakiki
Biarlah halaman ini menjadi saksi bisu bahwa kamu adalah manusia yang baik.

KATA PENGANTAR

Puja dan puji syukur ke hadirat Allah Subhaanahu wa ta'aala atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga tugas akhir berupa penyusunan skripsi dengan judul "Rancang Bangun Rupiah Digital Ritel Berbasis Hyperledger Fabric" ini telah terselesaikan dengan baik. Dalam hal penyusunan tugas akhir ini, penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu, pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. Bapak Prof. Ir. Hanung Adi Nugroho, Ph.D., IPM, SMIEEE., selaku Ketua Departemen Teknik Elektro dan Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada.
2. Bapak Dr. Ir. Ahmad Nasikun, S.T., M.Sc., IPM., selaku Ketua Program Sarjana Program Studi Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada.
3. Bapak Teguh Bharata Adji, S.T., M.T., M.Eng., Ph.D., selaku Dosen Pembimbing I yang senantiasa memberikan bimbingan dan masukan yang terbaik dalam pelaksanaan serta penulisan laporan tugas akhir.
4. Bapak Dr. Ir. Guntur Dharma Putra, S.T., M.Sc., selaku Dosen Pembimbing II yang telah memberikan arahan dan masukan teknis yang terbaik dalam pelaksanaan tugas akhir.
5. Ibu Dr. Eng. Silmi Fauziati, S.T., M.T., selaku Dosen Pembimbing Akademik yang senantiasa membantu penulis perihal dinamika perkuliahan.
6. Seluruh dosen dan tenaga pendidik Departemen Teknik Elektro dan Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada yang telah memberikan bekal ilmu kepada penulis selama menjalani masa perkuliahan.
7. Bapak Sujono, S.E., Ibu Mulyati, S.E., dr. Puji Arifianti Ramadhany, dan Muhammad Feizal Syafaat, S.T., selaku keluarga penulis yang senantiasa memanjatkan doa kepada penulis serta memberikan dukungan, semangat, dan motivasi kepada penulis dalam pengerjaan tugas akhir.
8. UKM UGM eSports 2024 yang telah memberikan penulis pandangan baru terkait dinamika eSports, dinamika UGM, hingga dinamika negara.
9. Cornelius Arden Satwika Hermawan, S.T., dan Mahsa Quereda Bahjah, S.T., selaku rekan kelompok mata kuliah Pemrograman Berorientasi Obyek yang terus berlanjut hingga Teknisiade, PPK Ormawa, Bengkel Inovasi, Kerja Praktik. Semoga kita dapat menjadi rekan kerja lagi di kesempatan lain.
10. WS yang telah menemani selama 24 jam dan membuat hidup penulis menjadi warna warni. Terimakasih sudah membuat kehidupan kuliah menjadi seperti tujuh belas lagi seperti yang dideskripsikan oleh Tulus, "Muda jiwa selamanya muda, Kisah kita abadi selamanya".
11. Abid, Ale, Caca, Ervina, Lintang, Nabila, Niko, Sanie, Vivian, Yajna, Abang Efrem, Bapa Poli, Abang Rinto, Abang Rino, Mama Fani, dan semuanya yang tidak bisa penulis sebutkan. Terimakasih sudah menjadi keluarga untuk penulis di Desa Iso, Kecamatan Kei Kecil Timur. Terimakasih juga sudah merayakan ulang tahun penulis dengan cara yang sangat berkesan.

12. Tim Jelajah Kei Kecil 2025, khususnya untuk Almarhum Bagas Amar Hakiki. Terimakasih sudah menjadi sahabat yang selalu menemani penulis bahkan saat berada di fase terendah.
13. Teman-teman Program Studi Teknologi Informasi Departemen Teknik Elektro dan Teknologi Informasi Universitas Gadjah Mada Angkatan 2022 yang telah menemani dan membantu penulis selama menjalani masa perkuliahan.

Tiada gading yang tak retak. Penulis menyadari bahwa skripsi ini masih jauh dari kata sempurna. Oleh karena itu, penulis sangat mengharapkan segala bentuk masukan, saran, dan kritik yang membangun. Akhir kata, penulis berharap semoga tulisan ini dapat memberikan manfaat dan wawasan tambahan bagi kita semua, aamiin

Yogyakarta, Juni 2026

Muhammad Fachrizal Giffari

DAFTAR ISI

HALAMAN PENGESAHAN	ii
PERNYATAAN BEBAS PLAGIASI	iii
HALAMAN PERSEMBAHAN	iv
KATA PENGANTAR	vi
DAFTAR ISI	vii
DAFTAR TABEL	x
DAFTAR GAMBAR	xi
DAFTAR SINGKATAN.....	xiii
INTISARI.....	xiv
ABSTRACT	xv
BAB I Pendahuluan	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Tujuan Penelitian	3
1.4 Batasan Penelitian	4
1.5 Manfaat Penelitian	6
1.6 Sistematika Penulisan.....	6
BAB II Tinjauan Pustaka dan Dasar Teori	7
2.1 Tinjauan Pustaka	7
2.1.1 Penelitian Terdahulu tentang Desain dan Keamanan CBDC.....	7
2.1.2 Penelitian Terdahulu Berbasis Hyperledger Fabric.....	7
2.1.3 Posisi dan Celah Penelitian	8
2.2 Dasar Teori	9
2.2.1 Central Bank Digital Currency	9
2.2.1.1 Model Pembayaran Berbasis Akun dan Token	11
2.2.1.2 Arsitektur Penerbitan CBDC	14
2.2.2 Blockchain dan Distributed Ledger Technology	15
2.2.2.1 Struktur Blockchain	15
2.2.2.2 Algoritme Konsensus	15
2.2.2.3 Tipe Blockchain	16
2.2.3 Hyperledger Fabric.....	16
2.2.3.1 Arsitektur Hyperledger Fabric	16
2.2.3.2 Hyperledger Fabric Ledger.....	17
2.2.3.3 Alur Transaksi Hyperledger Fabric	18
2.2.4 Hyperledger Caliper	19
2.2.5 Kepatuhan: KYC, Tingkatan Dompot, dan Limit Transaksi	19

2.2.6	Transfer Ritel Langsung	20
2.2.7	Application Programming Interface	20
2.2.8	Bahasa Pemrograman Go	21
2.3	Analisis Perbandingan Metode	21
2.3.1	Perbandingan Hyperledger Fabric dengan Kerangka Kerja Blockchain Lain	21
2.3.2	Pemilihan Algoritme Konsensus Hyperledger Fabric	24
2.3.3	Pemilihan Alat Pengujian Blockchain	25
2.3.4	Pemilihan Arsitektur CBDC Dua Jenjang	25
2.3.5	Pemetaan Purwarupa terhadap Dimensi Desain CBDC Optimal ...	26
BAB III Metode Penelitian.....		31
3.1	Metode yang Digunakan.....	31
3.1.1	Metode Perancangan	31
3.1.1.1	Analisis Kebutuhan Sistem.....	31
3.1.1.2	Spesifikasi Kebutuhan Perangkat Lunak.....	32
3.1.1.3	Perancangan Activity Diagram.....	35
3.1.1.4	Perancangan Arsitektur Sistem Blockchain	42
3.1.1.5	Perancangan Basis Data Luring	50
3.1.2	Metode Implementasi	51
3.1.2.1	Software Development Life Cycle	51
3.1.2.2	Kerangka Kerja Blockchain	51
3.1.2.3	Mekanisme Konsensus	52
3.1.2.4	State Database	52
3.1.2.5	Basis Data Luring.....	52
3.1.2.6	Bahasa Pemrograman.....	54
3.1.3	Metode Pengujian	54
3.2	Alur Tugas Akhir	60
3.2.1	Mempelajari Proses Bisnis & Sistem Blockchain	61
3.2.2	Melakukan Studi Literatur	61
3.2.3	Menentukan Metode Penelitian	61
3.2.4	Pengembangan Sistem Blockchain bi-coin	61
3.2.4.1	Instalasi Kerangka Kerja Hyperledger Fabric	61
3.2.4.2	Konfigurasi Jaringan Blockchain	62
3.2.4.3	Deploy Chaincode	62
3.2.4.4	Pengembangan Fabric Gateway	65
3.2.4.5	Pengembangan Backend.....	65
3.2.5	Pengujian Sistem Blockchain bi-coin	66
3.2.5.1	Unit Test Chaincode dan Backend	67
3.2.5.2	Konfigurasi Hyperledger Caliper	70

3.2.5.3	Menjalankan Pengujian Hyperledger Caliper	71
BAB IV	Hasil dan Pembahasan	72
4.1	Hasil Implementasi Sistem	72
4.1.1	Jaringan Hyperledger Fabric	72
4.1.2	Chaincode Digital Rupiah	72
4.1.3	Backend API	74
4.1.4	Hasil Pengujian Fungsional	75
4.2	Hasil Evaluasi Kinerja	76
4.3	Perbandingan Hasil Penelitian dengan Hasil Terdahulu	82
BAB V	Kesimpulan dan Saran	84
5.1	Kesimpulan	84
5.2	Saran	84
DAFTAR PUSTAKA		86
LAMPIRAN		L-1
L.1	Source Code Tipe Data dan Struktur <i>Chaincode</i>	L-2
L.2	Source Code Otorisasi MSP <i>Chaincode</i>	L-6
L.3	Source Code Inisialisasi <i>Ledger</i>	L-7
L.4	Source Code Penerbitan dan Distribusi Likuiditas	L-8
L.5	Source Code Transfer dan Penegakan Limit	L-10
L.6	Source Code Kepatuhan KYC	L-11
L.7	Source Code Pengawasan dan Rekonsiliasi	L-13
L.8	Source Code Konfigurasi Jaringan Caliper	L-15
L.9	Source Code Suite Konfigurasi <i>Benchmark</i> Caliper	L-16
L.10	Source Code Skrip Pembangkit Aset Jaringan Caliper	L-25
L.11	Source Code <i>Workload</i> Transfer Ritel	L-26
L.12	Source Code <i>Workload</i> Beban Dasar Ritel	L-28
L.13	Source Code <i>Workload Negative-Path</i>	L-33
L.14	Source Code <i>Workload Aggregate Overspend</i>	L-37

DAFTAR TABEL

Tabel 2.1	Perbandingan penelitian terdahulu dengan penelitian ini.....	9
Tabel 2.2	Perbandingan sistem terpusat, <i>permissionless blockchain</i> , dan <i>permissioned blockchain</i> untuk CBDC.....	22
Tabel 2.3	Perbandingan platform DLT <i>permissioned</i> untuk sistem CBDC.....	23
Tabel 2.4	Perbandingan algoritme konsensus untuk <i>blockchain permissioned</i> ..	24
Tabel 2.5	Perbandingan Hyperledger Caliper dan BlockBench berdasarkan [1, 2].	25
Tabel 2.6	Perbandingan arsitektur CBDC satu jenjang dan dua jenjang.....	26
Tabel 2.7	Pemetaan purwarupa terhadap dimensi desain CBDC optimal [3] ...	27
Tabel 3.1	Kebutuhan fungsional purwarupa bi-coin	33
Tabel 3.2	Kebutuhan nonfungsional purwarupa bi-coin	34
Tabel 3.3	Keterlacakan kebutuhan terhadap metode verifikasi.....	35
Tabel 3.4	Organisasi pada jaringan Rupiah Digital	47
Tabel 3.5	Objek status utama pada ledger.....	49
Tabel 3.6	Perbandingan basis data status Hyperledger Fabric berdasarkan dokumentasi resmi [4].	52
Tabel 3.7	Perbandingan basis data kandidat untuk penyimpanan luring KYC. .	54
Tabel 3.8	Skenario beban pengujian kinerja	56
Tabel 3.9	Profil dan kasus uji khusus beserta hasil yang diharapkan	57
Tabel 3.10	Lingkungan pengujian purwarupa bi-coin	71
Tabel 4.1	Kelompok fungsi chaincode digital-rupiah.....	72
Tabel 4.2	Batas tingkatan dompet ritel (dalam Rupiah)	74
Tabel 4.3	Peran akses pada backend API	75
Tabel 4.4	Lingkungan perangkat keras dan perangkat lunak evaluasi kinerja v3	77
Tabel 4.5	Pemakaian sumber daya Docker pada profil transfer v3.0	78
Tabel 4.6	Hasil konformitas Caliper pada <i>chaincode</i> v3.0	78
Tabel 4.7	Hasil profil transfer Caliper v3.0 dengan 1 <i>worker</i>	78
Tabel 4.8	Konfigurasi evaluasi transfer ritel.....	80
Tabel 4.9	Baseline historis v2: hasil transfer dasar pada ronde terukur	80
Tabel 4.10	Baseline historis v2: hasil lima ronde pengulangan beban 60 TPS dengan 2 <i>worker</i>	81
Tabel 4.11	Baseline historis v2: hasil profil fungsional.....	81
Tabel 4.12	Baseline historis v2: hasil oracle jalur negatif dan pengeluaran agregat	81
Tabel 4.13	Perbandingan penelitian ini dengan tinjauan literatur Aryani (2023) dan PoC Proyek Garuda (2024).	82

DAFTAR GAMBAR

Gambar 2.1	Taksonomi CBDC berdasarkan segmen pengguna dan model distribusi; model dua jenjang yang diadopsi pada penelitian ini ditampilkan dengan latar abu-abu sesuai rancangan Rupiah Digital Bank Indonesia	10
Gambar 2.2	Peta jalan pengembangan Rupiah Digital dalam Proyek Garuda. Sumber: Bank Indonesia [5].	11
Gambar 2.3	Perbandingan model pembayaran CBDC berbasis akun dan berbasis token: model akun memverifikasi identitas pemegang, sedangkan model token memverifikasi keaslian objek nilai. Diadaptasi dari Auer dan Böhme [6].	12
Gambar 2.4	Konfigurasi desain Rupiah Digital menurut White Paper CBDC Bank Indonesia. Sumber: Bank Indonesia [5].	13
Gambar 2.5	Perbandingan arsitektur penerbitan CBDC: tidak langsung (<i>indirect</i> /dua jenjang), langsung (<i>direct</i> /satu jenjang), dan hibrida (<i>hybrid</i>). Diadaptasi dari Auer dan Böhme [6]	14
Gambar 2.6	Struktur rantai blok: setiap blok memuat <i>hash</i> kriptografis dari blok sebelumnya	15
Gambar 2.7	Pohon Merkle: <i>hash</i> transaksi digabungkan secara berjenjang membentuk Merkle Root yang disimpan dalam kepala blok	15
Gambar 2.8	Diagram transisi <i>state</i> algoritme konsensus Raft: setiap simpul berstatus <i>Follower</i> , <i>Candidate</i> , atau <i>Leader</i>	16
Gambar 2.9	Arsitektur komponen Hyperledger Fabric: klien mengirim proposal ke <i>peer endorser</i> , mengirim transaksi terendorsement ke <i>orderer</i> , lalu menerima komitmen blok pada semua <i>peer</i> dalam <i>channel</i>	17
Gambar 2.10	Dualitas komponen <i>ledger</i> Hyperledger Fabric: <i>blockchain</i> sebagai catatan transaksi historis tambah-saja yang perubahan retrospektifnya dapat dideteksi, dan <i>world state</i> sebagai representasi status terkini pada CouchDB	18
Gambar 2.11	Siklus transaksi Hyperledger Fabric pada topologi uji dengan satu <i>consenter</i> Raft tanpa toleransi kegagalan <i>orderer</i>	19
Gambar 2.12	Perbandingan <i>throughput</i> transfer aset pada Ethereum, Hyperledger Fabric, dan Besu. Sumber: [7].	24
Gambar 2.13	Arsitektur Rupiah Digital yang diusulkan Aryani [8].	29
Gambar 3.14	Siklus SDLC dengan metode <i>prototyping</i> : empat fase berulang hingga purwarupa memenuhi kebutuhan fungsional dan kinerja ..	31
Gambar 3.15	Activity diagram pendaftaran peserta	36
Gambar 3.16	Activity diagram penerbitan Digital Rupiah	37
Gambar 3.17	Activity diagram distribusi likuiditas ke PJP	38
Gambar 3.18	Activity diagram onboarding KYC pelanggan	39
Gambar 3.19	Activity diagram transfer saldo ritel	40
Gambar 3.20	Activity diagram pembekuan dan pencairan peserta oleh Bank Indonesia	41
Gambar 3.21	Activity diagram laporan dan pengawasan supervisi	42

Gambar 3.22	Arsitektur distribusi dua jenjang pada purwarupa Rupiah Digital ritel.	43
Gambar 3.23	Arsitektur teknologi Rupiah Digital menurut White Paper CBDC Bank Indonesia. Sumber: Bank Indonesia [5].	44
Gambar 3.24	Arsitektur sistem Rupiah Digital: lapisan <i>client</i> , <i>backend API</i> (Go REST + JWT/RBAC), basis data PostgreSQL luring, <i>Fabric Gateway SDK</i> , serta jaringan Hyperledger Fabric 2.5 dengan lima organisasi <i>peer</i> , CouchDB, CA per organisasi, dan <i>orderer Raft</i>	46
Gambar 3.25	Arsitektur jaringan Hyperledger Fabric sistem Rupiah Digital (5 organisasi)	47
Gambar 3.26	Diagram use case keseluruhan sistem Rupiah Digital.	48
Gambar 3.27	Diagram use case operasi institusional.	48
Gambar 3.28	Diagram use case operasi retail	49
Gambar 3.29	Diagram <i>Entity-Relationship</i> basis data luring (<i>off-chain</i>) pada PostgreSQL. Tabel <i>auth_users</i> berdiri sendiri untuk autentikasi portal, sedangkan <i>kyc_profiles</i> menjadi pusat relasi profil KYC, riwayat pemeriksaan penyedia, dan jejak audit.	51
Gambar 3.30	Alur pelaksanaan tugas akhir: dari studi literatur hingga penarikan kesimpulan, dengan umpan balik iteratif antara implementasi dan pengujian.	61
Gambar 3.31	Implementasi fungsi Mint pada <i>chaincode</i> : memverifikasi status beku dan batas saldo tingkatan sebelum menambah saldo dompet.	63
Gambar 3.32	Penegakan kebijakan transfer bersama untuk batas keluar, batas masuk, saldo maksimum, dan kecukupan saldo	64
Gambar 3.33	Penurunan tingkatan dompet ritel dari jenis subjek, risiko, dan tingkat uji tuntas KYC	65
Gambar 3.34	Pengelompokan rute REST <i>backend</i> berdasarkan peran akses: setiap subrouter menerapkan <i>middleware RequireRole</i> yang berbeda sesuai hierarki wewenang	66
Gambar 3.35	Unit test penolakan pembuatan dompet saat tingkatan yang diminta tidak cocok dengan tingkatan hasil penurunan kebijakan KYC (berkas <i>chaincode/digital_rupiah_test.go</i>)	68
Gambar 3.36	Unit test penolakan transfer ketika dompet pengirim belum memiliki jangkar KYC yang disetujui (berkas <i>chaincode/digital_rupiah_test.go</i>)	69
Gambar 3.37	Unit test penolakan transfer ketika pemilik dompet pengirim tidak cocok dengan identitas JWT pemanggil (berkas <i>backend/handlers/authorization_test.go</i>)	70
Gambar 4.38	Siklus hidup peserta pada <i>chaincode</i> : dari <i>pending</i> (menunggu persetujuan) menjadi <i>active</i> , kemudian dapat dibekukan (<i>frozen</i>) dan dicairkan kembali. Penolakan menghapus peserta yang masih <i>pending</i> , sedangkan <i>OffboardParticipant</i> menyimpan status <i>offboarded</i> , membekukan dompet terkait, dan mensyaratkan saldo agregat nol.	73

DAFTAR SINGKATAN

API	=	<i>Application Programming Interface</i>
AML	=	<i>Anti-Money Laundering</i>
BI	=	Bank Indonesia
BSPI	=	Blueprint Sistem Pembayaran Indonesia
CBDC	=	<i>Central Bank Digital Currency</i>
CA	=	<i>Certificate Authority</i>
CFT	=	<i>Combating the Financing of Terrorism</i>
DLT	=	<i>Distributed Ledger Technology</i>
KYC	=	<i>Know Your Customer</i>
MSP	=	<i>Membership Service Provider</i>
OJK	=	Otoritas Jasa Keuangan
PJP	=	Penyedia Jasa Pembayaran
REST	=	<i>Representational State Transfer</i>
RTGS	=	<i>Real-Time Gross Settlement</i>
TEE	=	<i>Trusted Execution Environment</i>
TLS	=	<i>Transport Layer Security</i>
TPS	=	<i>Transactions Per Second</i>
UTXO	=	<i>Unspent Transaction Output</i>

INTISARI

Eksperimen publik Proyek Garuda Bank Indonesia berfokus pada mata uang digital bank sentral (*Central Bank Digital Currency*, CBDC) *wholesale*. Penelitian ini mengkaji cara purwarupa Rupiah Digital ritel merepresentasikan distribusi dua jenjang, kebijakan dompet, dan pengawasan institusional pada buku besar *permissioned*. Purwarupa menggunakan Hyperledger Fabric 2.5, *chaincode* dan layanan *backend* Go, CouchDB, serta PostgreSQL.

Data identitas mentah disimpan di PostgreSQL, sedangkan buku besar memuat status KYC, risiko, masa berlaku, dan hash dokumen. Pemisahan ini tidak memberi privasi transaksi pada kanal bersama. Bank atau PJP memasok keputusan KYC. *Chaincode* menurunkan tingkatan dompet BASIC, STANDARD, dan MERCHANT serta memvalidasi limit transaksi, harian, bulanan, dan saldo. Pemeriksaan kepemilikan pada *backend* dan MSP pada *chaincode* membatasi pengguna dan organisasi. Kontrol ini tidak membuktikan kepatuhan APU/PPT menyeluruh atau persetujuan empat mata.

Pengujian otomatis dan jalur negatif dengan diagnostik yang spesifik mengevaluasi otorisasi serta penegakan kebijakan. Hyperledger Caliper mengevaluasi populasi ritel tetap pada beban 10, 30, dan 60 TPS dengan buku besar bersih untuk setiap profil, *seed* deterministik, pengulangan beban puncak, dan skenario pengeluaran agregat yang melebihi saldo. Pada beban 30 TPS, *throughput* sukses mencapai 29,5 TPS tanpa kegagalan; pada 60 TPS, nilainya 41,2–47,0 TPS. Ketujuh kasus negatif ditolak dengan alasan yang tepat. Pada skenario pengeluaran berlebih, 4 dari 200 transfer dikomit dan saldo akhir tetap nonnegatif. Hasil ini merupakan garis dasar purwarupa lokal yang dapat diulang, bukan bukti reproduksibilitas lintas mesin atau kapasitas produksi. Purwarupa ini merupakan artefak yang dapat diaudit untuk eksperimen kebijakan Rupiah Digital ritel dalam batas yang dinyatakan.

Kata kunci: Rupiah Digital, CBDC ritel, Hyperledger Fabric, KYC berbasis risiko, limit dompet

ABSTRACT

Bank Indonesia’s public Project Garuda experiments focus on wholesale central bank digital currency. This study asks how a testable retail Digital Rupiah prototype can represent two-tier distribution, wallet policies, and institutional oversight on a permissioned ledger. It uses Hyperledger Fabric 2.5, Go chaincode and backend services, CouchDB, and PostgreSQL.

Raw identity data remains in PostgreSQL; the ledger stores KYC status, risk, expiry, and document hashes. This separation does not provide transaction privacy on the shared channel. Banks or payment service providers supply KYC decisions. The chaincode derives BASIC, STANDARD, and MERCHANT wallet tiers and validates transaction, daily, monthly, and balance limits. Backend ownership and chaincode MSP checks enforce user and organization boundaries. These controls do not demonstrate complete AML/CFT compliance or four-eyes approval.

Automated tests and exact-diagnostic negative paths evaluate authorization and policy enforcement. Hyperledger Caliper evaluates a fixed retail population under 10, 30, and 60 TPS offered loads, with a clean ledger for each profile, deterministic seeding, repeated peak rounds, and an aggregate-overspend scenario. At a 30 TPS offered load, successful throughput reaches 29.5 TPS without failures; at 60 TPS, it ranges from 41.2 to 47.0 TPS. All seven negative cases are rejected for the expected reason. In the overspend scenario, 4 of 200 transfers commit and the final balance remains nonnegative. These results form a repeatable local baseline, not evidence of cross-machine reproducibility or production capacity. The prototype is an auditable artifact for retail Digital Rupiah policy experiments within the stated limitations.

Keywords: Digital Rupiah, retail CBDC, Hyperledger Fabric, risk-based KYC, wallet limits

BAB I

PENDAHULUAN

1.1 Latar Belakang

Transformasi digital pada sektor sistem pembayaran Indonesia berlangsung sangat cepat dalam satu dekade terakhir. Penetrasi telepon pintar dan pertumbuhan ekonomi digital mendorong masyarakat beralih dari uang tunai menuju instrumen pembayaran nontunai. Bank Indonesia melalui Blueprint Sistem Pembayaran Indonesia (BSPI) menempatkan digitalisasi sebagai agenda utama untuk menjaga kedaulatan moneter, mendorong inklusi keuangan, serta memperkuat integrasi ekonomi dan keuangan digital nasional [9]. Dalam konteks tersebut, kehadiran mata uang digital bank sentral atau *Central Bank Digital Currency* (CBDC) menjadi salah satu inisiatif strategis untuk melengkapi uang kartal dan uang elektronik yang telah ada.

Pertumbuhan ekosistem digital tersebut, bagaimanapun, juga membawa risiko baru bagi stabilitas sistem keuangan. Pada tahun 2022, penetrasi internet Indonesia mencapai 74% dengan 204,7 juta pengguna, sementara pelanggan seluler mencapai 370,1 juta atau setara 133% populasi. Pandemi COVID-19 menambah 21 juta konsumen digital baru yang sebelumnya belum terlayani secara daring, sebagaimana dirangkum dalam Tabel 1 White Paper CBDC Bank Indonesia [5]. Pada periode yang sama, kapitalisasi pasar aset kripto global melonjak hingga hampir 3 triliun dolar AS, tumbuh 1.393% antara 2019 dan 2021, sementara investor aset kripto di Indonesia mencapai 16,3 juta per September 2022 dengan laju pertumbuhan 81,6% *year-over-year* [5]. Fenomena ini memunculkan *cryptoization*, yaitu kondisi di mana adopsi aset kripto oleh penduduk menyebar lebih cepat dibandingkan mata uang resmi [5]. Disrupsi digital tidak lagi terbatas pada isu *shadow banking*, melainkan telah merambah pada fenomena *shadow currency* dan *shadow central banking* yang mengikis efektivitas kebijakan bank sentral [5]. *Stablecoin* dan aset kripto tanpa aset dasar memperbesar risiko pencucian uang (*money laundering*), pendanaan terorisme, serta risiko makrofinansial berupa melemahnya kendali otoritas atas stabilitas moneter dan sistem keuangan [5]. Mata uang digital bank sentral dirancang untuk menjaga kepercayaan publik terhadap uang dan kedaulatan moneter, sekaligus mempertahankan peran bank sentral sebagai jangkar stabilitas dalam ekonomi digital [5, 10].

CBDC merupakan representasi digital dari uang yang diterbitkan dan menjadi kewajiban langsung bank sentral, sehingga memiliki kualitas bebas risiko yang setara dengan uang tunai [11]. Berbeda dengan uang elektronik yang diterbitkan oleh penyelenggara swasta dan merupakan kewajiban penerbitnya, CBDC menempatkan bank sentral sebagai penjamin nilai akhir. Bagi Indonesia, Bank Indonesia menamai

inisiatif CBDC sebagai Rupiah Digital melalui Proyek Garuda, yang menempuh pendekatan bertahap dimulai dari Rupiah Digital *wholesale* (w-Digital Rupiah) untuk transaksi antarlembaga, kemudian diperluas menuju Rupiah Digital *retail* (r-Digital Rupiah) bagi masyarakat luas [5, 12]. Desain Rupiah Digital mengadopsi model dua jenjang (*two-tier*), yaitu Bank Indonesia menerbitkan dan mengelola Rupiah Digital pada tingkat *wholesale*, sementara distribusi kepada masyarakat dilakukan melalui pihak perantara seperti perbankan dan penyedia jasa pembayaran [13].

Salah satu pilihan teknologi yang banyak dikaji untuk menopang CBDC adalah *Distributed Ledger Technology* (DLT) berbasis *blockchain*. Teknologi ini menawarkan keunggulan berupa integritas data yang terjaga melalui catatan yang bersifat *append-only*, jejak audit yang transparan, serta kemampuan otomasi aturan bisnis melalui *smart contract* [14, 15]. Untuk kebutuhan bank sentral, jenis *blockchain* yang sesuai adalah *permissioned blockchain*, yaitu jaringan dengan keanggotaan terbatas dan teridentifikasi, sehingga bank sentral tetap memegang kendali atas tata kelola jaringan [16]. Hyperledger Fabric merupakan salah satu kerangka kerja *permissioned blockchain* paling matang yang banyak digunakan pada ranah keuangan. Fabric mendukung arsitektur modular, identitas berbasis *Membership Service Provider* (MSP), pemisahan transaksi melalui kanal (*channel*), serta model eksekusi *execute-order-validate* yang mendukung pemrosesan paralel tanpa bergantung pada mekanisme *proof-of-work* yang boros energi [17].

Meskipun kajian mengenai CBDC berkembang pesat, fokus literturnya masih timpang. Sebagian besar penelitian menelaah CBDC dari sisi makroekonomi, kebijakan moneter, dan stabilitas keuangan [10, 11, 18], atau dari sisi taksonomi desain tingkat tinggi seperti pilihan *token-based* versus *account-based* dan model satu jenjang versus dua jenjang [6, 19–21]. Sebaliknya, artefak teknis yang utuh, terbuka, dan dapat diuji secara empiris untuk konteks CBDC ritel masih jarang tersedia.

Kajian teknis yang ditinjau masih membahas komponen CBDC secara terpisah. Privasi dan auditabilitas dibahas dalam [22–24], keamanan dan pemodelan ancaman dalam [25], serta arsitektur jaringan dalam [16, 26]. Kelayakan sistem CBDC ritel, sebaliknya, bergantung pada keterpaduan komponen tersebut. Dalam korpus literatur yang digunakan pada penelitian ini, penerbitan dan distribusi likuiditas berjenjang, purwarupa KYC-tier dan limit transaksi, transfer ritel, serta pengawasan belum dievaluasi sebagai satu artefak Fabric yang kinerjanya dapat diukur.

Untuk konteks Indonesia, Bank Indonesia telah menetapkan arah desain Rupiah Digital berupa model dua jenjang dan interoperabilitas sistem pembayaran [5, 13]. Desain CBDC bagi perekonomian Indonesia juga telah dirumuskan dalam [8, 27, 28]. Namun, realisasi teknis Proyek Garuda yang terdokumentasi secara publik baru mencakup lapisan *wholesale* [12], sedangkan purwarupa lapisan *retail* yang selaras

dengan spesifikasi tersebut belum tersedia secara terbuka. Dari sisi evaluasi kinerja, CBDC ritel yang telah diluncurkan maupun yang masih berupa pilot (seperti eNaira di Nigeria [29] dan pilot ritel di Thailand [30]) belum menyertakan tolok ukur kinerja terstandar yang dapat dibandingkan langsung dengan purwarupa ini. Sistem berkinerja tinggi seperti Hamilton/OpenCBDC menggunakan arsitektur *account-based* terpusat, bukan model *blockchain* [31]. Akibatnya, data *throughput* dan latensi yang terukur untuk skenario CBDC ritel di atas *permissioned blockchain*, khususnya untuk Rupiah Digital, masih langka.

Atas dasar literatur dan artefak publik yang ditinjau, celah penelitian berada pada kebutuhan akan purwarupa CBDC ritel yang (i) mengintegrasikan penerbitan berjenjang, kepatuhan, transfer ritel, serta pengawasan dalam satu sistem, (ii) selaras dengan rancangan dua jenjang Bank Indonesia, dan (iii) dievaluasi melalui pengujian Hyperledger Caliper yang dapat diulang dengan konfigurasi dan artefak yang dicatat. Penelitian ini merancang dan membangun sistem Rupiah Digital ritel berbasis Hyperledger Fabric yang merepresentasikan ekosistem multipihak: Bank Indonesia, bank validator, penyedia jasa pembayaran (PJP), pedagang, pelanggan ritel, pengawas, serta pengamat dari Otoritas Jasa Keuangan (OJK). Sistem tersebut dilengkapi jalur baca pengawasan dan catatan audit pada buku besar. Evaluasi dengan Hyperledger Caliper mengukur *throughput* dan latensi pada berbagai skenario beban untuk menilai karakteristik kinerjanya dalam ruang lingkup purwarupa.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah pada penelitian ini adalah sebagai berikut.

1. Bagaimana merancang dan membangun sistem Rupiah Digital ritel berbasis Hyperledger Fabric yang mendukung model penerbitan dan distribusi likuiditas berjenjang dari Bank Indonesia menuju bank, penyedia jasa pembayaran, dan pelanggan ritel?
2. Bagaimana menerapkan kontrol kepatuhan pada lapisan *chaincode* berupa kebijakan KYC-tier yang diinspirasi pendekatan berbasis risiko, penurunan tingkatan dompet, limit transaksi, dan pengawasan transaksi?
3. Bagaimana kinerja sistem yang dibangun dari sisi *throughput* dan latensi pada berbagai skenario beban transaksi yang diukur menggunakan Hyperledger Caliper?

1.3 Tujuan Penelitian

Tujuan yang ingin dicapai pada penelitian ini adalah sebagai berikut.

1. Merancang dan membangun purwarupa sistem Rupiah Digital ritel berbasis

Hyperledger Fabric dengan arsitektur multipihak yang mendukung penerbitan dan distribusi likuiditas berjenjang.

2. Mengimplementasikan kontrol kepatuhan pada *chaincode* berupa kebijakan KYC-tier yang diinspirasi pendekatan berbasis risiko, penurunan tingkatan dompet, limit transaksi masuk dan keluar, serta pengawasan transaksi.
3. Mengevaluasi kinerja sistem dari sisi *throughput* dan latensi pada berbagai skenario beban menggunakan Hyperledger Caliper.

1.4 Batasan Penelitian

1. Sistem yang dibangun merupakan purwarupa (*prototype*) yang dijalankan pada lingkungan jaringan tersimulasi secara lokal menggunakan *container*, bukan sistem produksi.
2. Jaringan *blockchain* menggunakan Hyperledger Fabric versi 2.5 dengan lima organisasi peserta dan mekanisme konsensus *ordering* berbasis Raft (etcdraft).
3. Penelitian berfokus pada perancangan, implementasi, dan evaluasi kinerja fungsional; aspek hukum, kebijakan moneter, dan dampak makroekonomi dari penerapan Rupiah Digital berada di luar cakupan. Analisis strategi kebijakan moneter dan dampak makroekonomi CBDC, termasuk peran *CBDC rate*, didelegasikan pada studi terpisah seperti [3].
4. Evaluasi kinerja dilakukan menggunakan Hyperledger Caliper dengan beban yang dihasilkan oleh sejumlah *worker* pada lingkungan pengujian lokal.
5. Pengujian dijalankan pada satu inang (*single host*) dengan satu, dua, dan empat *worker* Caliper dan satu *peer* per organisasi, tanpa latensi jaringan dan persebaran geografis yang nyata. Karena itu, angka *throughput* dan latensi yang diperoleh tidak dapat langsung digeneralisasi ke perangkat keras maupun topologi produksi [31,32].
6. Komposisi beban transaksi dan distribusi nilai yang digunakan merupakan asumsi desain yang menyerupai pola pembayaran ritel di Indonesia, bukan jejak transaksi empiris. Pada setiap *worker*, konfigurasi transfer mempertahankan 116 pelanggan, 16 pedagang, dan 52 slot transaksi pada seluruh konfigurasi *worker*. Ronde pemanasan berlangsung 30 detik, sedangkan ronde terukur berlangsung 120 detik.
7. Evaluasi transfer dasar menggunakan skenario perebutan kunci rendah (*low-contention*) dengan rencana transaksi berputar dan populasi tetap. Skenario terpisah menguji perebutan satu dompet melalui permintaan transfer agregat yang melebihi saldo awal. Penelitian tidak menggunakan jejak transaksi empiris untuk menetapkan tingkat konsentrasi maupun kebijakan *retry/backoff* yang representatif. Hasilnya karena itu tidak mewakili beban pedagang populer atau kapasitas

produksi.

8. Purwarupa ini tidak mengimplementasikan pembayaran CBDC tanpa koneksi jaringan. Kemampuan tersebut memerlukan desain perangkat aman, verifikasi kriptografis, pencegahan *double-spending*, dan rekonsiliasi setelah konektivitas kembali [33, 34]. Hasil penelitian ini hanya merepresentasikan transaksi yang diselesaikan melalui koneksi ke *ledger*.
9. Proses KYC disimulasikan secara terprogram tanpa integrasi penyedia identitas nyata. Tingkatan BASIC, STANDARD, dan MERCHANT beserta limitnya merupakan parameter purwarupa yang diinspirasi pendekatan berbasis risiko, bukan ketentuan BI/OJK/FATF atau bukti kepatuhan APU/PPT. Sistem tidak mencakup pemeriksaan sanksi/PEP, *beneficial ownership*, *ongoing due diligence*, pelaporan transaksi mencurigakan, atau integrasi PPATK.
10. Sistem memisahkan data identitas mentah di PostgreSQL dari jangkar dan status KYC pada buku besar, tetapi belum menyediakan privasi transaksi. Topologi memakai satu *application channel* tanpa *private data collection* atau teknik kriptografi penjaga privasi. Akibatnya, metadata dompet, KYC, dan graf transaksi terlihat oleh peserta kanal yang berwenang. Purwarupa ini belum menangani kompromi antara privasi dan keterlacakan [23, 35, 36].
11. Topologi hanya memiliki satu *consenter* Raft sehingga tidak menoleransi kegagalan *orderer*. Evaluasi juga tidak mencakup kompromi simpul, ketersediaan tinggi, RTO/RPO, rotasi atau revokasi identitas, peningkatan versi, respons insiden, dan pemulihan bencana. Kesiapan produksi serta dampak operasional dan inklusi keuangan tidak dinilai.
12. Proyek Garuda menetapkan tiga prinsip 3i sebagai prasyarat desain Rupiah Digital: integrasi dengan infrastruktur pasar keuangan (IPK) domestik, interoperabilitas teknis dan semantik dengan sistem yang ada maupun yang akan datang, serta interkoneksi lintas batas untuk jaringan CBDC antarnegara [5]. Purwarupa ini memenuhi prinsip tersebut hanya pada tataran internal: alur penerbitan–distribusi–pembayaran–pengawasan berjalan secara terpadu dalam satu jaringan Hyperledger Fabric. Namun, integrasi eksternal dengan BI-RTGS dan BI-FAST hanya dimodelkan sebagai titik masuk (*stub*) tanpa koneksi nyata, Penyimpan Digital Rupiah (KDR) belum diimplementasikan, konformitas terhadap standar SNAP API nasional tidak diterapkan, dan tidak ada gerbang (*gateway*) menuju IPK domestik maupun jaringan CBDC lintas batas. Pemenuhan prinsip 3i secara penuh merupakan pekerjaan lanjutan yang berada di luar cakupan purwarupa ini.

1.5 Manfaat Penelitian

Penelitian ini diharapkan memberikan manfaat sebagai berikut.

1. Memberikan rujukan teoretis tentang perancangan CBDC ritel berbasis *permissioned blockchain* yang mencakup penerbitan berjenjang, distribusi melalui PJP, transaksi antarpengguna, dan pengawasan oleh bank sentral.
2. Menjelaskan penerapan Hyperledger Fabric untuk memodelkan pemisahan peran antara BI, peserta distribusi, dan pengguna ritel tanpa menempatkan data identitas mentah pada buku besar.
3. Menyediakan purwarupa yang dapat digunakan sebagai bahan eksperimen teknis untuk menguji alur penerbitan, transfer, pembatasan dompet, kontrol kepatuhan, dan pencatatan peristiwa pengawasan.
4. Memberikan dasar evaluasi kinerja awal melalui pengujian beban terbatas sehingga karakteristik *throughput*, latensi, dan keberhasilan transaksi dapat dibahas secara terukur dalam ruang lingkup purwarupa.
5. Mendukung diskusi teknis pengembangan Rupiah Digital ritel dengan menunjukkan batas kemampuan purwarupa, termasuk kebutuhan integrasi identitas nyata, perluasan topologi, dan pengujian keamanan lanjutan, tanpa menyimpulkan dampak inklusi atau kesiapan produksi.

1.6 Sistematika Penulisan

Penulisan skripsi ini disusun ke dalam lima bab dengan sistematika sebagai berikut.

BAB I PENDAHULUAN berisi latar belakang, rumusan masalah, tujuan penelitian, batasan penelitian, manfaat penelitian, dan sistematika penulisan.

BAB II TINJAUAN PUSTAKA DAN DASAR TEORI memaparkan tinjauan terhadap penelitian terdahulu yang relevan serta dasar teori yang mendasari penelitian, meliputi CBDC, *blockchain* dan DLT, Hyperledger Fabric, Hyperledger Caliper, KYC berbasis risiko, dan limit transaksi.

BAB III METODE PENELITIAN menguraikan alat dan bahan, metode perancangan, metode implementasi, metode pengujian, serta alur penelitian yang ditempuh.

BAB IV HASIL DAN PEMBAHASAN menyajikan hasil implementasi sistem, hasil evaluasi kinerja menggunakan Hyperledger Caliper, serta perbandingan dengan penelitian terdahulu.

BAB V KESIMPULAN DAN SARAN memuat kesimpulan yang menjawab rumusan masalah serta saran untuk pengembangan penelitian selanjutnya.

BAB II

TINJAUAN PUSTAKA DAN DASAR TEORI

2.1 Tinjauan Pustaka

Kajian penerapan *blockchain* untuk mata uang digital bank sentral mencakup beragam fokus. Bagian ini meninjau penelitian terdahulu yang relevan dengan rancang bangun sistem Rupiah Digital ritel berbasis Hyperledger Fabric, meliputi arsitektur, kepatuhan, transfer ritel, pengawasan, dan evaluasi kinerja.

2.1.1 Penelitian Terdahulu tentang Desain dan Keamanan CBDC

Sethaput dan Innet [15] menyusun *systematization of knowledge* (SoK) mengenai aplikasi *blockchain* pada CBDC dan mengidentifikasi bahwa pilihan antara *token-based* dan *account-based*, model satu jenjang versus dua jenjang, serta tingkat privasi merupakan dimensi desain yang paling menentukan. Bhawana dan Kumar [16] membangun jaringan CBDC berbasis *permissioned blockchain* dan menegaskan bahwa keanggotaan terbatas memberikan kendali tata kelola yang dibutuhkan bank sentral tanpa mengorbankan transparansi. Sun dkk. [26] mengusulkan model *multi-blockchain* untuk memisahkan beban transaksi antarwilayah, sedangkan Islam dan In [22] menggunakan *consortium blockchain* dengan model *unspent transaction output* (UTXO) untuk menjaga privasi sekaligus auditabilitas.

Pada ranah kepatuhan, Kakebayashi [37] menekankan pentingnya penanaman aturan *Anti-Money Laundering* dan *Combating the Financing of Terrorism* (AML/CFT) langsung pada desain CBDC ritel. Kajian lain merumuskan model audit CBDC berbasis *blockchain* dari pengalaman *Digital Currency Electronic Payment* (DCEP) Tiongkok [38]. Hans dkk. [25] menerapkan pemodelan ancaman STRIDE untuk mengidentifikasi risiko keamanan pada sistem CBDC berbasis *blockchain*, menegaskan perlunya analisis keamanan sejak tahap perancangan. Block-PAD [39] menawarkan kerangka transaksi CBDC yang tangguh dan fleksibel dengan memanfaatkan identitas digital.

Pada sisi ketahanan layanan, Project Polaris dari *BIS Innovation Hub* menunjukkan bahwa dukungan pembayaran tanpa konektivitas memerlukan desain perangkat aman dan kontrol kriptografis khusus [33, 34]; kemampuan ini diposisikan sebagai keterbatasan dan arah pengembangan lanjutan pada penelitian ini.

2.1.2 Penelitian Terdahulu Berbasis Hyperledger Fabric

Pada aspek platform dan kinerja, Androulaki dkk. [17] memperkenalkan Hyperledger Fabric sebagai *distributed operating system* untuk *permissioned blockchain*

dengan arsitektur *execute-order-validate*. Evaluasi kinerja Fabric mencakup pemodelan latensi [40], evaluasi kerangka berbasis Fabric [41], serta perbandingan *throughput* antara *blockchain permissioned* dan *permissionless* [42]. Literatur tersebut menjadi dasar pemilihan Fabric dan perancangan pengujian Caliper, tetapi angka kinerjanya tidak dibandingkan langsung karena versi, topologi, dan beban kerja berbeda.

2.1.3 Posisi dan Celah Penelitian

Penelitian terdahulu dapat dikelompokkan menurut fokusnya. Arsitektur dan tata kelola jaringan dibahas dalam [16, 26], privasi dan auditabilitas dalam [22–24], serta keamanan dan kepatuhan dalam [25, 37, 38]. Kelompok lain membahas kinerja Hyperledger Fabric [17, 32, 40–42] dan desain CBDC untuk konteks Indonesia [8, 27, 28]. Setiap kelompok menjawab sebagian persoalan. Dalam korpus yang ditinjau pada penelitian ini, belum ditemukan evaluasi satu artefak Fabric ritel yang memadukan seluruh komponen tersebut.

Celah penelitian. Sintesis terbatas pada sumber yang dirujuk di bab ini menunjukkan empat kebutuhan yang belum dipenuhi secara bersamaan. (1) Satu artefak perlu mengintegrasikan penerbitan dan distribusi likuiditas berjenjang, kepatuhan KYC dan limit berbasis tingkatan dompet, transfer antarpelanggan dan pelanggan-ke-pedagang, serta pengawasan. (2) Artefak tersebut perlu selaras dengan rancangan dua jenjang Rupiah Digital Bank Indonesia karena dokumentasi publik Proyek Garuda baru menjangkau lapisan *wholesale* [12]. (3) Tolok ukur kinerja yang dapat diulang untuk CBDC ritel di atas *permissioned blockchain* masih langka pada sistem yang telah diterapkan maupun pada sistem percontohan [29–31]. (4) Rancangan konseptual perlu diwujudkan sebagai demonstrasi yang dapat diuji secara empiris. Penelitian ini menempati irisan keempat hal tersebut. Tabel 2.1 merangkum posisi penelitian ini terhadap penelitian terdahulu.

Kebaruan. Dalam batas korpus yang ditinjau, kontribusi penelitian ini terletak pada integrasi alur distribusi dua jenjang, kebijakan KYC-tier dan limit purwarupa, transaksi ritel, pengawasan, serta protokol Caliper dalam satu artefak Fabric yang dapat diperiksa dan diulang. Kontribusi ini bukan klaim kebaruan arsitektur CBDC secara universal maupun klaim bahwa platform yang digunakan Proyek Garuda adalah Fabric [12, 29]. Repositori artefak berada di <https://github.com/mfachrizalg/bi-coin>; manifes setiap batch benchmark mencatat commit dan hash masukan agar konfigurasi hasil dapat dilacak.

Tabel 2.1. Perbandingan penelitian terdahulu dengan penelitian ini

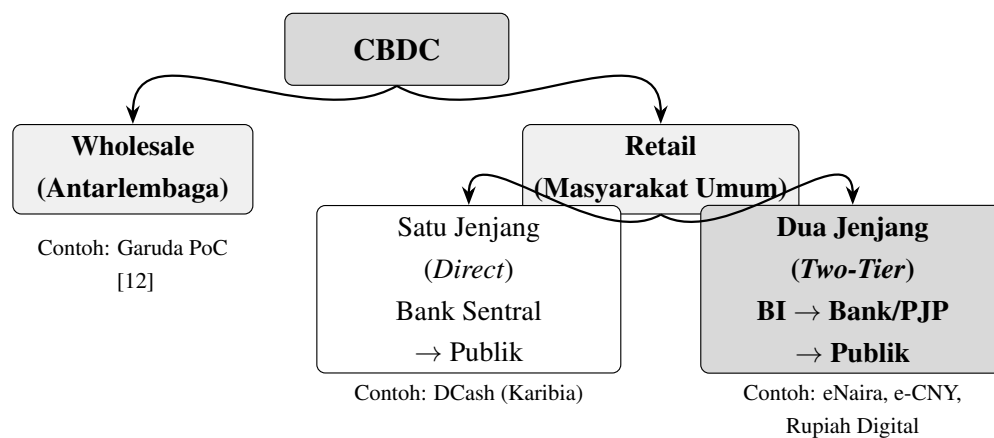
Penelitian	Platform	Fokus utama	Integrasi fitur ritel	Tolok ukur kinerja
Bhawana & Kumar [16]	Permissioned	Arsitektur CBDC	Tidak	Terbatas
Islam & In [22]	Consortium	Privasi (UTXO)	Tidak	Tidak
Hans dkk. [25]	Umum	Keamanan (STRIDE)	Tidak	Tidak
Auer & Böhme; Tsareva & Komarov [6, 21]	Agnostik	Taksonomi desain	Tidak (konsep.)	Tidak
PayOff [43]	Blockchain	Pembayaran tanpa konektivitas	Sebagian	Tidak
Block-PAD [39]	Blockchain	Identitas digital	Sebagian	Terbatas
Lovejoy dkk. (OpenCBDC) [31]	Account-based (non-DLT)	Kinerja tinggi	Sebagian	Ya (non-blockchain)
eNaira [29]	Hyperledger Fabric	CBDC ritel terdeploy	Ya	Tidak publik
Penelitian ini	Hyperledger Fabric	CBDC ritel multipihak	Ya	Caliper

2.2 Dasar Teori

2.2.1 Central Bank Digital Currency

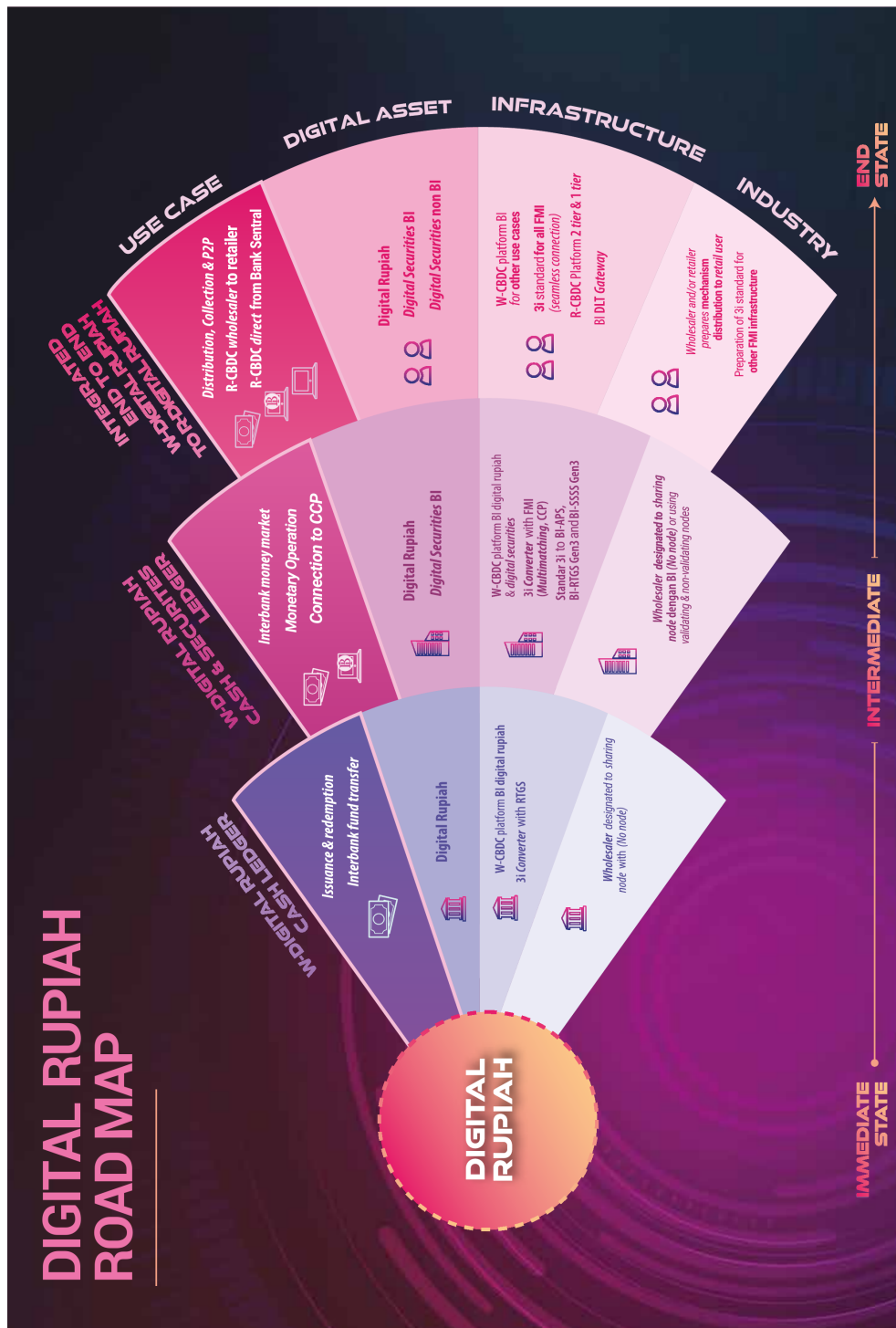
Central Bank Digital Currency (CBDC) adalah representasi digital dari mata uang suatu negara yang diterbitkan oleh dan menjadi kewajiban langsung bank sentral [11]. Sebagai kewajiban bank sentral, CBDC memiliki kualitas bebas risiko yang setara dengan uang tunai, berbeda dengan uang elektronik yang merupakan kewajiban penerbit swasta. Secara umum CBDC dibedakan menjadi dua jenis berdasarkan pengguna Sasarannya, yaitu CBDC *wholesale* yang ditujukan untuk transaksi antarlembaga keuangan, dan CBDC *retail* yang ditujukan untuk masyarakat luas [3, 44].

Dalam penerapannya, sebagian besar bank sentral memilih model dua jenjang (*two-tier*), yaitu bank sentral menerbitkan CBDC dan mengelola *ledger* pada tingkat *wholesale*, sementara distribusi kepada masyarakat dilakukan melalui perantara seperti bank dan penyedia jasa pembayaran. Model ini menjaga peran intermediasi lembaga keuangan dan mengurangi risiko *disintermediation* pada sistem perbankan [15]. Di Indonesia, Bank Indonesia mengembangkan CBDC dengan nama Rupiah Digital melalui Proyek Garuda, yang menempuh pendekatan bertahap mulai dari Rupiah Digital *wholesale* kemudian diperluas menuju Rupiah Digital *retail* [5]. Desain Rupiah Digital menekankan interoperabilitas dengan infrastruktur sistem pembayaran dan setelmen, serta penerapan model dua jenjang dengan pihak perantara [12, 13]. Taksonomi CBDC berdasarkan segmen pengguna dan model distribusi ditunjukkan pada Gambar 2.1.



Gambar 2.1. Taksonomi CBDC berdasarkan segmen pengguna dan model distribusi; model dua jenjang yang diadopsi pada penelitian ini ditampilkan dengan latar abu-abu sesuai rancangan Rupiah Digital Bank Indonesia

Peta jalan resmi Proyek Garuda menempatkan pengembangan Rupiah Digital dalam tiga fase, yaitu *immediate state* untuk w-Digital Rupiah berbasis *cash ledger*, *intermediate state* untuk w-Digital Rupiah dengan kas dan sekuritas digital, serta *end state* yang mengintegrasikan w-Digital Rupiah dengan r-Digital Rupiah. Pada fase *end state*, use case r-Digital Rupiah mencakup distribusi, penagihan, dan pembayaran P2P, baik melalui perantara maupun langsung dari bank sentral sebagaimana ditunjukkan pada Gambar 2.2 [5].

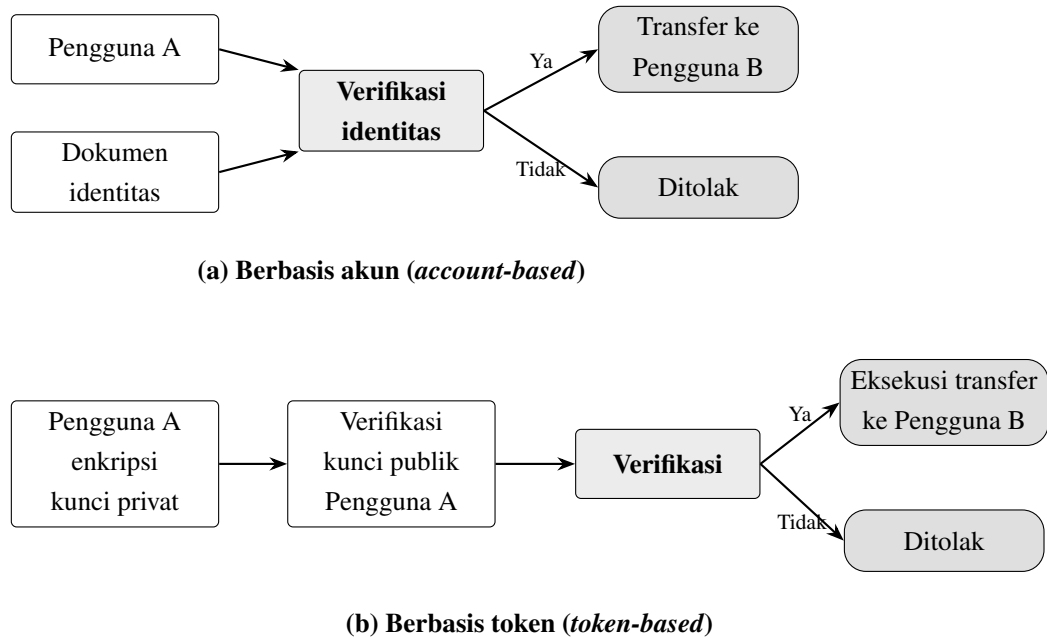


Gambar 2.2. Peta jalan pengembangan Rupiah Digital dalam Proyek Garuda. Sumber: Bank Indonesia [5].

2.2.1.1 Model Pembayaran Berbasis Akun dan Token

Selain dibedakan berdasarkan segmen pengguna dan model distribusi, CBDC juga dapat diklasifikasikan berdasarkan mekanisme pembayaran berbasis akun (*account-based*) atau berbasis token (*token-based*). Pada model berbasis akun, akses pembayaran bergantung pada hubungan antara identitas pengguna, akun atau dompet,

dan catatan saldo yang dikelola sistem. Pada model berbasis token, validitas pembayaran terutama bergantung pada verifikasi objek token atau instrumen nilai yang dipindahkan, sehingga penekanan desain berada pada keaslian token dan hak pemegangnya untuk membelanjakan token tersebut [6,45]. Perbandingan alur verifikasi kedua model ditunjukkan pada Gambar 2.3.



Gambar 2.3. Perbandingan model pembayaran CBDC berbasis akun dan berbasis token: model akun memverifikasi identitas pemegang, sedangkan model token memverifikasi keaslian objek nilai. Diadaptasi dari Auer dan Böhme [6]

White Paper CBDC Bank Indonesia menempatkan Rupiah Digital dalam konfigurasi desain yang mencakup pemisahan w-Digital Rupiah dan r-Digital Rupiah, pilihan satu jenjang atau dua jenjang, akses berbasis token dan akun, serta infrastruktur DLT *permissioned* sebagaimana ditunjukkan pada Gambar 2.4. Konfigurasi tersebut menjadi dasar pembatasan ruang lingkup penelitian ini: purwarupa difokuskan pada model ritel dua jenjang berbasis dompet akun di atas jaringan *permissioned blockchain*, bukan implementasi penuh seluruh konfigurasi Rupiah Digital [5].

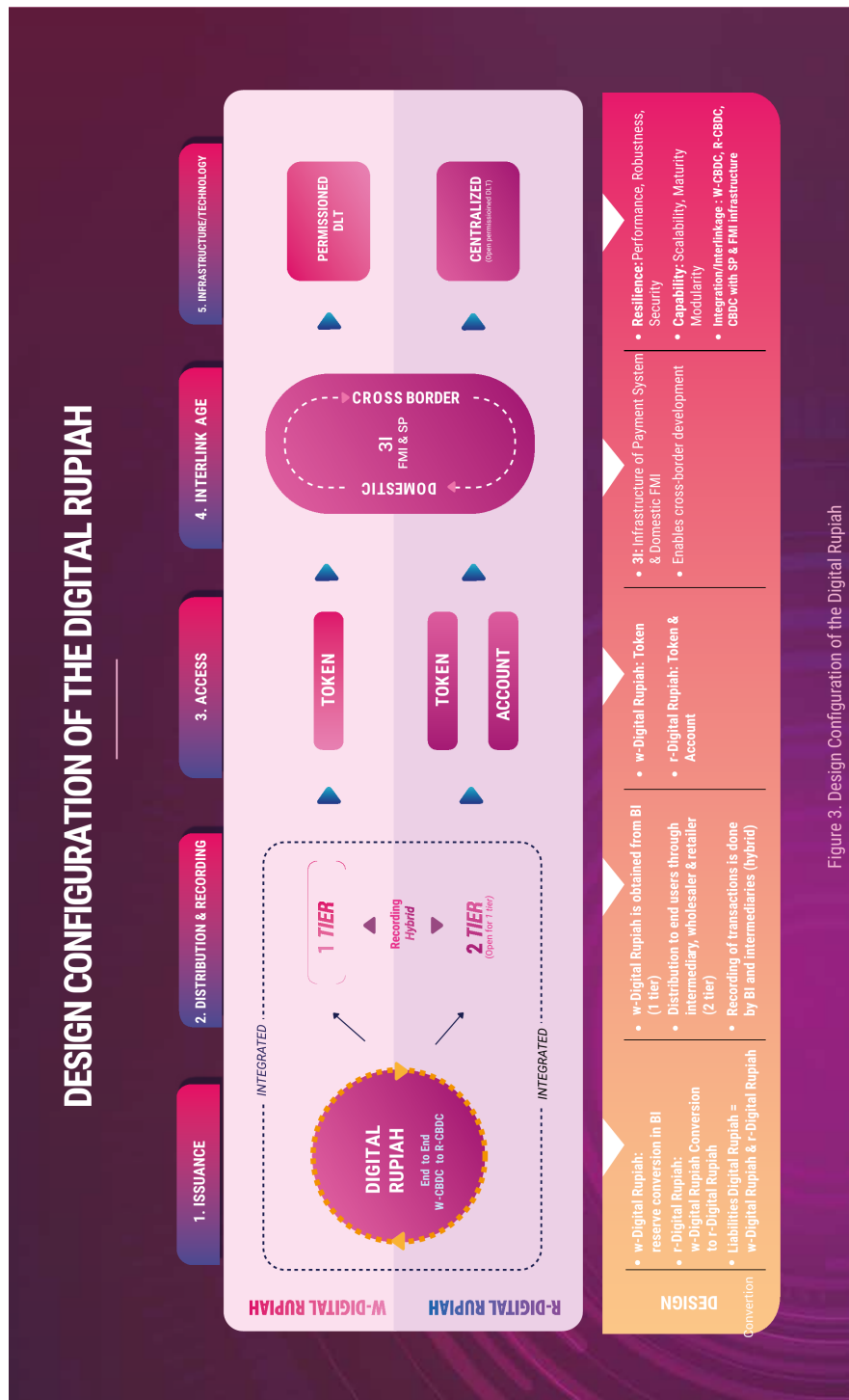


Figure 3. Design Configuration of the Digital Rupiah

Gambar 2.4. Konfigurasi desain Rupiah Digital menurut White Paper CBDC Bank Indonesia. Sumber: Bank Indonesia [5].

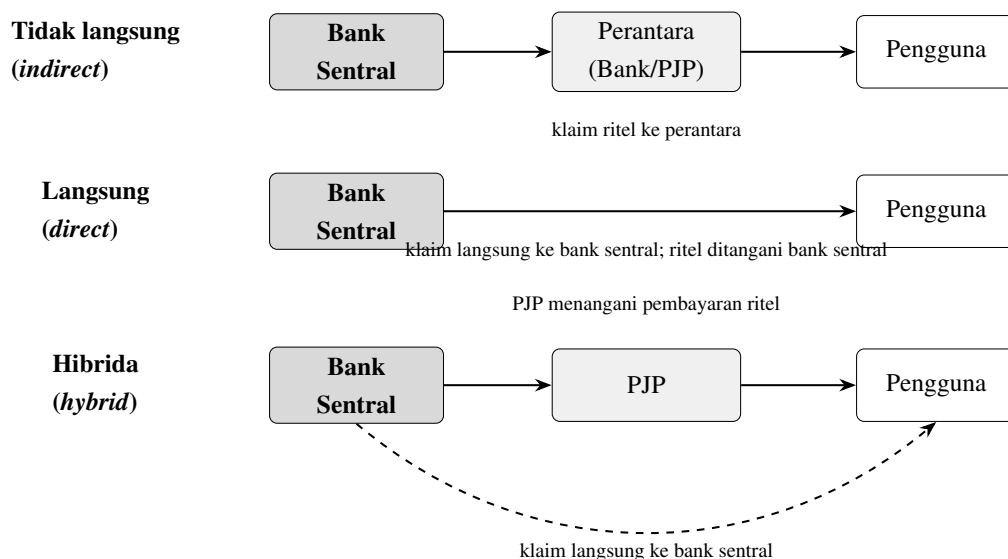
Pembedaan ini relevan karena memengaruhi desain KYC, AML/CFT, privasi, limit saldo, dan auditabilitas transaksi. Syarifuddin dan Bakhtiar menempatkan *account-based* dan *token-based payment system* sebagai salah satu dimensi desain CBDC yang perlu dipertimbangkan dalam strategi kebijakan moneter [3]. Dalam konteks Indonesia, dokumen konsultatif Rupiah Digital menyatakan bahwa akses pada

wholesale Digital Rupiah menggunakan verifikasi berbasis token karena dianggap sesuai untuk transaksi pasar keuangan yang lebih kompleks [13]. Namun, dokumen tersebut menempatkan Rupiah Digital ritel sebagai bagian dari tahap integrasi *end-state*; karena itu, penelitian ini tidak mengklaim bahwa desain pembayaran Rupiah Digital ritel Bank Indonesia telah final sebagai berbasis akun ataupun berbasis token.

Beberapa implementasi CBDC ritel menggunakan pendekatan dompet bertingkat yang dekat dengan model akun atau hibrida. eNaira menerapkan dompet pengguna dengan tingkat KYC dan batas transaksi [29, 46], sedangkan e-CNY menggunakan kategori dompet bertingkat dengan kekuatan identifikasi dan batas saldo/transaksi yang berbeda [47]. Preseden tersebut menunjukkan bahwa model dompet berbasis identitas dan limit merupakan pendekatan yang umum digunakan untuk mengendalikan risiko kepatuhan pada CBDC ritel, meskipun arsitektur teknis setiap yurisdiksi tidak identik.

2.2.1.2 Arsitektur Penerbitan CBDC

Selain model distribusi satu atau dua jenjang, penerbitan CBDC dikategorikan menurut struktur klaim menjadi tiga arsitektur, yaitu langsung (*direct*), tidak langsung (*indirect*), dan hibrida (*hybrid*) [6]. Pada arsitektur langsung, bank sentral menangani layanan pembayaran ritel secara penuh; pada arsitektur tidak langsung, klaim ritel berada pada perantara yang menjamin CBDC; sedangkan pada arsitektur hibrida, CBDC tetap menjadi klaim langsung bank sentral tetapi perantara mengelola interaksi dengan pengguna. Syarifuddin dan Bakhtiar menempatkan pilihan arsitektur ini sebagai salah satu dimensi desain CBDC yang memengaruhi strategi kebijakan moneter [3]. Ketiga arsitektur tersebut dibandingkan pada Gambar 2.5.

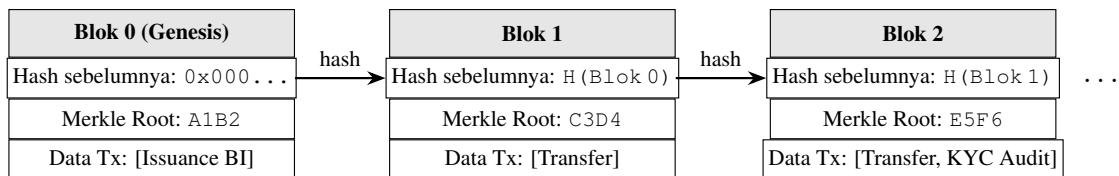


Gambar 2.5. Perbandingan arsitektur penerbitan CBDC: tidak langsung (*indirect*/dua jenjang), langsung (*direct*/satu jenjang), dan hibrida (*hybrid*). Diadaptasi dari Auer dan Böhme [6]

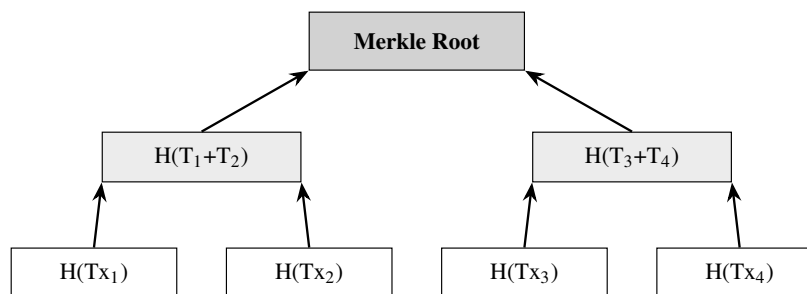
2.2.2 Blockchain dan Distributed Ledger Technology

2.2.2.1 Struktur Blockchain

Distributed Ledger Technology (DLT) adalah teknologi pencatatan terdistribusi yang memungkinkan banyak pihak memelihara salinan basis data yang konsisten tanpa otoritas terpusat tunggal. *Blockchain* merupakan salah satu bentuk DLT yang menyusun transaksi ke dalam blok-blok yang saling terhubung secara kriptografis. Setiap blok memuat *hash* dari blok sebelumnya sehingga membentuk riwayat tambah-saja yang perubahan retrospektifnya dapat dideteksi; perubahan pada satu blok akan merusak keterkaitan *hash* pada seluruh blok berikutnya [14]. Sifat ini mendukung integritas dan ketertelusuran data selama mekanisme konsensus dan kendali akses tetap berfungsi. Struktur rantai blok diilustrasikan pada Gambar 2.6. Setiap blok juga memuat *Merkle root*, yaitu nilai *hash* yang dihasilkan dari pohon Merkle seluruh transaksi dalam blok tersebut, sehingga integritas setiap transaksi dapat diverifikasi secara efisien tanpa harus membaca semua data transaksi [14]. Struktur pohon Merkle ditunjukkan pada Gambar 2.7.



Gambar 2.6. Struktur rantai blok: setiap blok memuat *hash* kriptografis dari blok sebelumnya

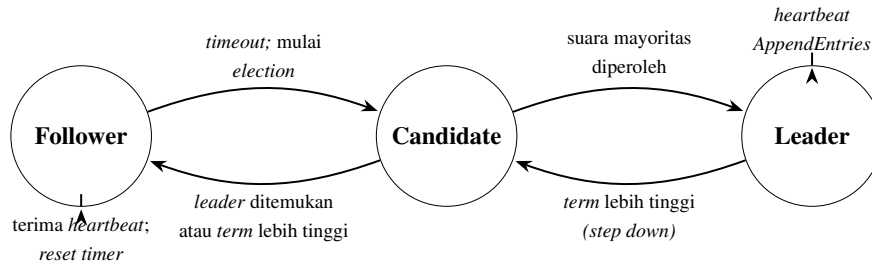


Gambar 2.7. Pohon Merkle: *hash* transaksi digabungkan secara berjenjang membentuk Merkle Root yang disimpan dalam kepala blok

2.2.2.2 Algoritme Konsensus

Algoritme konsensus menjamin seluruh simpul pada jaringan menyepakati satu urutan transaksi yang sama. Pada *blockchain* publik digunakan mekanisme seperti *proof-of-work* (PoW) dan *proof-of-stake* (PoS) yang dirancang tahan terhadap perilaku jahat namun berbiaya komputasi tinggi. Sebaliknya, *blockchain permissioned* dapat memakai varian *Byzantine Fault Tolerance* (BFT) maupun *Crash Fault Tolerance* (CFT) seperti

Raft. Identitas peserta mendukung autentikasi dan atribusi, tetapi pemilihan BFT atau CFT tetap harus mengikuti jenis kegagalan yang diasumsikan [17, 48]. Algoritme Raft membagi peran simpul menjadi tiga *state*: *Leader*, *Candidate*, dan *Follower*. Diagram transisi *state* Raft ditunjukkan pada Gambar 2.8.



Gambar 2.8. Diagram transisi *state* algoritme konsensus Raft: setiap simpul berstatus *Follower*, *Candidate*, atau *Leader*

2.2.2.3 Tipe Blockchain

Berdasarkan model aksesnya, *blockchain* dapat dikelompokkan menjadi *public* (permissionless), *private*, dan *consortium* atau *permissioned* [16]. Pada *public blockchain*, siapa pun dapat bergabung dan berpartisipasi dalam konsensus, sehingga memerlukan mekanisme seperti *proof-of-work* yang mahal secara komputasi. Sebaliknya, pada *permissioned blockchain*, partisipasi dibatasi pada entitas yang teridentifikasi dan memperoleh izin, sehingga konsensus dapat menggunakan algoritme yang lebih efisien dan menghasilkan *throughput* lebih tinggi. Untuk kebutuhan CBDC, *permissioned blockchain* menjadi pilihan yang sesuai karena bank sentral memerlukan kendali tata kelola, identitas peserta yang jelas, dan kinerja yang memadai [15, 42].

2.2.3 Hyperledger Fabric

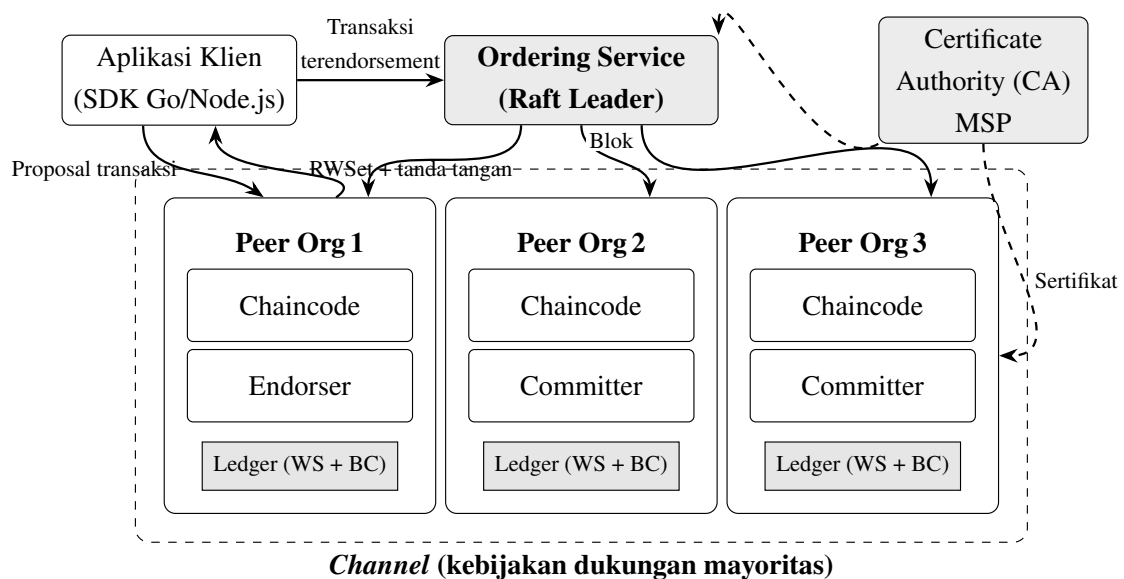
2.2.3.1 Arsitektur Hyperledger Fabric

Hyperledger Fabric adalah kerangka kerja *permissioned blockchain* sumber terbuka di bawah naungan *Hyperledger Foundation* yang dirancang untuk aplikasi perusahaan [17]. Fabric memiliki arsitektur modular dengan komponen utama sebagai berikut.

1. **Peer**, yaitu simpul yang menyimpan salinan *ledger* dan menjalankan *chaincode*. *Peer* berperan sebagai *endorser* yang menyimulasikan transaksi dan sebagai *committer* yang menulis blok ke *ledger*.
2. **Orderer**, yaitu layanan pengurut (*ordering service*) yang menyusun transaksi ke dalam blok dan mendistribusikannya ke *peer*. Fabric mendukung konsensus berbasis Raft (etcdraft) yang bersifat *crash fault tolerant*.

3. **Membership Service Provider (MSP)**, yaitu komponen yang mengelola identitas dan hak akses peserta berdasarkan sertifikat digital X.509 yang diterbitkan oleh *Certificate Authority (CA)*.
4. **Channel**, yaitu mekanisme pemisahan jaringan yang memungkinkan subkelompok peserta bertransaksi secara privat dengan *ledger* tersendiri.
5. **Chaincode**, yaitu program *smart contract* yang memuat logika bisnis dan aturan transaksi. Pada Fabric, *chaincode* dapat ditulis dalam bahasa Go, Java, atau Node.js.

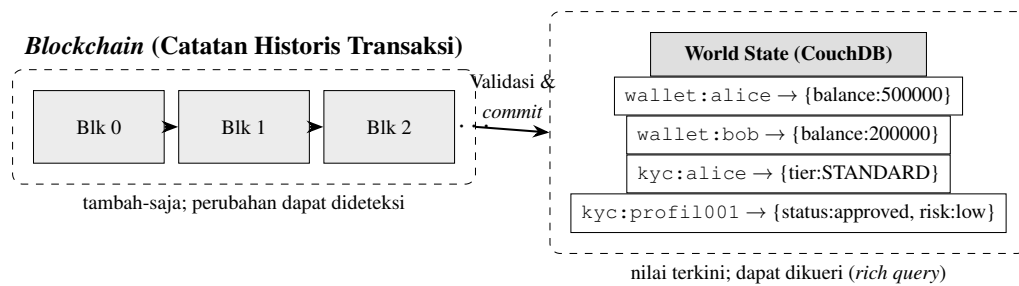
Gambar 2.9 menunjukkan hubungan antara aplikasi klien, *endorsing peer*, *ordering service*, dan *channel* dalam arsitektur Hyperledger Fabric.



Gambar 2.9. Arsitektur komponen Hyperledger Fabric: klien mengirim proposal ke *peer endorser*, mengirim transaksi terendorsement ke *orderer*, lalu menerima komitmen blok pada semua *peer* dalam *channel*.

2.2.3.2 Hyperledger Fabric Ledger

Ledger pada Fabric terdiri atas dua bagian, yaitu *world state* yang menyimpan nilai terkini dari setiap kunci (umumnya menggunakan basis data LevelDB atau CouchDB), dan *blockchain* yang menyimpan riwayat transaksi secara berurutan dalam struktur tambah-saja. Penggunaan CouchDB memungkinkan penyimpanan dokumen JSON serta kueri kaya (*rich query*) [49]. Dualitas kedua komponen *ledger* tersebut diilustrasikan pada Gambar 2.10: *blockchain* berfungsi sebagai catatan historis yang perubahan retrospektifnya dapat dideteksi, sedangkan *world state* menyimpan nilai terkini setiap kunci yang dapat dibaca secara langsung dan efisien oleh *chaincode*.



Gambar 2.10. Dualitas komponen *ledger* Hyperledger Fabric: *blockchain* sebagai catatan transaksi historis tambah-saja yang perubahan retrospektifnya dapat dideteksi, dan *world state* sebagai representasi status terkini pada CouchDB

2.2.3.3 Alur Transaksi Hyperledger Fabric

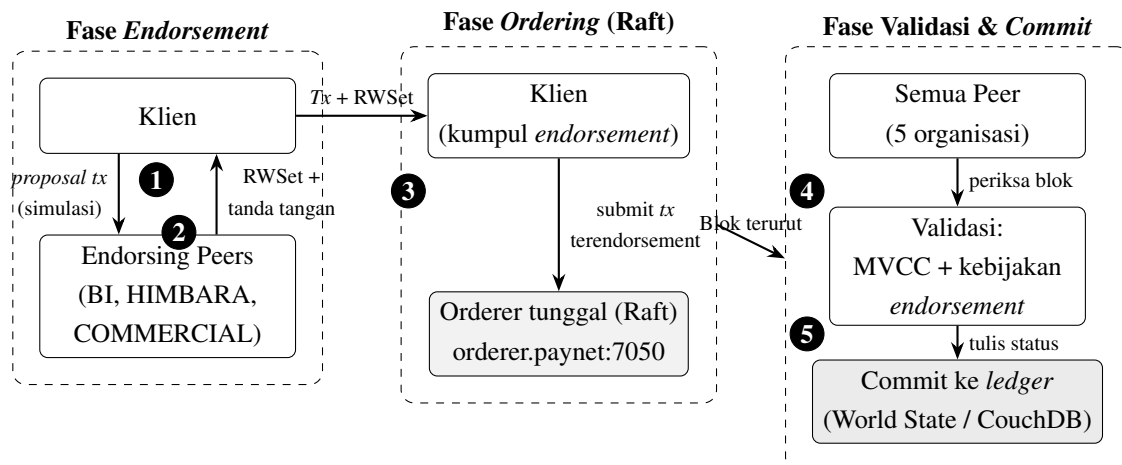
Karakteristik utama Fabric adalah model eksekusi *execute-order-validate* yang membedakannya dari model *order-execute* pada banyak *blockchain* lain. Alur transaksi Fabric berlangsung dalam lima tahap. (1) Aplikasi klien mengirim usulan transaksi (*transaction proposal*) ke sejumlah *peer endorser*. (2) *Endorser* mensimulasikan transaksi terhadap *chaincode* dan mengembalikan tanda tangan dukungan (*endorsement*) beserta himpunan baca-tulis (*read-write set*). (3) Klien mengumpulkan dukungan sesuai kebijakan dukungan (*endorsement policy*), lalu mengirimkannya ke *ordering service*. (4) *Orderer* mengurutkan transaksi ke dalam blok. (5) Setiap *peer* memvalidasi transaksi dengan memeriksa kebijakan dukungan dan konflik versi, kemudian menuliskannya ke *ledger* [17]. Model ini meningkatkan skalabilitas karena eksekusi *chaincode* dilakukan secara paralel sebelum pengurutan.

Pemeriksaan konflik versi pada tahap validasi menerapkan kontrol konkurensi multiversi (*Multiversion Concurrency Control*, MVCC). Setiap transaksi membawa himpunan baca (*read set*) berisi versi kunci status yang dibaca ketika endorsement. Sebelum *commit*, *peer* membandingkan versi tersebut dengan versi terkini pada *ledger*. Apabila sebuah kunci telah berubah oleh transaksi lain dalam blok yang sama atau blok sebelumnya, versi tidak lagi cocok sehingga transaksi ditandai tidak valid sebagai konflik MVCC meskipun endorsementnya sah [17]. Mekanisme ini menjaga konsistensi status pada penulisan bersamaan dan menjadi dasar analisis perebutan dompet (*hot-wallet contention*) pada BAB IV.

Endorsement policy menentukan organisasi mana yang dukungannya diperlukan agar suatu transaksi dianggap sah. Pada jaringan multipihak, kebijakan seperti *majority* mensyaratkan dukungan dari mayoritas organisasi, sehingga tidak ada satu pihak pun yang dapat memutakhirkan *ledger* secara sepihak.

Gambar 2.11 mengilustrasikan siklus transaksi Hyperledger Fabric secara lengkap, mulai dari *proposal* ke *endorsing peer* hingga *commit* ke *ledger* melalui

mekanisme konsensus Raft.



Gambar 2.11. Siklus transaksi Hyperledger Fabric pada topologi uji dengan satu *consenter* Raft tanpa toleransi kegagalan *orderer*.

2.2.4 Hyperledger Caliper

Hyperledger Caliper adalah kerangka kerja *benchmark* kinerja *blockchain* yang memungkinkan pengukuran kinerja terhadap sistem *blockchain* dengan beban kerja yang dapat dikustomisasi [1]. Caliper menghasilkan beban transaksi melalui sejumlah proses pekerja (*worker*) dan mengukur metrik kinerja utama berdasarkan metrik standar *Hyperledger Performance and Scale Working Group* [50], antara lain:

1. **Throughput**, yaitu jumlah transaksi yang berhasil diproses per satuan waktu, dinyatakan dalam *transactions per second* (TPS).
2. **Latency**, yaitu selang waktu antara pengiriman transaksi hingga transaksi tersebut tercatat dan terkonfirmasi pada *ledger*.
3. **Send rate**, yaitu laju pengiriman transaksi yang dikendalikan oleh *rate controller*.

Caliper juga dapat memantau penggunaan sumber daya *container* seperti penggunaan CPU dan memori selama pengujian, sehingga sesuai untuk mengevaluasi kelayakan kinerja sistem CBDC [32].

2.2.5 Kepatuhan: KYC, Tingkatan Dompot, dan Limit Transaksi

Know Your Customer (KYC) adalah proses verifikasi identitas nasabah yang menjadi syarat kepatuhan pada sistem keuangan untuk mencegah pencucian uang dan pendanaan terorisme. FATF menempatkan tingkat keandalan identitas digital dan risiko nasabah sebagai dasar penentuan intensitas uji tuntas, sedangkan risiko lebih tinggi memerlukan *enhanced due diligence* (EDD) [51]. Pendekatan berbasis risiko juga menjadi prinsip dalam ketentuan APU, PPT, dan PPPSPM Bank Indonesia [52].

Penerapan tingkatan dompet pada CBDC memiliki preseden pada e-CNY dan eNaira. Dokumen desain e-CNY menjelaskan dompet bertingkat dengan kekuatan identifikasi, batas saldo, dan batas pembayaran yang berbeda [47]; rancangan eNaira juga membedakan batas berdasarkan tingkat verifikasi pengguna [46]. Literatur ekonomi menunjukkan bahwa batas kepemilikan dan penggunaan CBDC merupakan instrumen desain untuk menyeimbangkan kegunaan, intermediasi perbankan, dan risiko sistemik [53]. Bukti ini mendukung klasifikasi berbasis risiko, tetapi tidak menentukan angka limit yang universal untuk Rupiah Digital.

Purwarupa memisahkan tiga dimensi yang sering tercampur. *Subject type* membedakan pelanggan ritel, pedagang, dan peserta institusional. *Due diligence level* mencatat uji tuntas *simplified*, *standard*, atau *enhanced*. *Risk level* mencatat risiko rendah, menengah, tinggi, atau terlarang. Kombinasi ketiganya menghasilkan tingkatan BASIC, STANDARD, atau MERCHANT. Risiko tinggi tidak otomatis ditolak, tetapi hanya dapat disetujui setelah EDD dan status persetujuan senior disuplai oleh penyedia KYC; purwarupa belum membuktikan alur *four-eyes* atau identitas pemberi persetujuan. Transaksi selanjutnya menghasilkan peristiwa pengawasan tanpa data identitas mentah.

Angka limit penelitian merupakan parameter kebijakan purwarupa. Ketentuan uang elektronik Bank Indonesia digunakan sebagai proksi domestik, bukan sebagai dasar hukum limit CBDC [54]. Nilai dipilih konservatif dan eksplisit agar aturan dapat diuji: semakin ringan uji tuntas, semakin kecil eksposur saldo dan arus dana. Validasi mencakup saldo maksimum, batas per transaksi, pengeluaran harian, pengeluaran bulanan, serta penerimaan bulanan. Fitur tersebut adalah purwarupa KYC-tier dan limit yang diinspirasi pendekatan berbasis risiko, bukan implementasi lengkap program APU/PPT.

2.2.6 Transfer Ritel Langsung

Unit pembayaran pada purwarupa adalah fungsi *Transfer* yang sama untuk transfer antarpelanggan dan pelanggan-ke-pedagang. Penyatuan jalur ini menjaga aturan KYC, penghitung limit, pemeriksaan saldo penerima, dan pencatatan audit tetap deterministik. Penelitian tidak memodelkan standar instrumen pembayaran atau proses akseptasi eksternal; fokusnya adalah kebijakan buku besar CBDC ritel.

2.2.7 Application Programming Interface

Application Programming Interface (API) adalah antarmuka yang memungkinkan dua perangkat lunak saling berkomunikasi melalui kontrak yang terdefinisi. Sistem yang dibangun menggunakan *REST API* (*Representational State Transfer*) [55] sebagai antarmuka komunikasi menuju lapisan *backend* melalui protokol HTTP. API ini menjembatani permintaan pengguna atau alat uji ke *chaincode* melalui

Fabric Gateway SDK tanpa menjadikan antarmuka pengguna sebagai bagian kontribusi penelitian.

2.2.8 Bahasa Pemrograman Go

Go adalah bahasa pemrograman terkompilasi yang dikembangkan oleh Google dengan penekanan pada kesederhanaan dan dukungan konkurensi melalui *goroutine* [56]. Pada penelitian ini Go digunakan untuk menulis *chaincode* dan *backend* karena dukungannya yang kuat terhadap Hyperledger Fabric melalui *Fabric Contract API* dan *Fabric Gateway SDK*, serta kinerja konkurensi yang baik untuk melayani banyak permintaan secara bersamaan.

2.3 Analisis Perbandingan Metode

Empat keputusan desain utama penelitian ini dibandingkan secara sistematis: pilihan antara DLT dan sistem terpusat, pemilihan platform DLT, pemilihan algoritme konsensus, dan pemilihan model arsitektur CBDC. Tabel perbandingan memperlihatkan perbedaan antar-alternatif secara eksplisit sebagai dasar pertanggungjawaban ilmiah.

2.3.1 Perbandingan Hyperledger Fabric dengan Kerangka Kerja Blockchain Lain

DLT versus Sistem Terpusat. Tabel 2.2 membandingkan tiga pendekatan infrastruktur untuk sistem pembayaran CBDC: sistem terpusat tradisional (non-DLT), *permissionless blockchain* (publik), dan *permissioned blockchain* (DLT berizin). Sistem terpusat menawarkan *throughput* sangat tinggi tetapi bergantung pada otoritas tunggal dan tidak memiliki auditabilitas multipartit secara inheren. *Permissionless blockchain* menyediakan transparansi tetapi *throughput*-nya rendah akibat mekanisme konsensus berbasis *proof-of-work* [15, 42]. DLT *permissioned* menggabungkan auditabilitas multipartit, kendali tata kelola yang dapat dikonfigurasi, dan *throughput* tinggi tanpa konsumsi energi besar [17].

Tabel 2.2. Perbandingan sistem terpusat, *permissionless blockchain*, dan *permissioned blockchain* untuk CBDC

Kriteria	Sistem Terpusat (non-DLT)	Permissionless Blockchain	Permissioned Blockchain
Kekebalan data	Tidak (admin dapat ubah)	Ya	Ya
Auditabilitas multipartit	Terbatas (otoritas tunggal)	Ya (publik)	Ya (terkontrol)
Kendali tata kelola	Penuh terpusat	Tidak ada	Dapat dikonfigurasi
<i>Throughput</i>	>50.000 TPS (indikatif)	7–50 TPS (indikatif)	>3.500 TPS [17]
Konsumsi energi	Rendah	Sangat tinggi (PoW)	Rendah
<i>Smart contract</i>	Opsional	Ya	Ya
Kapabilitas pemisahan data	Ya	Tidak (publik)	Tersedia melalui <i>channel</i> /data privat
Kepatuhan regulasi	Mudah	Sulit	Mudah
Contoh artefak CBDC yang dilaporkan	OpenCBDC [31]	Tidak direkomen dasikan	eNaira [29]

Simpulan: *Permissioned blockchain* dipilih karena menyeimbangkan auditabilitas multipartit yang tidak dapat diperoleh dari sistem terpusat, kendali tata kelola bank sentral yang tidak dimiliki *permissionless blockchain*, dan *throughput* tinggi yang dibutuhkan sistem pembayaran [15,42].

Perbandingan Platform DLT. Tabel 2.3 membandingkan empat platform DLT *permissioned* yang lazim digunakan dalam sistem keuangan dan CBDC: Hyperledger Fabric, R3 Corda, Quorum/Hyperledger Besu, dan Hyperledger Sawtooth [15, 17, 57].

Tabel 2.3. Perbandingan platform DLT *permissioned* untuk sistem CBDC

Kriteria	Hyperledger Fabric	R3 Corda	Quorum/ Besu	H. Sawtooth
Konsensus	Raft (CFT), <i>pluggable</i>	Notary (CFT)	IBFT/Clique	PoET, Raft
Model eksekusi	<i>Execute-order-validate</i>	<i>Flow-based</i>	<i>Order-execute</i>	<i>Order-execute</i>
Bahasa <i>smart contract</i>	Go, Java, Node.js	Kotlin, Java	Solidity (EVM)	Python, Go, Java
Manajemen identitas	MSP + X.509 CA	X.509 PKI	Berbasis akun	Berbasis peran
Privasi	<i>Channel, private data</i>	<i>Need-to-know</i>	TX privat	Terbatas
Throughput	>3.500 TPS [17]	~170 TPS [57]	~300 TPS (indikatif)	~1.000 TPS (indikatif)
Penerapan CBDC yang dilaporkan	eNaira [29]	PoC Proyek Garuda [12]	PoC Proyek Garuda [12]	Tidak

Simpulan: Hyperledger Fabric dipilih karena identitas organisasi berbasis MSP, model eksekusi modular, dukungan kontrak Go, dan integrasi langsung dengan alat uji Caliper. MSP mengautentikasi organisasi jaringan, bukan menggantikan KYC nasabah. *Channel* dan data privat merupakan kapabilitas platform, tetapi purwarupa penelitian memakai satu *application channel* dan tidak memisahkan buku besar *wholesale* dan ritel. PoC Proyek Garuda menggunakan Corda dan Hyperledger Besu; laporan itu menjadi pembandingan kebutuhan Indonesia, bukan bukti penerapan Fabric [12].

Sebagai pembandingan eksternal, Urbančok membandingkan kinerja transfer aset pada Ethereum, Hyperledger Fabric, dan Besu menggunakan skenario *send rate* 10, 20, 50, dan 100 TPS [7]. Hasil pada Gambar 2.12 menunjukkan bahwa Fabric kompetitif untuk beban transfer aset dan unggul pada beberapa titik uji, tetapi angka tersebut tidak digunakan sebagai hasil evaluasi purwarupa penelitian ini karena konfigurasi jaringan, versi perangkat lunak, dan beban kerja berbeda.

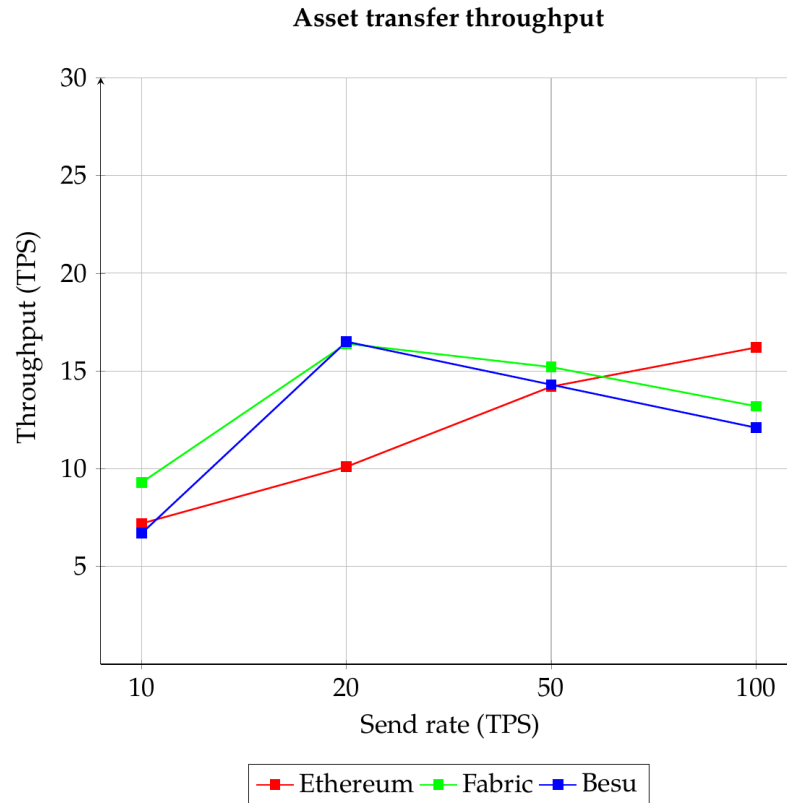


Figure 8.6: Throughput comparison of asset transfer

Gambar 2.12. Perbandingan *throughput* transfer aset pada Ethereum, Hyperledger Fabric, dan Besu. Sumber: [7].

2.3.2 Pemilihan Algoritme Konsensus Hyperledger Fabric

Tabel 2.4 membandingkan algoritme konsensus yang relevan untuk jaringan *blockchain permissioned*. Identitas peserta mendukung autentikasi dan atribusi tindakan, tetapi tidak menghilangkan perilaku *Byzantine* dari simpul yang terkompromi. Finalisasi deterministik tetap diperlukan karena sistem keuangan tidak dapat mentoleransi ketidakpastian *fork* [17, 48].

Tabel 2.4. Perbandingan algoritme konsensus untuk *blockchain permissioned*

Algoritme	Toleransi kesalahan	Kompleksitas pesan	Throughput	Latensi	Finalisasi
Raft	CFT	$O(n)$	Tinggi	Rendah	Deterministik
PBFT	BFT	$O(n^2)$	Menengah	Menengah	Deterministik
HotStuff	BFT	$O(n)$	Menengah	Rendah	Deterministik
PoW	—	$O(n)$	Sangat rendah	Sangat tinggi	Probabilistik
PoS/PoA	BFT/CFT	$O(n)$	Menengah	Menengah	Semi-deterministik

Simpulan: Raft (*etcdraft*) dipilih karena didukung secara *native* oleh Hyperledger Fabric dan memberi finalisasi deterministik dalam model *crash fault* [17]. Identitas MSP menyediakan autentikasi dan atribusi, tetapi tidak mencegah perilaku *Byzantine* dari simpul yang terkompromi. Selain itu, purwarupa hanya memakai satu *consenter*; konfigurasi ini tidak menoleransi satu pun kegagalan *orderer*. Penelitian karena itu mengevaluasi pilihan *ordering* purwarupa, bukan mendemonstrasikan toleransi kegagalan.

2.3.3 Pemilihan Alat Pengujian Blockchain

Hyperledger Caliper dan BlockBench sama-sama dapat mengevaluasi jaringan *blockchain permissioned*, tetapi memiliki cakupan berbeda. Caliper menjalankan beban kerja yang dikonfigurasi terhadap kontrak aktual dan melaporkan metrik tingkat sistem, sedangkan BlockBench dirancang untuk membandingkan kinerja keseluruhan dan komponen internal beberapa platform [1, 2]. Perbandingan ringkas disajikan pada Tabel 2.5.

Tabel 2.5. Perbandingan Hyperledger Caliper dan BlockBench berdasarkan [1, 2].

Kriteria	Hyperledger Caliper	BlockBench
Fokus	Pengujian jaringan dengan beban kerja dan kontrak yang dapat dikonfigurasi	Perbandingan lintas platform pada tingkat makro dan mikro
Metrik utama	Tingkat keberhasilan, <i>throughput</i> , latensi, dan penggunaan sumber daya	<i>Throughput</i> , latensi, skalabilitas, dan toleransi kegagalan
Beban kerja	Modul beban kerja yang memanggil fungsi kontrak aktual	Kontrak pintar nyata dan sintetis melalui API benchmark
Kesesuaian	Integrasi langsung dengan jaringan Fabric dan <i>chaincode</i> penelitian	Tepat untuk analisis komponen dan perbandingan beberapa platform

Simpulan: Hyperledger Caliper dipilih karena tujuan penelitian adalah mengukur perilaku *chaincode* CBDC ritel aktual pada satu jaringan Fabric, bukan membandingkan implementasi internal beberapa platform.

2.3.4 Pemilihan Arsitektur CBDC Dua Jenjang

Istilah “dua jenjang” pada bagian ini menyatakan pembagian peran distribusi antara bank sentral dan perantara. Istilah tersebut berbeda dari “tingkatan dompet

KYC”, yang menyatakan klasifikasi kebijakan pengguna ritel. Peserta institusional tidak menerima tingkatan BASIC, STANDARD, atau MERCHANT.

Tabel 2.6 membandingkan model arsitektur satu jenjang (*one-tier/direct*), di mana bank sentral mendistribusikan CBDC langsung kepada publik, dengan model dua jenjang (*two-tier*), di mana distribusi dilakukan melalui bank dan penyedia jasa pembayaran (PJP) [6, 15, 21].

Tabel 2.6. Perbandingan arsitektur CBDC satu jenjang dan dua jenjang

Kriteria	Satu Jenjang (<i>Direct</i>)	Dua Jenjang (<i>Two-Tier</i>)
Distribusi CBDC	Bank sentral langsung ke publik	Bank sentral → bank/PJP → publik
Risiko <i>disintermediation</i>	Tinggi	Rendah [15]
Beban infrastruktur bank sentral	Sangat tinggi	Dibagi ke perantara
KYC dan <i>onboarding</i>	Dikelola bank sentral	Dikelola bank/PJP
Privasi pengguna dari bank sentral	Bergantung pada desain implementasi	Bergantung pada pemisahan data dan hak akses; bukan sifat otomatis dua jenjang
Interoperabilitas ekosistem	Sulit	Mudah (bank/PJP sudah terintegrasi)
Contoh CBDC nyata	DCash (Karibia)	eNaira, e-CNY, Sand Dollar [29]
Keselarasn Proyek Garuda	Tidak	Ya [5, 12]

Simpulan: Model dua jenjang dipilih berdasarkan empat pertimbangan. (1) Model ini selaras dengan desain Rupiah Digital Bank Indonesia dalam Proyek Garuda [12]. (2) Pembagian peran antarlapis meminimalkan risiko *disintermediation* perbankan yang menjadi kekhawatiran utama bank sentral [15]. (3) Bank dan PJP yang telah berpengalaman mengelola KYC dan *onboarding* nasabah. (4) Volume transaksi *retail* skala masif tidak membebani *ledger wholesale* [6, 21].

2.3.5 Pemetaan Purwarupa terhadap Dimensi Desain CBDC Optimal

Syarifuddin dan Bakhtiar merumuskan desain CBDC optimal pendukung strategi kebijakan moneter ke dalam beberapa dimensi: cakupan ritel dan *wholesale*; remunerasi

berbunga (*wholesale*) dan tanpa bunga (ritel); model pembayaran berbasis akun dan token; derajat anonimitas yang terlacak; arsitektur hibrida; sistem buku besar berbasis DLT; serta cakupan domestik dan lintas batas [3, 28]. Karena penelitian ini berfokus pada perancangan, implementasi, dan evaluasi kinerja fungsional, dimensi yang bersifat makroekonomi dan kebijakan moneter berada di luar cakupan; Tabel 2.7 memetakan pemenuhan tiap dimensi pada purwarupa beserta justifikasinya.

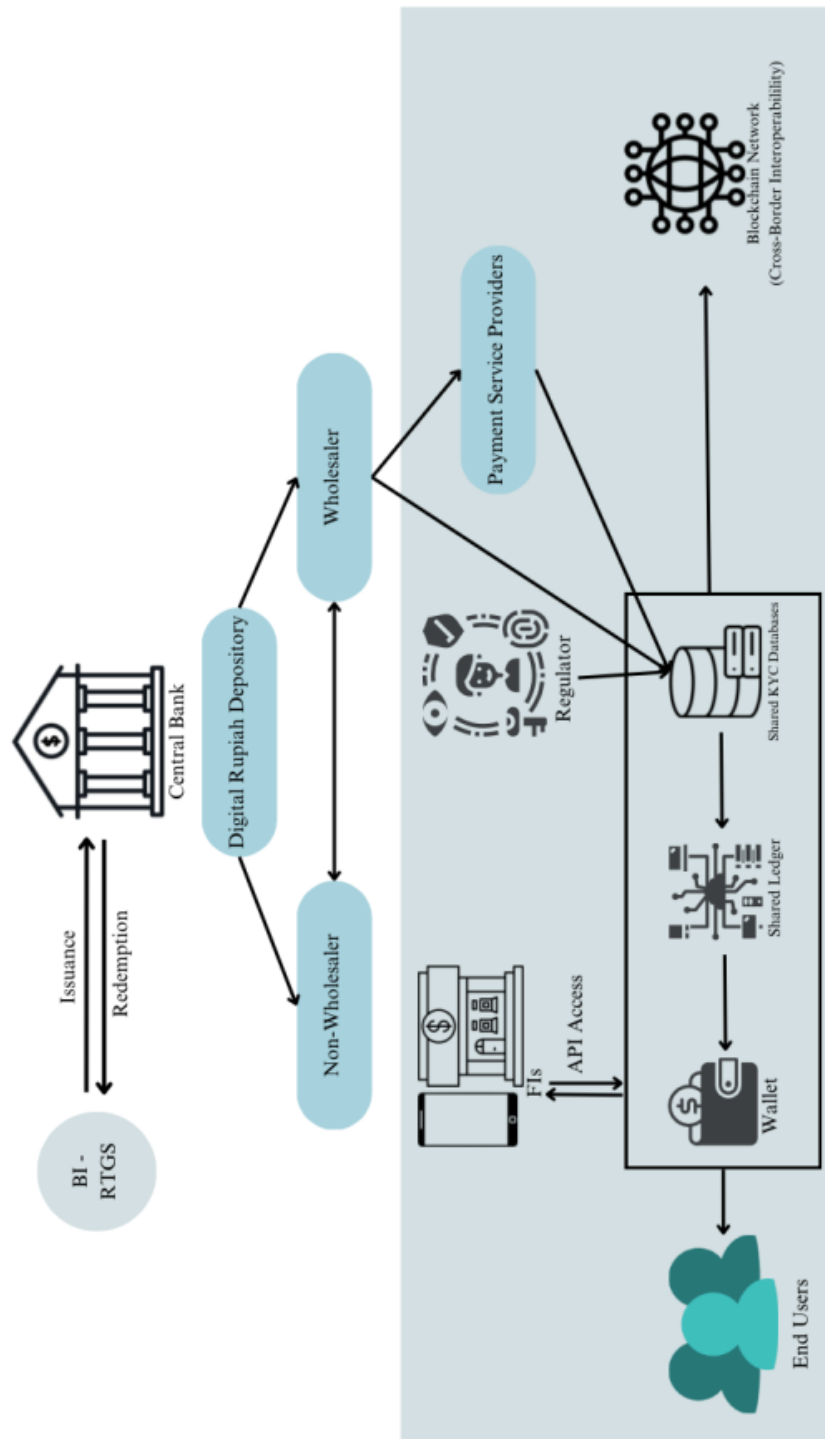
Tabel 2.7. Pemetaan purwarupa terhadap dimensi desain CBDC optimal [3]

Dimensi desain optimal	Status purwarupa	Keterangan
Sistem buku besar DLT	Terpenuhi	Hyperledger Fabric 2.5 sebagai buku besar terdistribusi <i>permissioned</i>
Arsitektur hibrida	Tidak didemonstrasikan	Distribusi dua jenjang diimplementasikan pada satu buku besar bersama; pembagian tanggung jawab pencatatan hibrida tidak diuji
Cakupan ritel dan <i>wholesale</i>	Ritel saja	Dompot institusional memodelkan distribusi likuiditas, bukan instrumen atau buku besar <i>wholesale</i> yang terpisah
Model akun dan token	Sebagian	Implementasi berbasis akun (saldo); dimensi token dibahas namun tidak diklaim final (lihat Subbab Model Pembayaran Berbasis Akun dan Token)
Remunerasi ritel tanpa bunga	Konsisten	Rupiah Digital ritel purwarupa bersifat non-remunerasi, selaras dengan rekomendasi CBDC ritel optimal [3]
Remunerasi <i>wholesale</i> berbunga / <i>CBDC rate</i>	Di luar cakupan	Instrumen kebijakan moneter; tidak dimodelkan pada purwarupa fungsional
Anonimitas terlacak	Di luar cakupan	Privasi transaksi tidak diimplementasikan; lihat batasan privasi pada Bab I
Cakupan domestik dan lintas batas	Sebagian	Domestik; interkoneksi CBDC lintas batas merupakan kesenjangan prinsip 3i

Simpulan: purwarupa mengimplementasikan buku besar DLT, distribusi dua jenjang, fungsi ritel berbasis saldo, dan sifat non-remunerasi. Arsitektur hibrida, instrumen *wholesale*, *CBDC rate*, anonimitas terlacak, serta interkoneksi lintas batas tidak

didemonstrasikan. Pemisahan ini mencegah model distribusi, cakupan instrumen, dan bentuk arsitektur diperlakukan sebagai dimensi yang sama [3, 28].

Aryani [8] mengusulkan arsitektur dua jenjang Digital Rupiah dengan lapisan tambahan berupa *Digital Rupiah Depository* sebagai entitas perantara antara Bank Indonesia dan peserta jaringan, pemisahan *wholesaler* dan *non-wholesaler* di jenjang atas, serta basis data KYC bersama (*shared KYC database*) yang terpisah dari *ledger* transaksi. Arsitektur tersebut juga mencakup koneksi ke BI-RTGS dan jaringan *blockchain* lintas batas untuk interoperabilitas. Arsitektur yang diusulkan Aryani ditunjukkan pada Gambar 2.13.



Gambar 2.13. Arsitektur Rupiah Digital yang diusulkan Aryani [8].

Penelitian ini mengadopsi prinsip inti arsitektur tersebut (distribusi berjenjang melalui bank validator dan PJP, serta KYC berbasis risiko) namun dengan tiga penyederhanaan yang disengaja sesuai cakupan purwarupa. Pertama, lapisan *Digital Rupiah Depository* dan pemisahan *wholesaler/non-wholesaler* diciutkan menjadi dua organisasi validator (*HimbaraBankOrg* dan *CommercialBankOrg*) yang langsung menerima penerbitan dari *BankIndonesiaOrg*. Kedua, data identitas KYC mentah

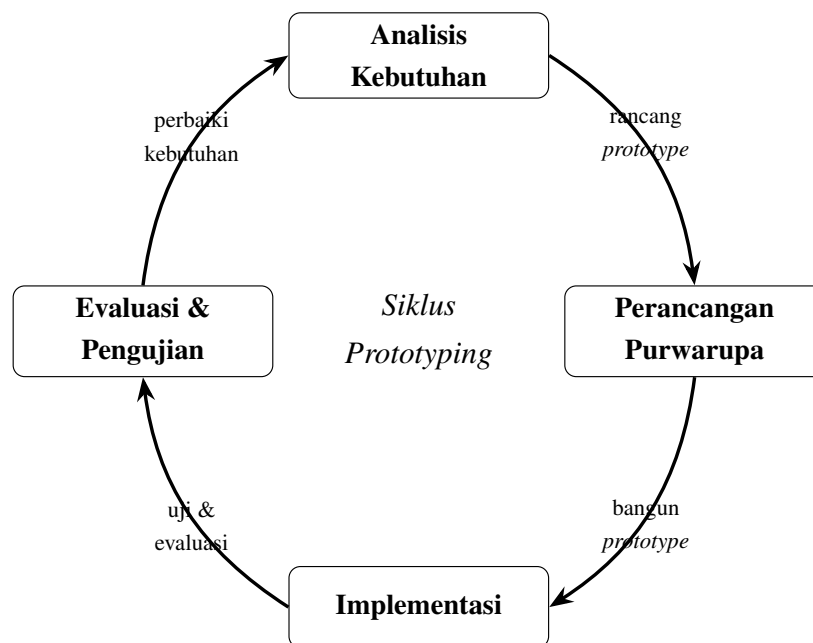
disimpan pada PostgreSQL di luar buku besar, sedangkan *world state* hanya menyimpan status, risiko, tingkat uji tuntas, persetujuan senior, masa berlaku, dan hash dokumen. Pemisahan ini sejalan dengan pola arsitektur hibrid yang dijabarkan BIS, yakni bank sentral mempertahankan salinan jangkar kepatuhan sementara data sensitif dikelola di luar *ledger* oleh perantara yang menjalankan KYC [6]. Pendekatan serupa, yakni penyimpanan identitas KYC pada basis data terpisah dari buku besar, diadopsi pada proyek CBDC ritel Sand Dollar (Bahama) [58] dan uji coba e-Krona (Swedia) [59]. Bank Indonesia dan pengawas memperoleh akses baca terhadap jangkar kepatuhan dan peristiwa pengawasan, bukan kewenangan menjalankan KYC nasabah. Ketiga, integrasi BI-RTGS dan jaringan *blockchain* lintas batas tidak diimplementasikan dan dinyatakan secara eksplisit sebagai batasan cakupan penelitian (Bab I).

BAB III

METODE PENELITIAN

3.1 Metode yang Digunakan

Metode yang digunakan adalah *prototyping* dengan tahapan perancangan, implementasi, dan pengujian. Sistem dibangun sebagai purwarupa fungsional yang kemudian dievaluasi kinerjanya secara kuantitatif menggunakan Hyperledger Caliper. Siklus SDLC metode *prototyping* yang diterapkan ditunjukkan pada Gambar 3.14: keempat fase (analisis kebutuhan, perancangan, implementasi, dan evaluasi) diulang secara iteratif hingga purwarupa memenuhi spesifikasi.



Gambar 3.14. Siklus SDLC dengan metode *prototyping*: empat fase berulang hingga purwarupa memenuhi kebutuhan fungsional dan kinerja

3.1.1 Metode Perancangan

Perancangan sistem dimulai dari analisis kebutuhan berdasarkan model Rupiah Digital dua jenjang, kemudian dituangkan ke dalam activity diagram, arsitektur sistem, arsitektur jaringan, perancangan use case, dan model data *ledger*.

3.1.1.1 Analisis Kebutuhan Sistem

Analisis kebutuhan mengidentifikasi aktor sistem (Bank Indonesia, bank validator, PJP, pedagang, pelanggan ritel, supervisor, dan pengamat OJK) beserta kebutuhan fungsionalnya, yaitu penerbitan dan distribusi likuiditas berjenjang, pengelolaan KYC berbasis risiko, penurunan tingkatan dan limit dompet, transfer

antarpelanggan serta pelanggan-ke-pedagang, pengawasan, dan pelaporan. Kebutuhan ini menjadi dasar perancangan activity diagram dan arsitektur pada subbab berikutnya.

3.1.1.2 Spesifikasi Kebutuhan Perangkat Lunak

Analisis kebutuhan tersebut diformalkan dalam bentuk Spesifikasi Kebutuhan Perangkat Lunak (*Software Requirements Specification/SRS*) yang diadaptasi ke struktur tesis ini. SRS digunakan untuk menetapkan ruang lingkup purwarupa, aktor dan batas wewenangnya, antarmuka utama sistem, kebutuhan fungsional, dan kebutuhan nonfungsional sebelum implementasi dilakukan. Pada penelitian ini, ruang lingkup perangkat lunak mencakup empat antarmuka utama, yaitu aplikasi klien, *backend API*, basis data luring PostgreSQL, dan jaringan *blockchain* Hyperledger Fabric beserta *chaincode*-nya.

Pemangku kepentingan utama terdiri atas Bank Indonesia sebagai penerbit dan pengawas, bank validator sebagai distributor likuiditas ke PJP, PJP sebagai penyalur layanan ritel, pedagang dan pelanggan ritel sebagai pengguna transaksi, serta supervisor dan pengamat OJK sebagai pihak yang membutuhkan fungsi baca dan pelaporan. Setiap pemangku kepentingan tersebut kemudian diterjemahkan menjadi peran akses pada *backend API* dan aturan bisnis pada *chaincode* agar kebutuhan operasional dan kepatuhan dapat ditelusuri secara konsisten.

Tabel 3.1. Kebutuhan fungsional purwarupa bi-coin

ID	Deskripsi kebutuhan
FR-01	Sistem harus mendukung pendaftaran, persetujuan, pembekuan, pencairan, dan penghentian peserta sesuai peran Bank Indonesia, bank validator, dan PJP.
FR-02	Sistem harus mendukung penerbitan Digital Rupiah oleh Bank Indonesia dan distribusi likuiditas dua jenjang dari bank validator ke PJP.
FR-03	Sistem harus mendukung pengajuan dan pembaruan profil KYC secara luring dengan hanya menyimpan jangkar kepatuhan pada <i>ledger</i> .
FR-04	Sistem harus menurunkan tier dompet BASIC, STANDARD, atau MERCHANT secara deterministik dari profil KYC yang disetujui.
FR-05	Sistem harus menolak transfer yang melanggar status KYC, status beku, batas per transaksi, batas keluar harian/bulanan, batas masuk bulanan, atau batas saldo maksimum.
FR-06	Sistem harus menyediakan laporan rekonsiliasi, metrik, riwayat transaksi, dan <i>event</i> pengawasan bagi peran yang berwenang.
FR-07	Sistem harus menegakkan pemisahan hak akses pada <i>backend API</i> antara peran publik, terautentikasi, terverifikasi KYC, PJP/bank, merchant, dan supervisor.
FR-08	Sistem harus menyimpan data autentikasi, data KYC mentah, pemeriksaan penyedia, dan jejak audit pada basis data luring PostgreSQL.

Tabel 3.2. Kebutuhan nonfungsional purwarupa bi-coin

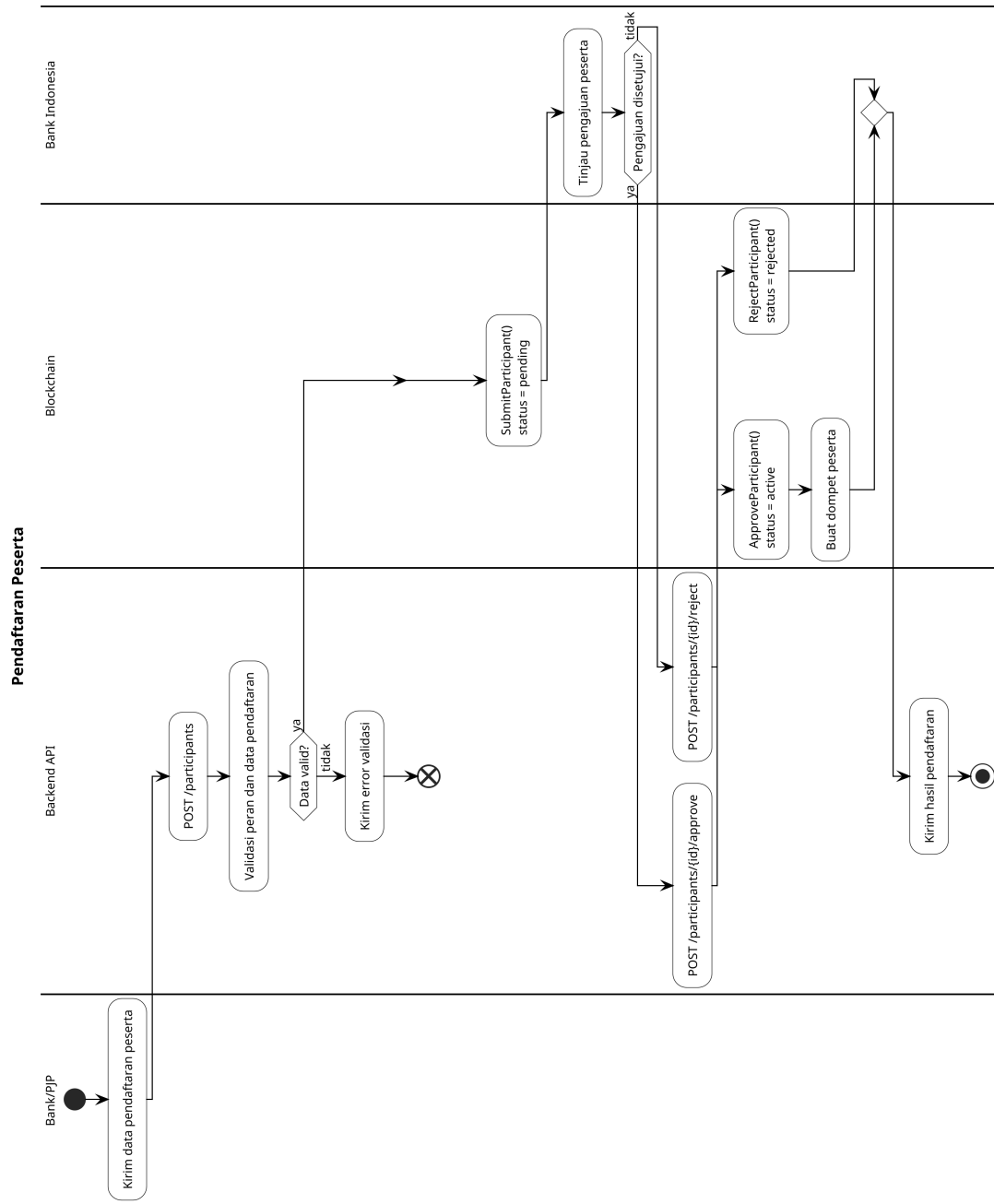
ID	Deskripsi kebutuhan
NFR-01	Sistem harus berjalan pada jaringan <i>permissioned</i> Hyperledger Fabric yang mendukung partisipasi multipihak dan <i>endorsement</i> mayoritas.
NFR-02	Sistem harus menjaga kerahasiaan data identitas dengan menempatkan data KYC mentah di PostgreSQL dan hanya menyimpan <i>hash</i> atau jangkar kepatuhan pada <i>ledger</i> .
NFR-03	Sistem harus menyediakan keterlacakan melalui catatan audit, <i>event</i> pengawasan, dan relasi data luring yang dapat ditelusuri kembali ke subjek KYC atau peserta terkait.
NFR-04	Sistem harus memisahkan antarmuka klien, <i>backend API</i> , basis data luring, dan <i>chaincode</i> agar kontrol akses dan integrasi dapat dikelola secara modular.
NFR-05	Sistem harus dapat dievaluasi kinerjanya secara kuantitatif melalui metrik <i>throughput</i> , latensi, dan tingkat keberhasilan transaksi menggunakan Hyperledger Caliper.
NFR-06	Sistem harus mendukung pengulangan eksperimen pada lingkungan yang sama melalui konfigurasi, <i>seed</i> , hash artefak, topologi jaringan, dan skenario uji yang terdokumentasi.

Tabel 3.3. Keterlacakan kebutuhan terhadap metode verifikasi

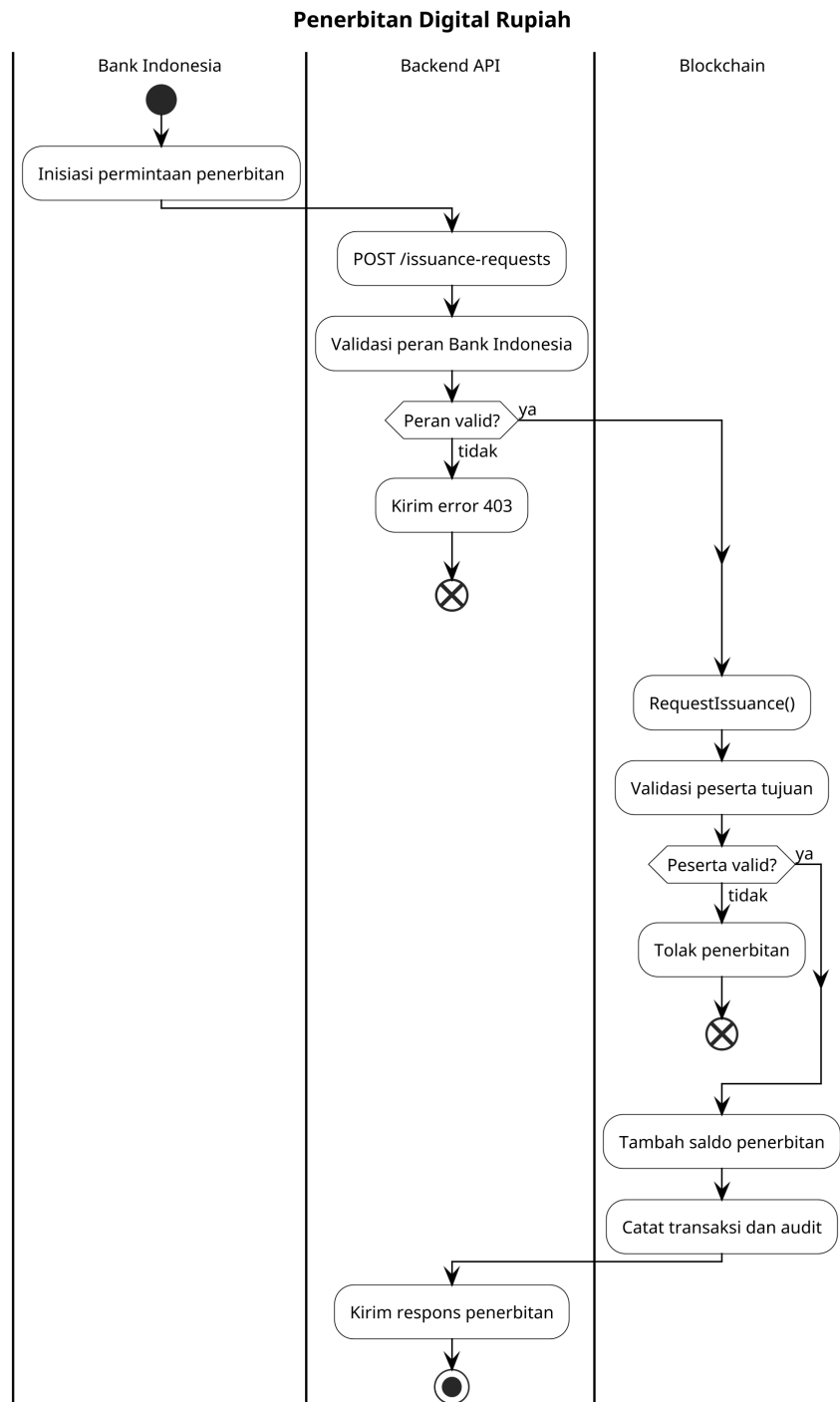
ID kebutuhan	Metode verifikasi utama	Bukti yang dituju
FR-01, FR-02	Skenario fungsional dan <i>unit test chaincode</i>	Siklus hidup peserta, penerbitan, dan distribusi dua jenjang berjalan sesuai aturan
FR-03, FR-08	<i>Unit test backend</i> , inspeksi desain data luring, dan pengujian fungsional KYC	Data KYC mentah tetap luring, sedangkan jangkar kepatuhan dicatat pada <i>ledger</i>
FR-04, FR-05	<i>Unit test chaincode</i> dan skenario fungsional transfer	Penurunan tier, status beku, dan limit transaksi ditegakkan konsisten
FR-06, FR-07	Skenario fungsional, <i>unit test backend</i> , dan inspeksi rute API	Hak akses, pelaporan, dan <i>event</i> pengawasan tersedia bagi peran berwenang
NFR-01, NFR-04, NFR-06	Inspeksi arsitektur, konfigurasi, dan manifes benchmark	Struktur jaringan, pemisahan lapisan, dan <i>repeatability</i> purwarupa terdokumentasi
NFR-02, NFR-03	Inspeksi model data dan <i>unit test</i> terpilih	Kerahasiaan data identitas dan keterlacakan audit dapat dipertahankan
NFR-05	<i>Benchmark</i> Hyperledger Caliper	Nilai <i>throughput</i> , latensi, dan tingkat keberhasilan transaksi diperoleh secara kuantitatif

3.1.1.3 Perancangan Activity Diagram

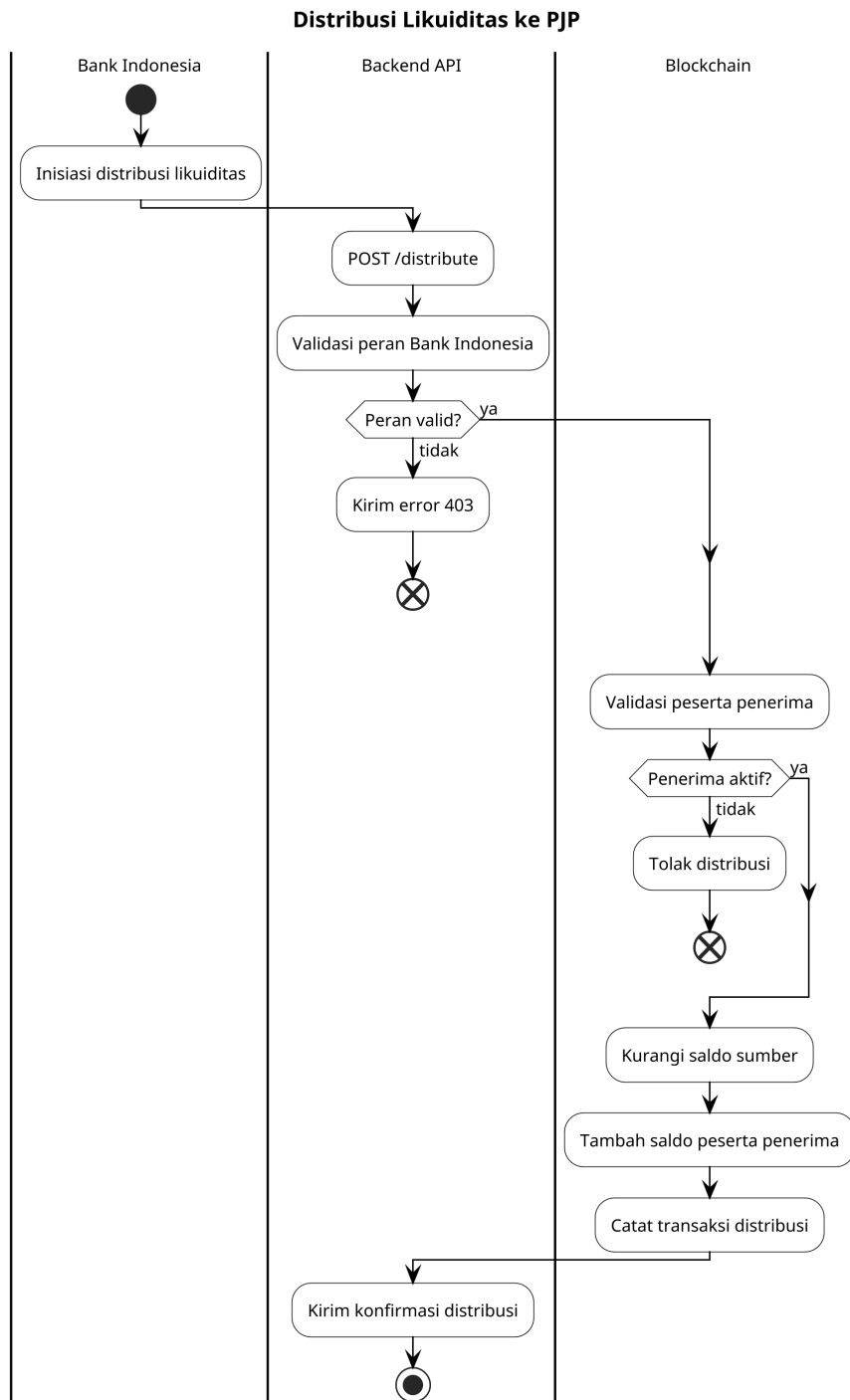
Setiap proses utama dimodelkan dengan activity diagram bergaya *swimlane* tiga jalur, yaitu jalur aktor, jalur *backend API*, dan jalur *blockchain (chaincode)*. Diagram tersebut mencakup pendaftaran peserta (Gambar 3.15), penerbitan Digital Rupiah (Gambar 3.16), distribusi likuiditas ke PJP (Gambar 3.17), dan onboarding KYC pelanggan (Gambar 3.18). Proses lainnya meliputi transfer saldo ritel (Gambar 3.19), pembekuan dan pencairan peserta (Gambar 3.20), serta laporan dan pengawasan (Gambar 3.21).



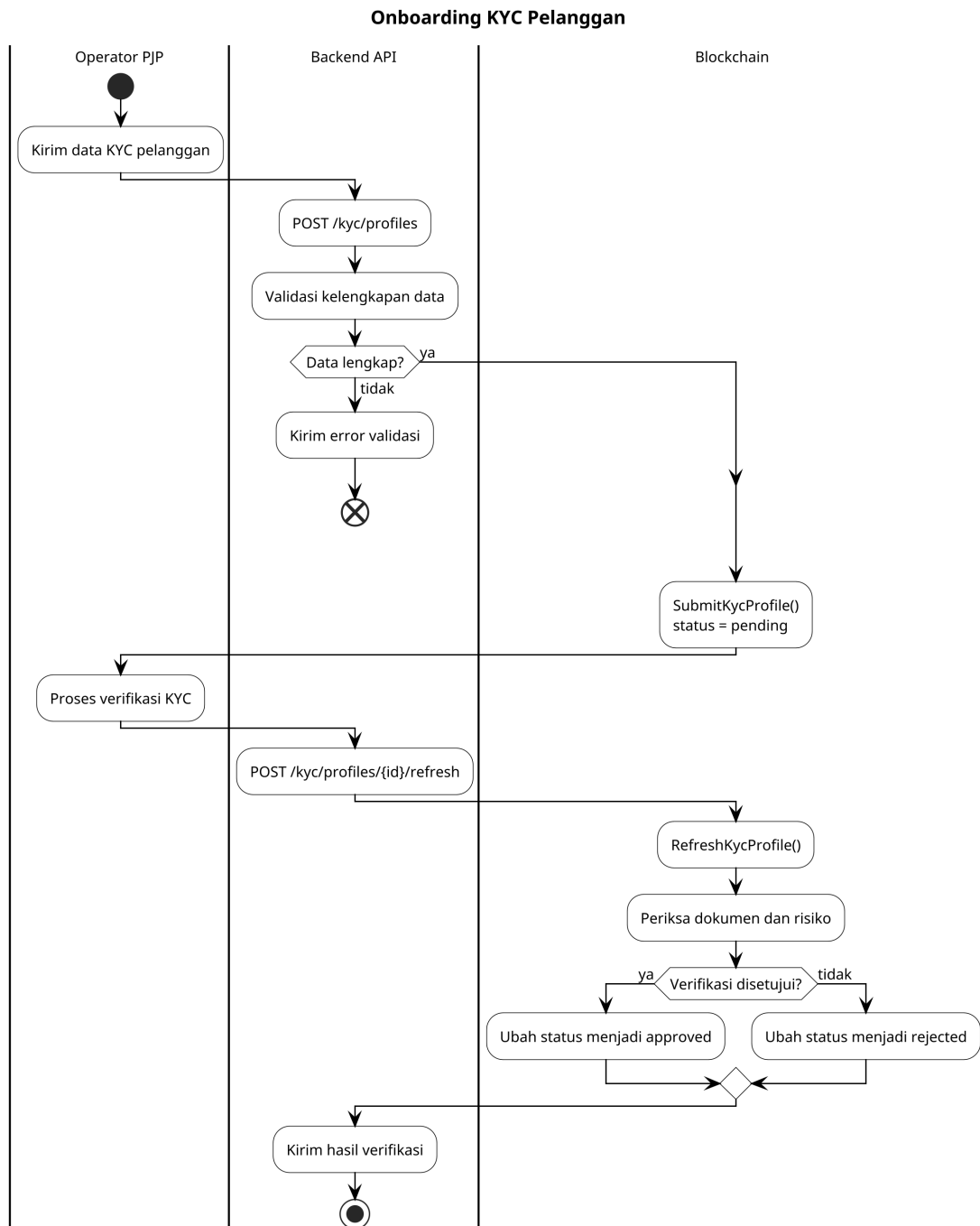
Gambar 3.15. Activity diagram pendaftaran peserta



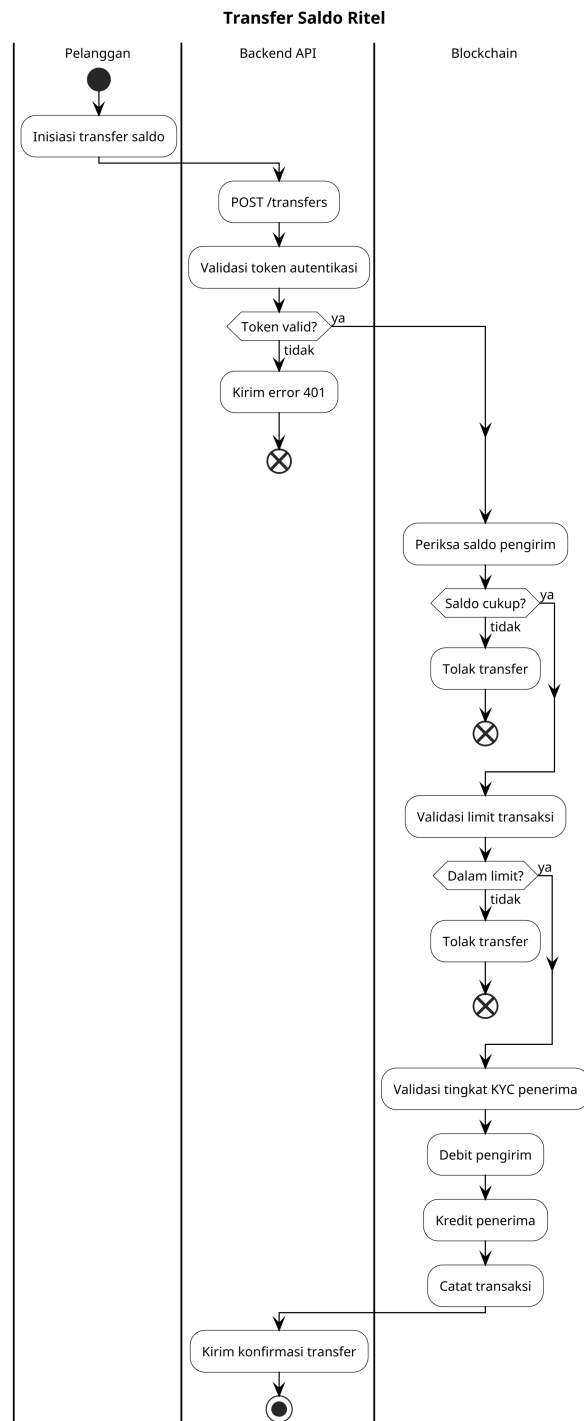
Gambar 3.16. Activity diagram penerbitan Digital Rupiah



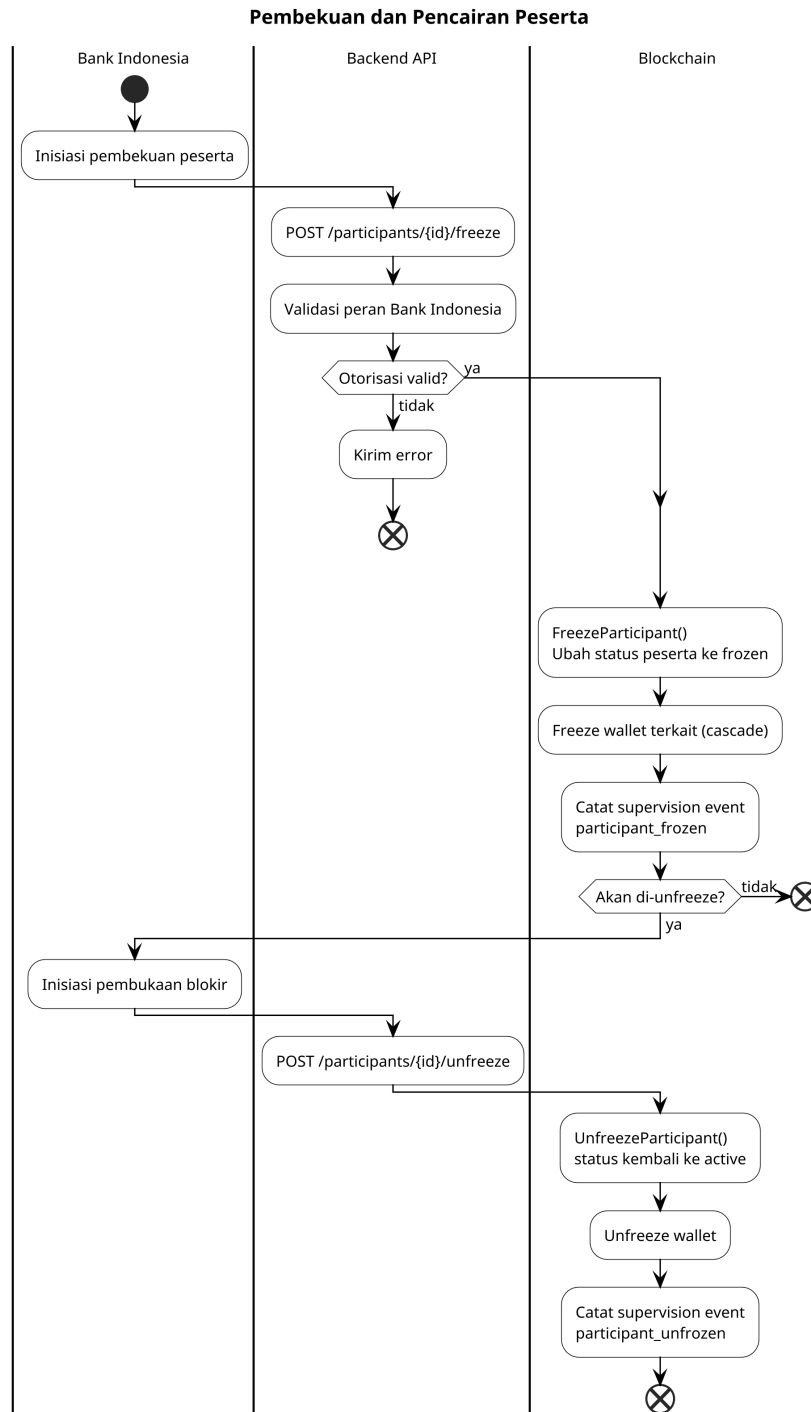
Gambar 3.17. Activity diagram distribusi likuiditas ke PJP



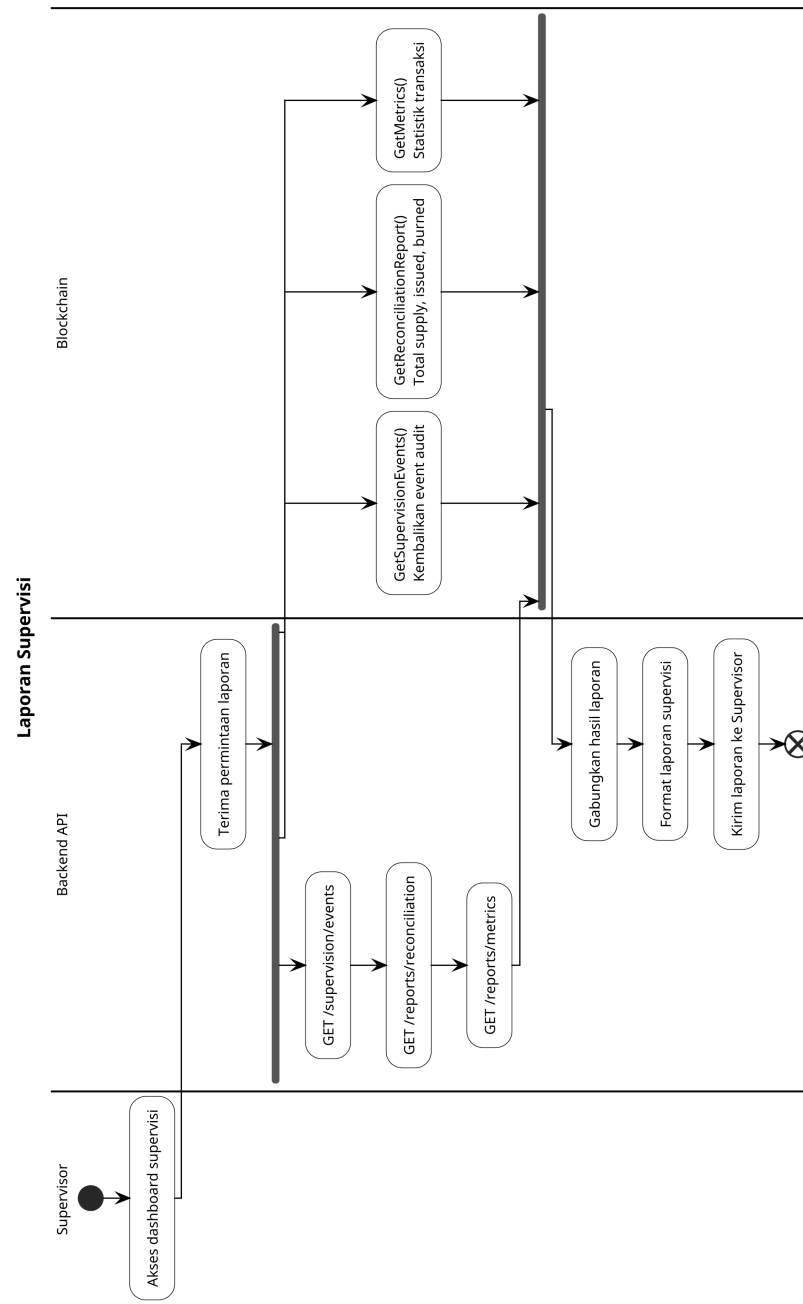
Gambar 3.18. Activity diagram onboarding KYC pelanggan



Gambar 3.19. Activity diagram transfer saldo ritel



Gambar 3.20. Activity diagram pembekuan dan pencairan peserta oleh Bank Indonesia

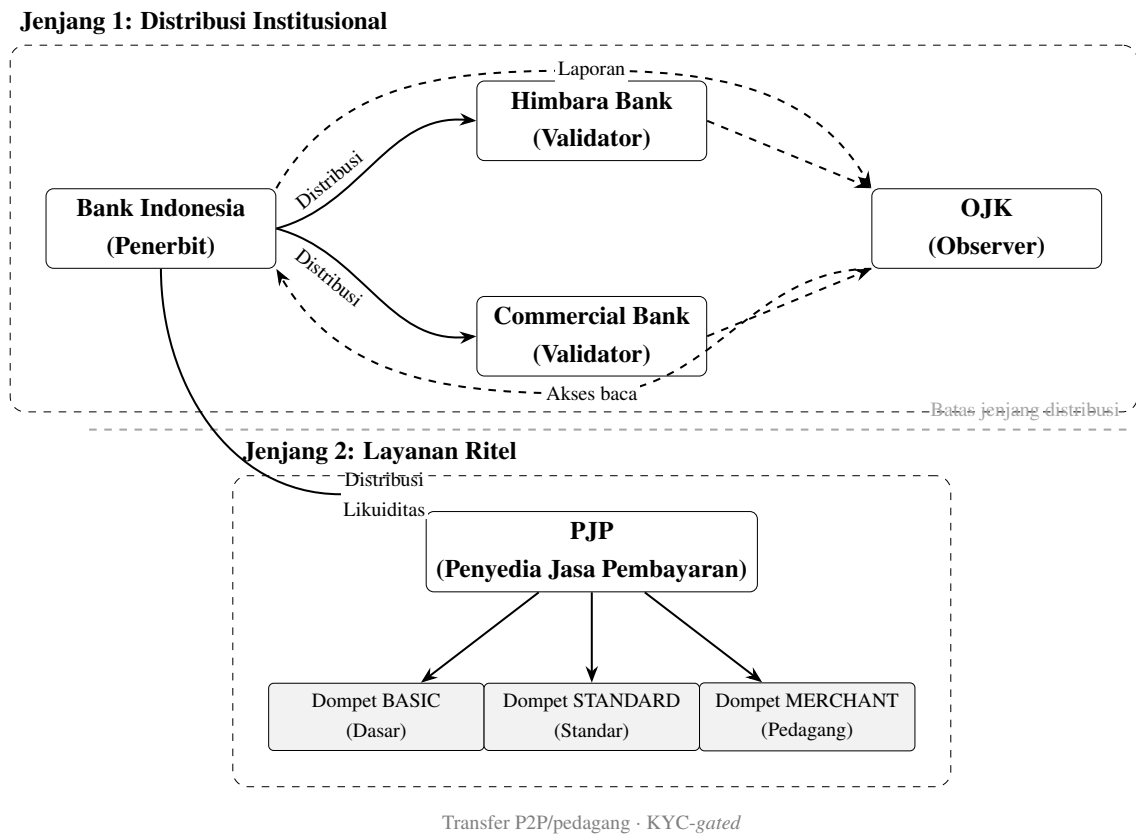


Gambar 3.21. Activity diagram laporan dan pengawasan supervisi

3.1.1.4 Perancangan Arsitektur Sistem Blockchain

Secara konseptual, sistem menggunakan model distribusi Rupiah Digital dua jenjang sebagaimana ditunjukkan pada Gambar 3.22. Jenjang distribusi institusional menghubungkan Bank Indonesia selaku penerbit dengan bank validator (HimbaraBank dan CommercialBank) serta PJP melalui jaringan Hyperledger Fabric. Jenjang layanan ritel menyediakan pembayaran melalui PJP, termasuk transfer antarpelanggan, transfer pelanggan-ke-pedagang, dan kontrol kepatuhan berbasis KYC. Jenjang institusional tersebut menjelaskan jalur distribusi menuju layanan ritel, bukan implementasi

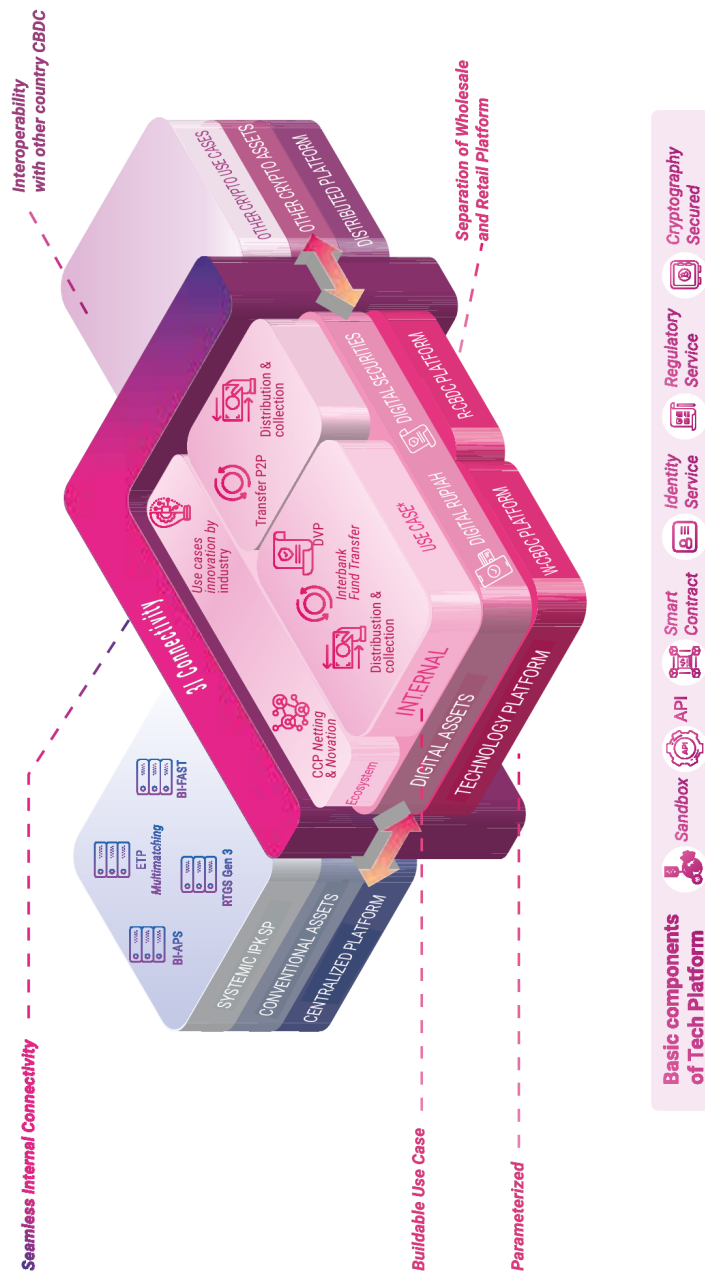
instrumen w-Digital Rupiah yang terpisah.



Gambar 3.22. Arsitektur distribusi dua jenjang pada purwarupa Rupiah Digital ritel.

Arsitektur teknologi Rupiah Digital dalam White Paper CBDC Bank Indonesia terdiri atas lapisan platform teknologi, lapisan aset digital, dan lapisan use case. Komponen dasarnya mencakup *sandbox*, API, *smart contract*, layanan identitas, layanan regulasi, dan pengamanan kriptografi sebagaimana ditunjukkan pada Gambar 3.23 [5]. Purwarupa *bi-coin-fabric* memetakan gagasan tersebut secara terbatas melalui API backend, *chaincode*, identitas dan KYC, peran pengawasan, serta *ledger* Hyperledger Fabric. Pemetaan terbatas ini menjadikan purwarupa sebagai model eksperimental yang selaras dengan prinsip arsitektur BI, bukan representasi penuh arsitektur produksi Rupiah Digital.

Figure 8. The Technology Architecture of Digital Rupiah

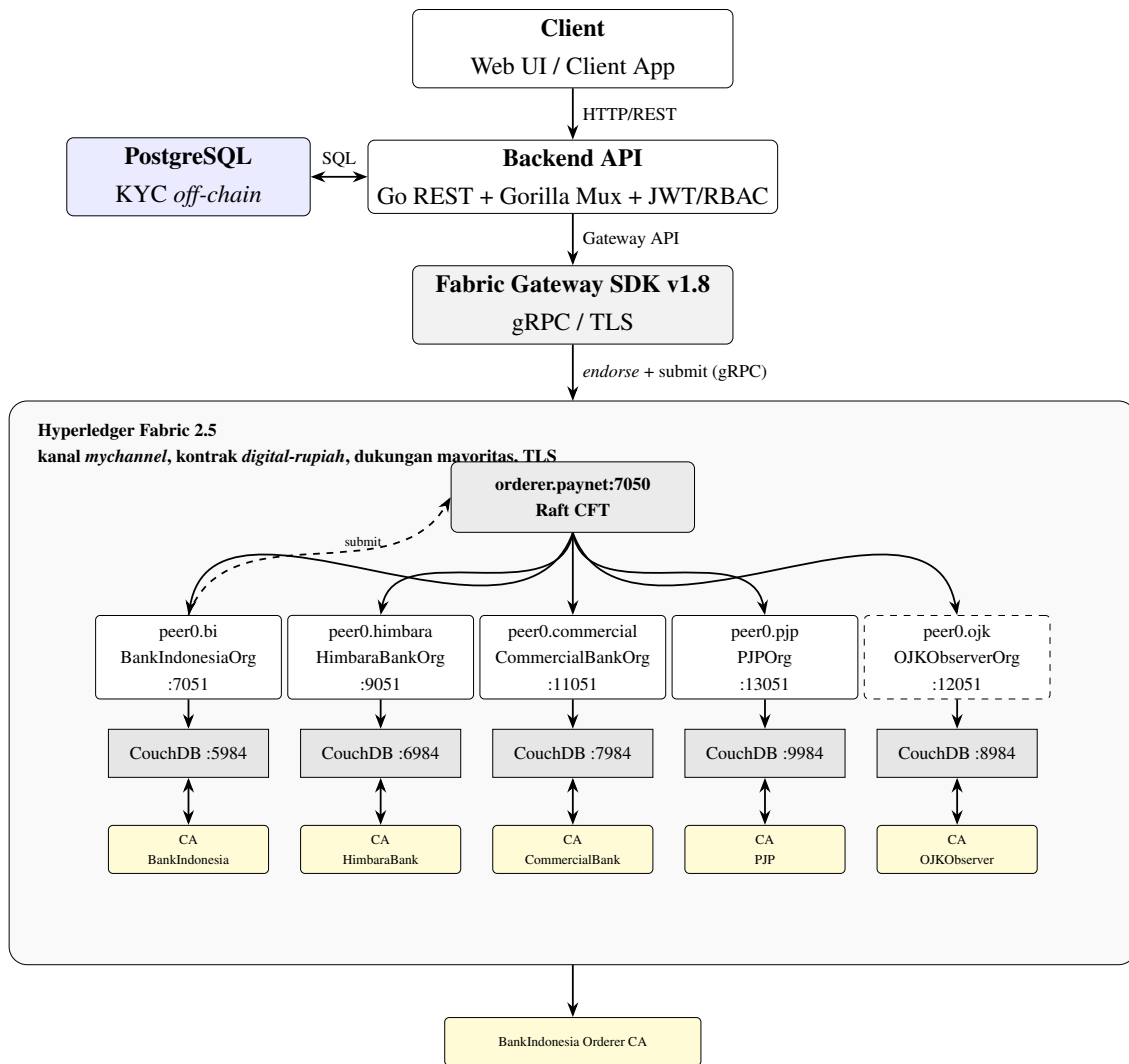


Gambar 3.23. Arsitektur teknologi Rupiah Digital menurut White Paper CBDC Bank Indonesia. Sumber: Bank Indonesia [5].

Sistem dirancang sebagai arsitektur layanan *backend API* dan jaringan *blockchain* sebagaimana ditunjukkan pada Gambar 3.24. Lapisan *backend API* bertindak sebagai *gateway* yang menerjemahkan permintaan HTTP menjadi pemanggilan *chaincode* sekaligus menegakkan kontrol akses berbasis peran, sedangkan lapisan jaringan *blockchain* menjalankan *chaincode* serta menyimpan *ledger*.

Kontrol akses memakai dua batas kepercayaan. Backend memvalidasi JWT, membatasi rute menurut peran, mencatat subjek dan peserta pemanggil, lalu memilih gateway Fabric berdasarkan MSP kustodian pada konteks permintaan. Untuk pengguna ritel, backend juga mencocokkan subjek JWT dengan `owner_id` dompet pengirim. Chaincode memvalidasi MSP pemanggil dan MSP kustodian dompet: mutasi moneter, limit, dan siklus hidup peserta hanya menerima MSP yang diizinkan, sedangkan MSP pengamat tidak dapat memutasi buku besar. Kepemilikan nasabah tetap menjadi batas kepercayaan backend karena identitas pemilik tidak digantikan oleh identitas klien Fabric pada chaincode.

Berdasarkan klasifikasi model pembayaran pada Bab 2, purwarupa *bi-coin-fabric* dikategorikan sebagai model dompet ritel berbasis akun (*account-based wallet model*) yang diimplementasikan di atas *ledger* Hyperledger Fabric. Klasifikasi ini diturunkan dari rancangan data *on-chain*: objek `Wallet` menyimpan `OwnerID`, `Tier`, dan `Balance`; fungsi `Transfer` memeriksa status KYC, limit tingkatan, status beku, dan saldo, kemudian mendebit saldo pengirim serta mengkredit saldo penerima. Sistem tidak membentuk objek token pembawa (*bearer token*), UTXO, atau nomor seri token yang berpindah antarpemilik. Mekanisme pembayaran purwarupa berupa dompet berbasis akun yang disederhanakan, bukan model token murni. Model tersebut digunakan untuk menguji KYC bertingkat, limit saldo/transaksi, dan auditabilitas transfer ritel.



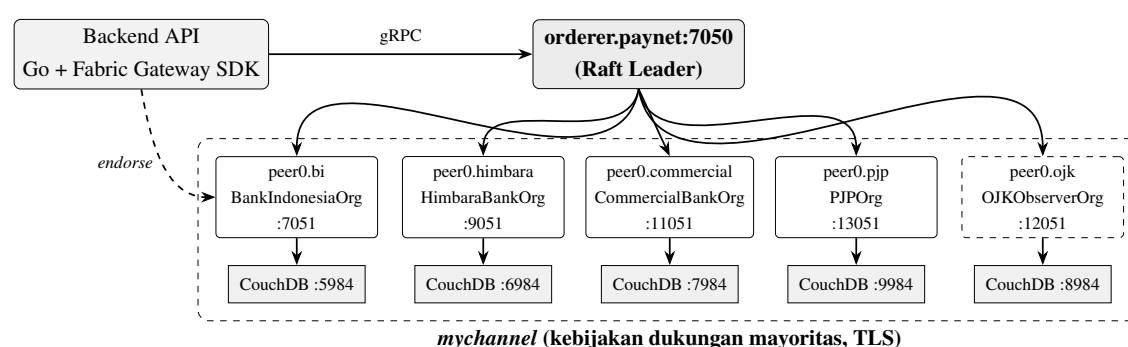
Gambar 3.24. Arsitektur sistem Rupiah Digital: lapisan *client*, *backend API* (Go REST + JWT/RBAC), basis data PostgreSQL luring, *Fabric Gateway SDK*, serta jaringan Hyperledger Fabric 2.5 dengan lima organisasi *peer*, CouchDB, CA per organisasi, dan *orderer* Raft

Jaringan *blockchain* dirancang dengan lima organisasi peserta dan satu organisasi *orderer*, merepresentasikan ekosistem Rupiah Digital multipihak. Setiap organisasi memiliki MSP dan *peer* tersendiri dengan basis data status CouchDB. Rincian organisasi disajikan pada Tabel 3.4.

Tabel 3.4. Organisasi pada jaringan Rupiah Digital

Organisasi	Peran	Keterangan
BankIndonesiaOrg	Penerbit	Penerbitan dan penarikan Digital Rupiah, pengawasan
HimbaraBankOrg	Validator	Validator jaringan; penerima distribusi BI
CommercialBank Org	Validator	Validator jaringan; penerima distribusi BI
PJPOrg	PJP	Penyedia layanan ritel; penerima distribusi BI
OJKObserverOrg	Observer	Pengamatan/pengawasan (baca)
BankIndonesia Orderer	Orderer	Layanan pengurutan (Raft)

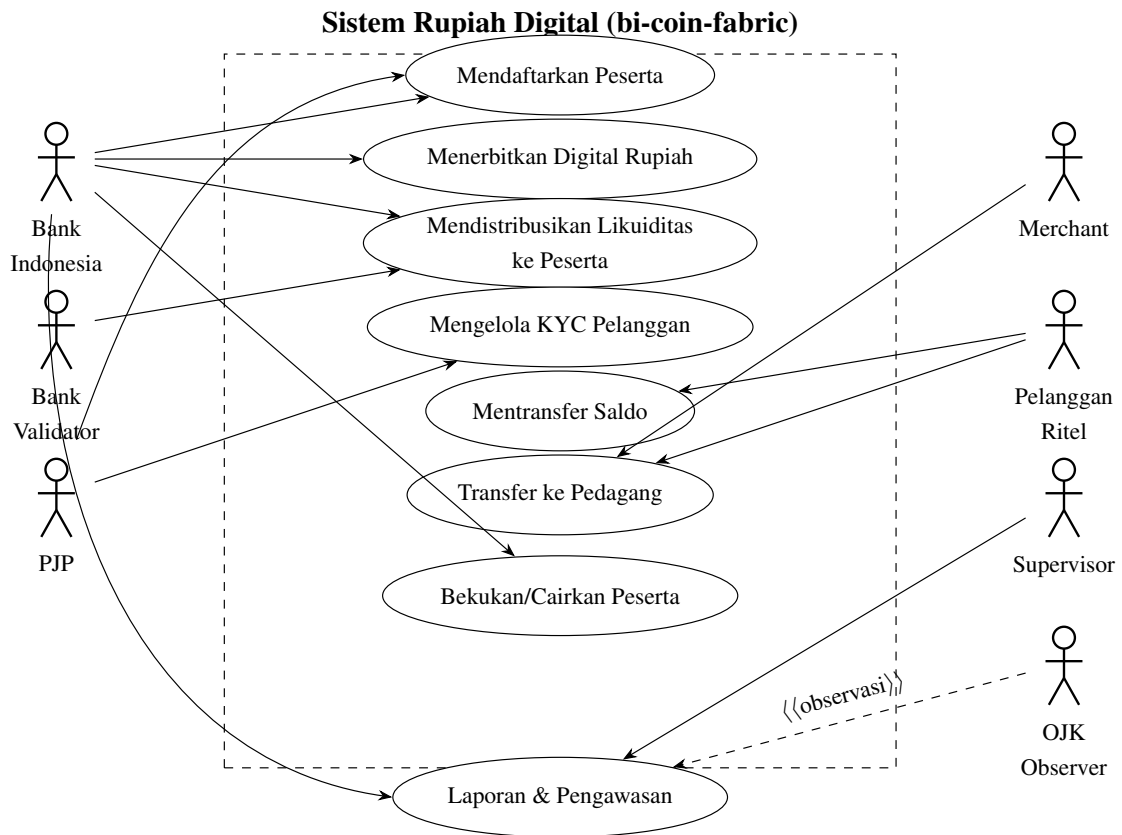
Seluruh peserta tergabung dalam satu kanal (*channel*) dengan kebijakan dukungan *majority*, sehingga pemutakhiran *ledger* memerlukan dukungan dari mayoritas organisasi. Komunikasi antarsimpul diamankan dengan TLS. Topologi jaringan secara lengkap ditunjukkan pada Gambar 3.25, yang memperlihatkan hubungan antara *orderer*, kelima *peer*, basis data CouchDB masing-masing organisasi, serta *backend API* yang terhubung ke jaringan melalui *Fabric Gateway SDK*.



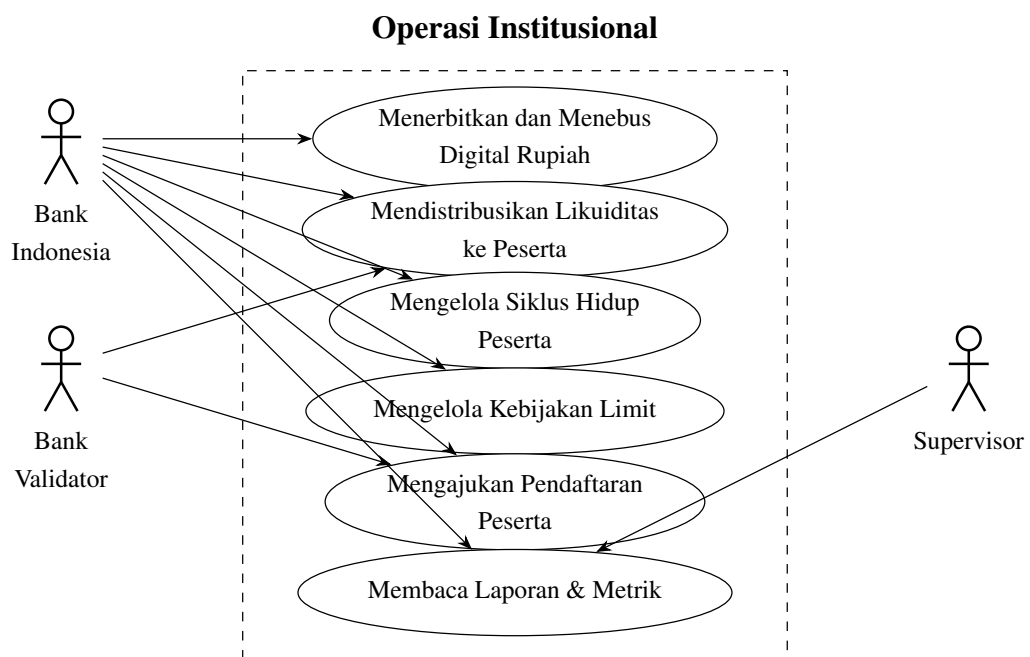
Gambar 3.25. Arsitektur jaringan Hyperledger Fabric sistem Rupiah Digital (5 organisasi)

Kebutuhan fungsional sistem dimodelkan dengan diagram use case. Aktor sistem terdiri atas Bank Indonesia, bank validator, PJP, pedagang (*merchant*), pelanggan ritel, supervisor, dan pengamat OJK. Diagram use case keseluruhan ditunjukkan pada

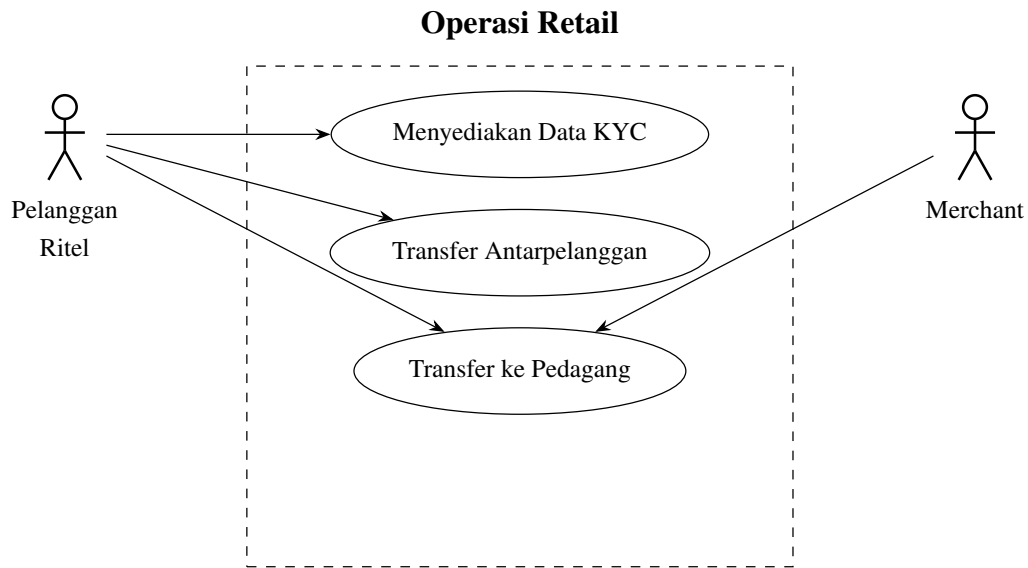
Gambar 3.26, sedangkan rincian operasi institusional dan ritel ditunjukkan pada Gambar 3.27 dan Gambar 3.28.



Gambar 3.26. Diagram use case keseluruhan sistem Rupiah Digital



Gambar 3.27. Diagram use case operasi institusional



Gambar 3.28. Diagram use case operasi retail

Pemetaan aktor pada diagram tersebut mengikuti kendali peran pada lapisan *backend*. Pendaftaran peserta dapat diajukan oleh institusi, sedangkan persetujuan, penolakan, pembekuan, dan *offboarding* hanya dapat dilakukan oleh Bank Indonesia. Supervisor dan pengamat OJK hanya memperoleh jalur baca pengawasan. Purwarupa belum menerapkan persetujuan dua orang (*four-eyes*); parameter persetujuan senior pada profil KYC karena itu merupakan status purwarupa, bukan bukti alur persetujuan terpisah. Pada operasi ritel, data KYC disediakan oleh pelanggan, tetapi pengajuan profil KYC ke sistem (`POST /kyc/profiles`) dilakukan oleh operator PJP, bukan oleh pelanggan secara mandiri.

Status sistem disimpan pada *world state* dalam bentuk objek-objek yang dirangkum pada Tabel 3.5. Objek-objek ini menjadi dasar seluruh operasi *chaincode*.

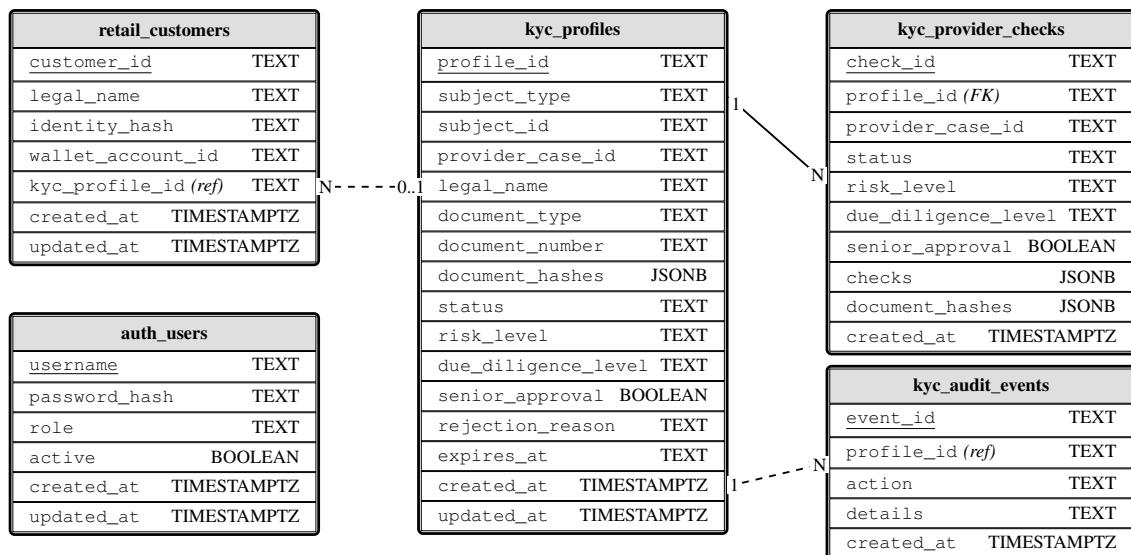
Tabel 3.5. Objek status utama pada ledger

Objek	Keterangan
Participant	Entitas peserta (validator/observer/PJP), MSP kustodian, dan BIC opsional beserta status siklus hidup
Wallet	Dompet dengan pemilik, peserta dan MSP kustodian, saldo, <i>tingkatan</i> ritel, status beku, serta penghitung keluar harian/bulanan dan masuk bulanan

Objek	Keterangan
TierLimit	Batas saldo, per transaksi, keluar, dan masuk per tingkatan
KycProfile	Jangkar KYC: jenis subjek, status, risiko, uji tuntas, persetujuan senior, masa berlaku, dan hash dokumen
TransactionRecord	Catatan transaksi untuk audit dan pelaporan
SystemLimit	Batas tingkat sistem (pasokan global, saldo per peserta, nilai per transaksi)

3.1.1.5 Perancangan Basis Data Luring

Selain status *on-chain* pada *world state* yang dirangkum pada Tabel 3.5, sistem memerlukan basis data relasional luring (*off-chain*) pada PostgreSQL untuk menyimpan data yang tidak layak atau tidak perlu ditempatkan pada *ledger*. Data tersebut mencakup kredensial autentikasi portal, pengayaan data KYC (nama legal, jenis dan nomor dokumen), serta riwayat pemeriksaan penyedia dan jejak audit yang bersifat *append-only*. Diagram *Entity-Relationship* pada Gambar 3.29 menunjukkan rancangan basis data luring tersebut. Tabel `auth_users` berdiri sendiri untuk autentikasi portal, sedangkan `kyc_profiles` menjadi pusat relasi yang ditautkan oleh `kyc_provider_checks` melalui kendala *foreign key* berkebijakan *ON DELETE CASCADE*. Tautan lain, yakni `kyc_audit_events` dan `retail_customers`, menggunakan referensi lunak (tanpa kendala basis data) karena pengidentifikasinya seperti `subject_id` dan `wallet_account_id` mencerminkan pengidentifikasi *on-chain* yang otoritasnya berada pada *ledger*.



Keterangan: teks = *primary key* (PK); (FK) = *foreign key* dengan kendala basis data; (ref) = referensi lunak (tanpa kendala).
Garis utuh = relasi berkendala (ON DELETE CASCADE); garis putus-putus = referensi lunak.

Gambar 3.29. Diagram *Entity-Relationship* basis data luring (*off-chain*) pada PostgreSQL. Tabel `auth_users` berdiri sendiri untuk autentikasi portal, sedangkan `kyc_profiles` menjadi pusat relasi profil KYC, riwayat pemeriksaan penyedia, dan jejak audit.

3.1.2 Metode Implementasi

Pemilihan teknologi dalam implementasi didasarkan pada kesesuaian dengan karakteristik sistem Rupiah Digital yang bersifat *permissioned*, multitenjang, dan memerlukan kueri status yang kompleks.

3.1.2.1 Software Development Life Cycle

Metode *prototyping* dipilih sebagai SDLC penelitian ini. Sistem dibangun sebagai purwarupa fungsional yang mencakup seluruh komponen inti (*chaincode*, jaringan Fabric, dan *backend API*), kemudian dievaluasi secara kuantitatif. Pendekatan ini sesuai dengan karakteristik penelitian eksplorasi pada sistem *blockchain* yang belum memiliki spesifikasi teknis baku dari regulator [13].

3.1.2.2 Kerangka Kerja Blockchain

Hyperledger Fabric 2.5 dipilih sebagai kerangka kerja *blockchain* karena mendukung model keanggotaan berbasis MSP yang sesuai dengan struktur hierarkis Rupiah Digital dua jenjang [17]. Fabric memungkinkan pembatasan akses per peran melalui *endorsement policy*, pemisahan kanal, dan integrasi basis data status yang fleksibel.

3.1.2.3 Mekanisme Konsensus

Konsensus Raft (*etcdraft*) dipilih karena mendukung model *crash fault tolerance* (CFT) yang memadai untuk skenario jaringan berizin antarpeserta terpercaya [17]. Dibanding Kafka, Raft memiliki latensi *ordering* lebih rendah dan tidak bergantung pada infrastruktur tambahan.

3.1.2.4 State Database

CouchDB dipilih sebagai basis data status *world state* menggantikan LevelDB bawaan. CouchDB mendukung kueri kaya berbasis JSON (*rich query*) yang diperlukan untuk pelaporan portofolio dompet, pemfilteran KYC, dan pengawasan saldo. Kemampuan ini tidak dapat dicapai dengan efisien menggunakan LevelDB yang hanya mendukung kueri berbasis kunci. Perbandingan kedua pilihan tersebut dirangkum pada Tabel 3.6.

Tabel 3.6. Perbandingan basis data status Hyperledger Fabric berdasarkan dokumentasi resmi [4].

Kriteria	LevelDB	CouchDB
Model data	Pasangan kunci–nilai yang tertanam pada proses <i>peer</i>	Dokumen JSON pada layanan basis data terpisah
Kueri	Pembacaan berdasarkan kunci dan rentang kunci	Kueri kaya berbasis bidang JSON dan selector
Indeks	Tidak menyediakan indeks JSON	Mendukung indeks JSON untuk kueri yang deterministik
Operasional	Konfigurasi dan kebutuhan sumber daya lebih sederhana	Memerlukan kontainer dan pengelolaan indeks tambahan
Kesesuaian	Status sederhana dengan pola akses berbasis kunci	Pelaporan dompet, pemfilteran KYC, dan pengawasan saldo

3.1.2.5 Basis Data Luring

Sementara CouchDB menampung *world state* Fabric, basis data luring (*off-chain*) yang menampung data sensitif memerlukan paradigma yang berbeda. Pemilihan basis data relasional PostgreSQL didasarkan pada lima kriteria yang dipaparkan berikut ini, kemudian dirangkum pada Tabel 3.7.

Relasional versus non-relasional. Data KYC dan kredensial autentikasi merupakan data terstruktur dengan skema kaku, hubungan antarentitas yang jelas (profil → pemeriksaan penyedia → jejak audit), serta persyaratan integritas transaksional yang ketat. Basis data relasional menjamin ketiga aspek tersebut melalui kepatuhan ACID penuh, kendala *foreign key*, dan transaksi DDL yang atomik. Sebaliknya, basis data dokumen NoSQL (mis. MongoDB) dirancang untuk data semiterstruktur dengan skema fleksibel serta beban baca dominan, dan secara tradisional mengorbankan konsistensi transaksional demi skalabilitas horizontal [60]. Untuk jejak audit yang bersifat *append-only* dan riwayat pemeriksaan KYC, penjaminan bahwa suatu peristiwa benar-benar tersimpan sebelum komit (MVCC tanpa *read skew*) merupakan kebutuhan mutlak, bukan opsional.

PostgreSQL versus MySQL dan SQL Server. Antara Alternatif relasional, PostgreSQL dipilih berdasarkan pertimbangan teknis berikut:

1. *JSON/JSONB dengan indeks GIN.* Dokumen pendukung KYC (luisan metadata, struktur respons penyedia) disimpan sebagai JSONB yang dapat diindeks secara native melalui GIN, memungkinkan kueri *rich* tanpa kehilangan kepatuhan ACID [61]. MySQL hanya mendukung tipe JSON dengan kapabilitas pengindeksan yang lebih terbatas.
2. *Row-Level Security (RLS).* PostgreSQL menyediakan RLS secara native, sehingga kebijakan isolasi data antarorganisasi peserta (mis. bank validator hanya membaca KYC nasabahnya sendiri) dapat ditegakkan pada tingkat basis data, bukan sekadar lapisan aplikasi. Hal ini sejalan dengan prinsip regulasi UU Pelindungan Data Pribadi (UU PDP) yang mensyaratkan kontrol akses berbasis peran. MySQL tidak memiliki fitur ini dan SQL Server hanya menyediakannya pada edisi Enterprise [61].
3. *MVCC dan transaksi DDL.* PostgreSQL mengelola konkurensi via MVCC tanpa *lock* memblokir pembaca, dan membungkus perubahan skema (DDL) dalam transaksi yang dapat dieksekusi secara atomik. Fitur ini tidak dimiliki MySQL karena DDL-nya bersifat *implicit commit* [62].
4. *Lisensi terbuka.* Lisensi PostgreSQL mirip BSD memungkinkan penggunaan, modifikasi, dan distribusi tanpa biaya royalti, menghindari *vendor lock-in*. SQL Server bersifat proprietari, sedangkan MySQL berlisensi ganda (GPL/komersial) yang membatasi integrasi pada produk berlisensi tidak kompatibel [62].
5. *Kesesuaian ekosistem Fabric.* Pola arsitektur Fabric merekomendasikan penyimpanan data sensitif di luar *ledger* melalui *private data collections* atau basis data eksternal, dengan hanya hash kriptografis yang dibukukan [63]. PostgreSQL merupakan *backend* relasional yang banyak diadopsi dalam implementasi Fabric

pendidikan dan industri, sehingga ketersediaan *driver* Go (pgx) dan pola integrasi yang teruji lebih matang dibandingkan alternatif lain.

Pola pemisahan on-chain/off-chain yang diadopsi penelitian ini sejalan dengan rekomendasi BIS untuk arsitektur CBDC ritel hibrid, yakni bank sentral mempertahankan jangkar kriptografis pada *ledger* sementara perantara menyimpan data identitas pada sistem terpisah yang menjalankan KYC [6]. Pendekatan serupa terlihat pada proyek Sand Dollar (Bahama) yang mengelola data KYC pada *layer* identitas digital terpisah dari buku besar [58], serta uji coba e-Krona (Swedia) yang memisahkan registri peserta dari *ledger* token [59].

Tabel 3.7. Perbandingan basis data kandidat untuk penyimpanan luring KYC.

Kriteria		PostgreSQL	MySQL	SQL Server
Kepatuhan penuh	ACID	Ya (DDL transaksional)	Sebagian (DDL non-atomik)	Ya
JSONB + GIN	indeks	Native	Terbatas	Terbatas
Row-Level Security		Native	Tidak ada	Edisi Enterprise saja
MVCC baca	tanpa lock	Ya	Ya (InnoDB)	Ya (snapshot)
Lisensi		BSD-like (bebas)	GPL / komersial	Proprietari
Adopsi Fabric	ekosistem	Umum	Jarang	Jarang

3.1.2.6 Bahasa Pemrograman

Chaincode dan *backend API* ditulis dalam bahasa Go. Go dipilih karena (1) SDK *Fabric Contract API* dan *Fabric Gateway SDK* tersedia secara resmi dan aktif dipelihara untuk ekosistem Fabric; (2) model konkurensi goroutine cocok untuk beban I/O tinggi pada *backend API*; dan (3) kompilasi statis mempermudah distribusi kontainer tanpa dependensi eksternal [17].

3.1.3 Metode Pengujian

Pengujian terdiri atas pengujian fungsional dan pengujian kinerja. Pengujian fungsional memverifikasi bahwa setiap alur proses berjalan sesuai rancangan, termasuk penegakan limit dan kontrol akses. Verifikasi fungsional dilakukan melalui dua bentuk

pengujian yang saling melengkapi, yaitu skenario fungsional *end-to-end* pada purwarupa serta *unit test* otomatis pada logika layanan *backend* dan aturan bisnis *chaincode*. Pengujian skenario digunakan untuk memastikan integrasi antarkomponen berjalan sesuai rancangan, sedangkan *unit test* digunakan untuk memeriksa ketepatan aturan pada lapisan yang lebih terisolasi, seperti penurunan tier KYC, penegakan limit transaksi, kontrol akses berbasis peran, dan pembaruan status dompet. Pengujian kinerja dilakukan menggunakan Hyperledger Caliper untuk mengukur *throughput* dan latensi pada berbagai skenario beban. Suite pengujian Caliper dibagi menjadi tiga kelompok, yaitu garis dasar transfer ritel, profil *repeatability* untuk beban puncak, serta profil khusus yang menekankan konformansi kebijakan dan konservasi saldo pada permintaan agregat berlebih.

Skenario pengujian kinerja dirancang untuk mengamati pengaruh laju pengiriman transaksi (*send rate*), jumlah *worker* Caliper, dan jenis fungsi rantai blok terhadap *throughput* dan latensi. Pengujian transfer ritel dijalankan dengan 1, 2, dan 4 *worker*. Pengujian fungsi lain dijalankan terpisah agar hasil transfer, KYC, operasi moneter, pembacaan pengawasan, dan kebijakan administratif tidak dicampur menjadi satu angka TPS umum. Selain itu, lima ronde identik pada beban puncak dipakai untuk melaporkan rerata dan *confidence interval* rerata, sedangkan dua profil khusus dipakai untuk memisahkan bukti penegakan kebijakan dan bukti konservasi saldo dari pembacaan *throughput* garis dasar. Konfigurasi ronde pengujian dirangkum pada Tabel 3.8, sedangkan rincian profil khusus beserta hasil yang diharapkan dipaparkan pada Tabel 3.9.

Tabel 3.8. Skenario beban pengujian kinerja

Ronde	Durasi/jumlah	Laju (TPS)	Beban kerja
Pemanasan	30 detik	10	Transfer ritel, 1/2/4 <i>worker</i>
Beban stabil	120 detik	30	Transfer ritel, 1/2/4 <i>worker</i>
Beban puncak	120 detik	60	Transfer ritel, 1/2/4 <i>worker</i>
Pengulangan puncak	5 × 60 detik	60	Transfer ritel, 2 <i>worker</i> , lima ronde identik
Fungsional terpisah	40–80	5–30	KYC, moneter, pengawasan, administratif
Konformansi kebijakan	7	5	<i>negative-path</i> , 1 <i>worker</i>
Agregat <i>overspend</i>	200	80	Pengirim bersama, 4 <i>worker</i> , permintaan melebihi saldo

Tabel 3.9. Profil dan kasus uji khusus beserta hasil yang diharapkan

Profil	Tx	Laju (TPS)	Hasil yang diharapkan
KYC & <i>onboarding</i>	40	5	Data KYC mentah tidak ditulis ke buku besar dan tingkatan diturunkan deterministik dari profil
Operasi moneter	80	20	<i>Mint</i> dan <i>Burn</i> berpasangan berhasil sesuai batas sistem
Kueri pengawasan	80	30	Fungsi baca-saja mengembalikan data tanpa mengubah status
Kebijakan administratif	40	5	Limit per transaksi, harian, dan bulanan ditegakkan serta pembelanjaan ditolak tanpa KYC disetujui
<i>Negative-path</i>	7	5	Setiap pelanggaran kebijakan ditolak dengan alasan yang tepat; indikator utama adalah <i>enforced/gaps</i> dari validator log, bukan rasio sukses/gagal mentah
Agregat <i>overspend</i>	200	80	Saldo akhir tidak negatif, jumlah komit dan penolakan tercatat, serta sedikitnya satu transfer ditolak

Metrik yang diukur meliputi *throughput* (TPS), latensi rata-rata, latensi maksimum dan minimum, serta tingkat keberhasilan transaksi [50]. Mengikuti definisi metrik kinerja blockchain, tingkat keberhasilan transaksi dihitung dengan Persamaan 3-1.

$$R_s = \frac{N_{\text{valid}}}{N_{\text{submitted}}} \times 100\% \quad (3-1)$$

Throughput valid dihitung dengan Persamaan 3-2.

$$T = \frac{N_{\text{valid}}}{t_{\text{end}} - t_{\text{start}}} \quad (3-2)$$

Latensi transaksi ke- i dihitung dengan Persamaan 3-3.

$$L_i = t_{\text{confirm},i} - t_{\text{submit},i} \quad (3-3)$$

Pada persamaan tersebut, N_{valid} adalah jumlah transaksi valid, $N_{\text{submitted}}$ adalah jumlah transaksi yang dikirim, dan selang $t_{\text{end}} - t_{\text{start}}$ dinyatakan dalam detik. Kegagalan transaksi akibat konflik MVCC dicatat dan dianalisis sebagai bagian dari karakterisasi sistem, bukan dihilangkan dari hasil pengukuran. Untuk profil garis dasar transfer dan fungsi lain, rasio sukses/gagal Caliper dibaca langsung sebagai hasil benchmark. Untuk profil *negative-path*, penolakan transaksi justru merupakan keluaran yang diharapkan sehingga indikator utamanya adalah kecocokan alasan penolakan dengan oracle setiap kasus; kegagalan konektor atau alasan lain menandai pengujian tidak valid. Pada profil agregat *overspend*, bukti utamanya adalah saldo akhir tidak negatif, adanya penolakan karena total permintaan melebihi saldo, dan klasifikasi alasan penolakan aktual. Penolakan tidak diasumsikan sebagai konflik MVCC.

Justifikasi pemilihan parameter. Parameter pengujian dipilih berdasarkan dua pertimbangan: literatur dan rekayasa sistem.

Berdasarkan literatur. Variasi 1, 2, dan 4 *worker* digunakan untuk memisahkan pengaruh pembangkit beban Caliper dari kapasitas jaringan Fabric. Dengan kebijakan *endorsement majority* (3 dari 5 organisasi), penambahan *worker* tidak otomatis meningkatkan kapasitas jaringan karena bottleneck dapat berpindah ke eksekusi *peer*, validasi blok, atau pemrosesan *orderer* [1, 17, 32]. Kontrol laju *fixed-rate* dipilih karena menahan laju pengiriman sebagai variabel independen sehingga penurunan *throughput* dan peningkatan latensi dapat diamati sebagai fungsi dari laju tersebut; kontrol *fixed-load* menyesuaikan TPS secara otomatis berdasarkan latensi sehingga menyembunyikan perilaku saturasi [1]. Laju 10 TPS digunakan sebagai garis dasar, 30 TPS sebagai beban stabil, dan 60 TPS sebagai beban puncak pada purwarupa lokal. Nilai tersebut dipilih agar profil transfer dasar dapat diamati pada topologi Garuda lokal tanpa mengubah semantik bisnis, bukan untuk mengklaim kapasitas produksi nasional. Hasil aktual, termasuk kegagalan *endorsement* atau konflik versi yang muncul pada ronde tertentu, tetap dilaporkan apa adanya.

Berdasarkan pertimbangan teknis sistem. Ronde pemanasan berlangsung 30 detik, sedangkan ronde stabil dan puncak masing-masing berlangsung 120 detik. Ketiga konfigurasi mempertahankan total 116 pelanggan, 16 pedagang, dan 52 slot transaksi:

116/16/52 pada 1 *worker*, 58/8/26 per *worker* pada 2 *worker*, serta 29/4/13 per *worker* pada 4 *worker*. Nilai transfer dibatasi Rp50.000 agar penggunaan berulang tetap berada dalam limit harian purwarupa. Populasi menyediakan pengirim STANDARD, penerima STANDARD, dan pedagang untuk rencana transaksi berputar dengan perebutan kunci rendah. Jenis transaksi ditentukan secara deterministik berdasarkan slot global sehingga komposisi gabungan tetap 70% transfer antarpelanggan, 25% transfer pelanggan-ke-pedagang, dan 5% pembacaan dompet. Profil pengulangan puncak menggunakan 2 *worker* dan lima ronde berdurasi 60 detik agar rerata serta interval kepercayaan reratanya dihitung tanpa mengubah komposisi beban. Setiap ronde memakai ruang nama kunci yang berbeda, tetapi kelimanya berjalan pada satu *ledger* profil yang sama. Interval tersebut karena itu mengukur *repeatability* dalam satu lingkungan, bukan lima replikasi infrastruktur yang independen.

Profil *negative-path* mengirim tujuh permintaan berurutan pada 5 TPS dengan 1 *worker*. Setiap permintaan sengaja melanggar satu aturan *chaincode*. Setelah Caliper selesai, validator memeriksa detail galat *peer* pada log; run valid hanya jika seluruh penolakan cocok dengan regex diagnostik kasus tanpa *gap* maupun *oracle error*. Profil agregat *overspend* mengirim 200 transfer masing-masing Rp1.000 pada 80 TPS dengan 4 *worker* dari satu dompet bersaldo awal Rp50.000 menuju 20 penerima. Karena permintaan agregat Rp200.000 melebihi saldo, skenario menguji konservasi saldo sekaligus perebutan satu kunci. Status generik yang dikembalikan konektor tidak digunakan untuk menetapkan penyebab setiap penolakan. Skenario ini tidak diklaim sebagai simulasi serangan *double-spend* protokol.

Setiap profil dijalankan pada jaringan dan buku besar baru. Urutan konfigurasi 1, 2, dan 4 *worker* diacak secara deterministik dengan *seed* 20260725. Generator bilangan acak beban kerja juga memakai *seed* yang dicampur dengan indeks ronde dan *worker*. Runner menghapus laporan lama sebelum eksekusi, membatalkan validitas ronde jika setup gagal, dan mencatat status, versi runtime, hash konfigurasi/beban kerja, revisi Git, keadaan *working tree*, urutan profil, serta *seed* dalam manifes JSON. Langkah ini mendukung *repeatability* pada lingkungan yang sama; hasil belum membuktikan *reproducibility* lintas mesin. Saldo awal dan seluruh parameter populasi merupakan pilihan rekayasa, bukan estimasi distribusi pengguna nasional.

Kesesuaian dengan skenario CBDC ritel. Beban kerja pengujian dirancang untuk merepresentasikan alur transaksi Rupiah Digital ritel secara struktural. Campuran transaksi garis dasar terdiri atas 70% transfer antarpelanggan, 25% transfer pelanggan-ke-pedagang, dan 5% pembacaan dompet. Komposisi ini dipilih karena CBDC ritel adalah instrumen pembayaran umum yang harus melayani pembayaran antarpengguna dan pembayaran ke pedagang, bukan hanya setelmen antarlembaga.

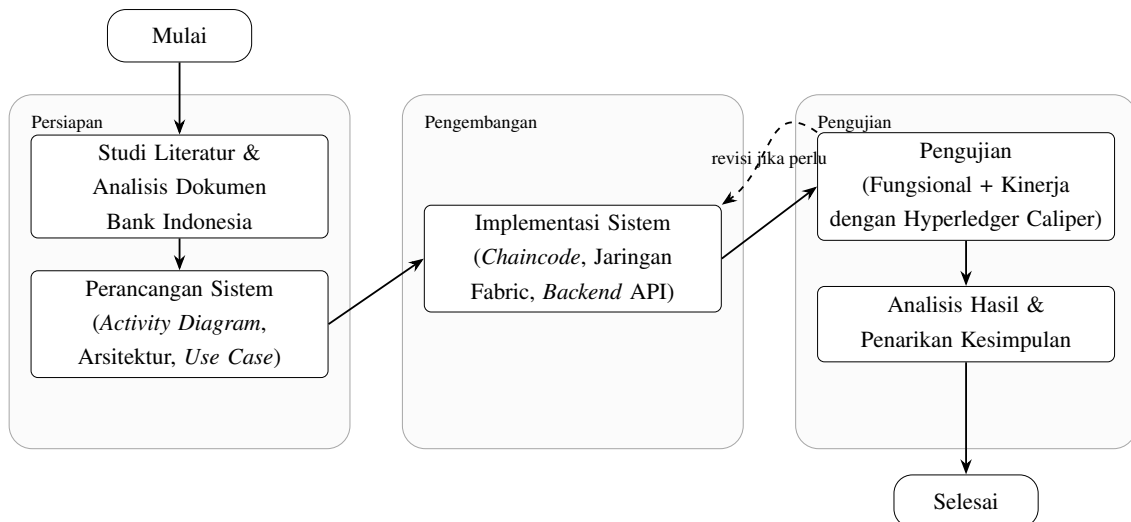
Literatur teknologi CBDC ritel menempatkan desain dompet, privasi, dan arsitektur distribusi sebagai elemen inti sistem pembayaran publik [6], sedangkan e-CNY dan eNaira menunjukkan preseden operasional bahwa dompet ritel dapat dibedakan menurut tingkat identifikasi dan limit [46,47]. Oleh karena itu, beban kerja yang hanya menguji transfer institusional tidak memadai untuk menjawab rumusan masalah purwarupa Rupiah Digital ritel.

Model dua jenjang penerbitan ($BI \rightarrow bank \rightarrow PJP \rightarrow pelanggan\ ritel$) sesuai dengan rancangan Rupiah Digital yang ditetapkan dalam dokumen konsultatif Bank Indonesia [12, 13]. Logika bisnis yang diuji bukan beban sintetik yang terlepas dari sistem, melainkan *chaincode* aktual yang mengimplementasikan profil KYC, penurunan tingkatan BASIC, STANDARD, dan MERCHANT, limit transaksi masuk dan keluar, serta pembacaan status dompet. Penggunaan Caliper dan metrik *throughput*/latensi mengikuti praktik pengukuran kinerja blockchain yang menekankan laju transaksi, latensi, dan karakterisasi kegagalan transaksi [1, 32, 50]. Profil *negative-path* menambah bukti bahwa aturan kepatuhan benar-benar menolak transaksi tidak sah, sedangkan profil adversarial *double-spend* memeriksa apakah konflik versi pada dompet pengirim bersama menghasilkan penolakan yang konsisten.

Perbedaan utama dengan sistem CBDC ritel produksi adalah skala (80 pelanggan per *worker* berbanding jutaan pengguna nyata), topologi jaringan (satu *peer* per organisasi dalam satu mesin lokal berbanding jaringan terdistribusi geografis), dan integrasi KYC nyata (disimulasikan secara terprogram). Keterbatasan ini berlaku umum pada studi prototipe CBDC berbasis *permissioned blockchain* [15] dan bukan kelemahan metodologis melainkan batasan cakupan yang dinyatakan secara eksplisit pada Batasan Penelitian (Bab I).

3.2 Alur Tugas Akhir

Tugas akhir ini dilaksanakan melalui tahapan berikut. Gambar 3.30 menyajikan alur dari studi literatur hingga analisis hasil. Implementasi dan pengujian saling memberi umpan balik sesuai siklus *prototyping*.



Gambar 3.30. Alur pelaksanaan tugas akhir: dari studi literatur hingga penarikan kesimpulan, dengan umpan balik iteratif antara implementasi dan pengujian

3.2.1 Mempelajari Proses Bisnis & Sistem Blockchain

Penelitian diawali dengan mempelajari konsep Rupiah Digital berdasarkan dokumen White Paper dan Consultative Paper Bank Indonesia [5, 12, 13], serta memahami mekanisme *blockchain* permissioned berbasis Hyperledger Fabric sebagai landasan implementasi.

3.2.2 Melakukan Studi Literatur

Mengkaji konsep CBDC, desain Rupiah Digital, Hyperledger Fabric, KYC berbasis risiko, dan limit dompet sebagai dasar perancangan sistem.

3.2.3 Menentukan Metode Penelitian

Menentukan aktor, kebutuhan fungsional, dan aturan bisnis sistem Rupiah Digital ritel berdasarkan model dua jenjang, serta menetapkan metode prototyping, pemilihan teknologi, dan parameter pengujian.

3.2.4 Pengembangan Sistem Blockchain bi-coin

Pengembangan sistem dilakukan secara iteratif meliputi instalasi infrastruktur Fabric, konfigurasi jaringan, *deploy chaincode*, pengembangan *gateway*, dan pengembangan *backend API*.

3.2.4.1 Instalasi Kerangka Kerja Hyperledger Fabric

Lingkungan pengembangan disiapkan dengan memasang Docker, Docker Compose, dan biner Fabric. Material kriptografi (sertifikat MSP dan TLS) dibangkitkan

menggunakan `cryptogen`. Blok genesis dan profil konfigurasi kanal dibuat dengan `configtxgen`.

3.2.4.2 Konfigurasi Jaringan Blockchain

Jaringan Fabric dibangun melalui skrip otomatisasi yang membangkitkan material kriptografi, blok genesis, dan konfigurasi kanal, kemudian menjalankan kontainer untuk lima *peer*, lima CouchDB, dan satu *orderer*. Chaincode dipaketkan, dipasang pada seluruh organisasi, disetujui sesuai kebijakan dukungan, lalu di-*commit* ke kanal.

3.2.4.3 Deploy Chaincode

Logika bisnis Rupiah Digital ditulis dalam bahasa Go menggunakan *Fabric Contract API* v2. Chaincode mengimplementasikan fungsi penerbitan (`Mint`, `RequestIssuance`), penarikan (`Burn`, `RequestRedemption`), distribusi berjenjang (`DistributeToParticipant`), transfer dengan penegakan limit (`Transfer`), KYC berbasis risiko, serta pelaporan dan pengawasan.

Gambar 3.31 menunjukkan implementasi fungsi `Mint` yang menambah saldo dompet setelah memverifikasi status beku dan batas tingkatan.

```

1 func (s *SmartContract) Mint(ctx contractapi.
    TransactionContextInterface ,
2     walletID string , amount int64) error {
3     // ... validasi MSP Bank Indonesia , limit transaksi , dan
    pasokan global ...
4     if amount <= 0 {
5         return fmt.Errorf("amount must be positive")
6     }
7     wallet , err := s.getWallet(ctx , walletID)
8     if err != nil { return err }
9     if wallet.Frozen {
10        return fmt.Errorf("wallet %s is frozen", walletID)
11    }
12    limit , err := s.getLimit(ctx , wallet.Tier)
13    newBalance := wallet.Balance + amount
14    if newBalance > limit.MaxBalance {
15        return fmt.Errorf("mint would exceed max balance for
    tier %s",
16            wallet.Tier)
17    }
18    wallet.Balance = newBalance
19    wallet.UpdatedAt = txNow(ctx).Format(time.RFC3339)
20    if err := s.emitAudit(ctx , "MINT" , walletID , "" , amount);
    err != nil {
21        return err
22    }
23    return s.recordTransaction(ctx , TxMint , walletID , "" , amount
    , TxSettled , "")
24 }

```

Gambar 3.31. Implementasi fungsi `Mint` pada *chaincode*: memverifikasi status beku dan batas saldo tingkatan sebelum menambah saldo dompet

Gambar 3.32 menunjukkan pemanggilan kebijakan transfer bersama. Fungsi kebijakan memeriksa batas per transaksi, pengeluaran harian dan bulanan, penerimaan bulanan, saldo pengirim, serta saldo maksimum penerima sebelum memperbarui penghitung secara deterministik.

```

1 func (s *SmartContract) Transfer(ctx contractapi.
    TransactionContextInterface ,
2     senderID string , receiverID string , amount int64 ,
    referenceID string) (map[string]interface {}, error) {
3     // ... periksa sender & receiver tidak beku ...
4     senderLimit , _ := s.getLimit(ctx , sender.Tier)
5     receiverLimit , _ := s.getLimit(ctx , receiver.Tier)
6     if err := applyRetailTransferPolicy(&sender , &receiver ,
    amount ,
7         senderLimit , receiverLimit , txNow(ctx)); err != nil {
8         return err
9     }
10    sender.Balance -= amount
11    receiver.Balance += amount
12    // ... putWallet , recordTransaction , audit , dan receipt
    idempotensi ...
13 }

```

Gambar 3.32. Penegakan kebijakan transfer bersama untuk batas keluar, batas masuk, saldo maksimum, dan kecukupan saldo

Gambar 3.33 menunjukkan penurunan tingkatan dari bukti KYC. Peserta institusional ditolak dari fungsi ini karena memakai batas tingkat sistem, bukan tingkatan dompet ritel.

```

1 func deriveWalletTier(profile KycProfile, now time.Time) (
    WalletTier, error) {
2     if err := validateKycEligibility(profile, now); err != nil {
3         return "", err
4     }
5     if profile.SubjectType == SubjectParticipant {
6         return "", fmt.Errorf("participant wallets are outside
    retail tiering")
7     }
8     if profile.DueDiligenceLevel == DueDiligenceSimplified &&
9         profile.SubjectType == SubjectRetailCustomer &&
10        profile.RiskLevel == RiskLow {
11        return TierBasic, nil
12    }
13    if profile.SubjectType == SubjectMerchant {
14        return TierMerchant, nil
15    }
16    return TierStandard, nil
17 }

```

Gambar 3.33. Penurunan tingkatan dompet ritel dari jenis subjek, risiko, dan tingkat uji tuntas KYC

3.2.4.4 Pengembangan Fabric Gateway

Fabric Gateway SDK v1.8 digunakan untuk menghubungkan *backend API* ke jaringan Fabric melalui gRPC. *Gateway* mengelola siklus transaksi mulai dari pembuatan proposal, pengumpulan *endorsement*, pengiriman ke *orderer*, hingga konfirmasi blok, sehingga lapisan *backend* tidak perlu menangani protokol Fabric secara langsung.

3.2.4.5 Pengembangan Backend

Lapisan *backend* ditulis dalam bahasa Go menggunakan Gorilla mux dan *Fabric Gateway SDK*. Backend menyediakan antarmuka REST, menegakkan kontrol akses berbasis peran melalui *middleware*, serta menerjemahkan permintaan menjadi pemanggilan `SubmitTransaction` atau `EvaluateTransaction` pada *chaincode*. Gambar 3.34 menunjukkan pengelompokan rute REST berdasarkan peran akses menggunakan *middleware* `RequireRole`.

```

1 func (h *Handler) RegisterRoutes(r *mux.Router) {
2     // Publik: tidak memerlukan autentikasi
3     r.HandleFunc("/health", h.Health).Methods("GET")
4
5     // Pembacaan dompet oleh aktor terautentikasi
6     auth := r.NewRoute().Subrouter()
7     auth.Use(middleware.RequireRole(
8         middleware.RoleAuthenticated, middleware.RoleKycVerified
9         ,
10        middleware.RoleBankPjp, middleware.RoleBankIndonesia))
11    auth.HandleFunc("/wallets", h.ListWallets).Methods("GET")
12
13    // Bank/PJP melakukan onboarding dan keputusan KYC
14    kycMutation := r.NewRoute().Subrouter()
15    kycMutation.Use(middleware.RequireRole(middleware.
16        RoleBankPjp))
17    kycMutation.HandleFunc("/kyc/profiles", h.SubmitKycProfile).
18    Methods("POST")
19    kycMutation.HandleFunc("/kyc/profiles/{id}/refresh",
20        h.RefreshKycProfile).Methods("POST")
21    kycMutation.HandleFunc("/wallets", h.CreateWallet).Methods("
22    POST")
23
24    // Pelanggan/pedagang yang disetujui melakukan transfer
25    payments := r.NewRoute().Subrouter()
26    payments.Use(middleware.RequireRole(
27        middleware.RoleKycVerified, middleware.RoleMerchant))
28    payments.HandleFunc("/transfers", h.Transfer).Methods("POST")
29 }

```

Gambar 3.34. Pengelompokan rute REST *backend* berdasarkan peran akses: setiap subrouter menerapkan *middleware* `RequireRole` yang berbeda sesuai hierarki wewenang

3.2.5 Pengujian Sistem Blockchain bi-coin

Pengujian mencakup dua tahap: pengujian fungsional untuk memverifikasi kebenaran seluruh alur proses, dan pengujian kinerja menggunakan Hyperledger Caliper untuk mengukur *throughput* dan latensi.

Pada tahap pengujian fungsional, verifikasi tidak hanya dilakukan melalui

eksekusi skenario pada aplikasi, tetapi juga melalui *unit test* otomatis yang menargetkan dua lapisan utama. Lapisan pertama adalah layanan *backend* yang menjembatani permintaan API dengan pemanggilan fungsi pada *ledger*. Lapisan kedua adalah *chaincode* yang merealisasikan aturan bisnis Rupiah Digital, termasuk siklus hidup peserta, KYC, dompet, transfer, dan batas transaksi. Pendekatan ini dipilih agar kebenaran aturan pada purwarupa dapat diperiksa secara lebih terisolasi sebelum hasilnya dikaitkan dengan pengujian kinerja pada BAB IV.

3.2.5.1 Unit Test Chaincode dan Backend

Unit test chaincode memverifikasi fungsi `CreateWallet` dan `Transfer` secara terisolasi menggunakan konteks transaksi tiruan (*mock*). Gambar 3.35 menunjukkan pengujian yang menolak pembuatan dompet saat tingkatan yang diminta tidak cocok dengan tingkatan hasil penurunan otomatis dari profil KYC, berkaitan dengan Gambar 3.33. Gambar 3.36 menunjukkan pengujian yang menolak `Transfer` pada Gambar 3.32 ketika dompet pengirim belum memiliki jangkar KYC yang disetujui.

```

1 func TestCreateWalletRejectsTierMismatchAndUsesPolicyDerivedTier
   (t *testing.T) {
2     sc := &SmartContract{}
3     ctx, _ := newMockTransactionContext("tx-kyc", now)
4     // ... simpan profil KYC disetujui (RiskMedium,
   DueDiligenceStandard) ...
5     err := sc.CreateWallet(ctx, "wlt-cust-1", "cust-1", string(
   TierBasic))
6     if err == nil || !strings.Contains(err.Error(),
7         "does not match policy-derived tier") {
8         t.Fatalf("create wallet mismatch err = %v, want mismatch
9         error", err)
10    }
11
12    if err := sc.CreateWallet(ctx, "wlt-cust-1", "cust-1",
13        string(TierStandard)); err != nil {
14        t.Fatalf("create wallet: %v", err)
15    }
16    wallet, _ := sc.getWallet(ctx, "wlt-cust-1")
17    if wallet.Tier != TierStandard {
18        t.Fatalf("wallet tier = %s, want %s", wallet.Tier,
19        TierStandard)
20    }
21 }

```

Gambar 3.35. Unit test penolakan pembuatan dompet saat tingkatan yang diminta tidak cocok dengan tingkatan hasil penurunan kebijakan KYC (berkas `chaincode/digital_rupiah_test.go`)

```

1 func TestTransferRequiresApprovedKycAnchor(t *testing.T) {
2     sc := &SmartContract{}
3     ctx, _ := newMockTransactionContext("tx-transfer", now)
4     // ... simpan TierLimit dan dompet pengirim/penerima BASIC
5     ...
6     _, err := sc.Transfer(ctx, "wlt-a", "wlt-b", 100_000, "
transfer-test-1")
7     if err == nil || !strings.Contains(err.Error(),
"approved KYC anchor not found") {
8         t.Fatalf("transfer err = %v, want missing KYC anchor
error", err)
9     }
10 }

```

Gambar 3.36. Unit test penolakan transfer ketika dompet pengirim belum memiliki jangkar KYC yang disetujui (berkas `chaincode/digital_rupiah_test.go`)

Pada lapisan *backend*, *unit test* memverifikasi bahwa batas kepemilikan dompet ritel ditegakkan sebelum permintaan transfer diteruskan ke *chaincode*. Gambar 3.37 menunjukkan pengujian yang menolak permintaan transfer ketika identitas JWT pemanggil tidak cocok dengan pemilik dompet pengirim, sehingga permintaan tidak pernah mencapai *chaincode*.

```

1 func TestRetailTransferRejectsForeignSenderWallet(t *testing.T)
    {
2     contract := &authorizationContract{
3         walletOwners: map[string]string{"wlt_alice": "alice"},
4     }
5     router := mux.NewRouter()
6     svc := services.NewLedgerServiceForTest(contract, nil, "")
7     New(svc, nil).RegisterRoutes(router)
8
9     req := httptest.NewRequest(http.MethodPost, "/transfers",
10         strings.NewReader(`{"sender_id":"wlt_alice","receiver_id":"
            wlt_bob","amount":"1000"}`))
11     req.Header.Set("Idempotency-Key", "transfer-foreign")
12     req = requestWithIdentity(req, middleware.RoleKycVerified, "
            mallory")
13     response := httptest.NewRecorder()
14     router.ServeHTTP(response, req)
15
16     if response.Code != http.StatusForbidden {
17         t.Fatalf("foreign-wallet transfer got %d, want 403",
            response.Code)
18     }
19     if len(contract.submissions) != 0 {
20         t.Fatalf("foreign-wallet transfer reached chaincode: %v",
            contract.submissions)
21     }

```

Gambar 3.37. Unit test penolakan transfer ketika pemilik dompet pengirim tidak cocok dengan identitas JWT pemanggil (berkas `backend/handlers/authorization_test.go`)

Verifikasi lengkap kedua lapisan dijalankan melalui `go test ./...` pada masing-masing modul. Jumlah pengujian yang lulus pada setiap lapisan dipaparkan pada Bab IV.

3.2.5.2 Konfigurasi Hyperledger Caliper

Hyperledger Caliper dikonfigurasi dengan file jaringan (*network config*) yang mendeskripsikan topologi *peer*, *orderer*, kanal, dan identitas MSP. Modul beban kerja (*workload modules*) ditulis dalam JavaScript untuk mensimulasikan transfer antarpelanggan, transfer pelanggan-ke-pedagang, pembacaan dompet, KYC, operasi

moneter, kueri pengawasan, dan kebijakan administratif sesuai Tabel 3.8.

Nilai `version`: "2.0.0" pada tingkat teratas konfigurasi jaringan menyatakan versi skema konfigurasi Caliper, bukan versi Hyperledger Fabric atau *chaincode*. Lingkungan pengujian menggunakan Hyperledger Fabric 2.5, sedangkan entri kontrak menetapkan *digital-rupiah* versi 3.0.

Lingkungan pengujian dirangkum pada Tabel 3.10. Seluruh komponen dijalankan pada satu inang lokal berbasis *container* Docker, sehingga parameter pemotongan blok *orderer* dan versi perangkat lunak menjadi bagian dari konteks yang menentukan batas *throughput* yang terukur pada BAB IV.

Tabel 3.10. Lingkungan pengujian purwarupa bi-coin

Komponen	Spesifikasi
Inang	Satu mesin lokal, <i>container</i> Docker 29.6.2
CPU	Intel Core i7-7700HQ @ 2,80 GHz (4 inti, 8 utas)
Memori (RAM)	16 GB
Sistem operasi	Fedora Linux 44 (<i>Workstation Edition</i>), x86_64
Hyperledger Fabric	2.5 (rilis 2.5.16)
Basis data status	CouchDB
Konsensus <i>ordering</i>	Raft (<i>etcdraft</i>)
<i>BatchTimeout orderer</i>	2 detik
<i>MaxMessageCount orderer</i>	10 pesan
Ukuran blok maksimum	10 MB (<i>preferred</i> 2 MB)
Kanal	<i>mychannel</i> , kebijakan dukungan mayoritas
Chaincode	<i>digital-rupiah</i> v3.0, sequence 1 (Go 1.22, <i>fabric-contract-api-go</i> v2.0.0)
Backend	Go 1.23, <i>fabric-gateway</i> v1.8.0
Hyperledger Caliper	0.6.0

3.2.5.3 Menjalankan Pengujian Hyperledger Caliper

Pengujian dijalankan dengan kontrol laju *fixed-rate*. Profil transfer ritel dijalankan untuk 1, 2, dan 4 *worker*, sedangkan profil fungsi lain dijalankan secara terpisah. Caliper mengumpulkan metrik *throughput* (TPS), latensi rata-rata, latensi maksimum, dan tingkat keberhasilan transaksi, yang kemudian dianalisis pada BAB IV.

BAB IV

HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil rancang bangun sistem Rupiah Digital ritel berbasis Hyperledger Fabric beserta pembahasannya. Bagian pertama memaparkan hasil implementasi sistem yang menjawab tujuan pertama dan kedua, bagian kedua memaparkan hasil evaluasi kinerja yang menjawab tujuan ketiga, dan bagian ketiga membandingkan hasil penelitian dengan penelitian terdahulu.

4.1 Hasil Implementasi Sistem

4.1.1 Jaringan Hyperledger Fabric

Jaringan *blockchain* berhasil dibangun dengan lima organisasi peserta (BankIndonesiaOrg, HimbaraBankOrg, CommercialBankOrg, PJPOrg, dan OJKObserverOrg) serta satu organisasi *orderer*. Setiap organisasi peserta menjalankan satu *peer* dengan basis data status CouchDB, dan layanan pengurutan menggunakan konsensus Raft (etcdraft). Seluruh komponen dijalankan sebagai *container* melalui Docker Compose. Material kriptografi, blok genesis, dan konfigurasi kanal dibangkitkan secara otomatis melalui skrip, kemudian *chaincode* digital-rupiah dipaketkan, dipasang pada seluruh organisasi, disetujui sesuai kebijakan dukungan *majority*, dan di-*commit* ke kanal. Dengan kebijakan dukungan *majority*, setiap pemutakhiran *ledger* memerlukan dukungan mayoritas organisasi sehingga tidak ada satu pihak yang dapat mengubah status secara sepihak.

4.1.2 Chaincode Digital Rupiah

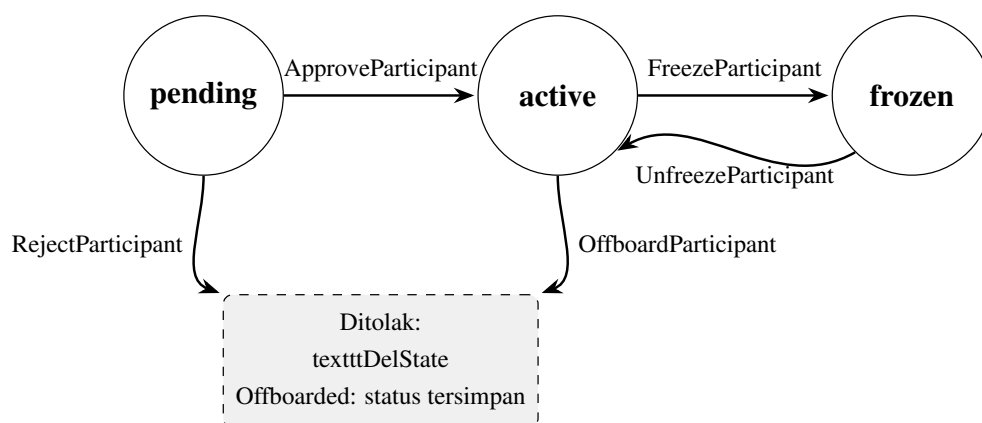
Chaincode digital-rupiah v3.0 mengimplementasikan logika bisnis Rupiah Digital pada bahasa Go. Fungsi-fungsi *chaincode* dikelompokkan menurut domainnya sebagaimana dirangkum pada Tabel 4.1. Seluruh operasi yang mengubah status *ledger* mencatatkan jejak audit dan/atau *event* pengawasan sehingga dapat ditelusuri oleh pengawas.

Tabel 4.1. Kelompok fungsi chaincode digital-rupiah

Kelompok	Fungsi utama
Peserta	SubmitParticipant, ApproveParticipant, FreezeParticipant, UnfreezeParticipant, RejectParticipant, OffboardParticipant

Kelompok	Fungsi utama
Dompot	CreateWallet, CreateWholesaleWallet, Mint, Burn, Transfer, FreezeWallet, UnfreezeWallet
Likuiditas	RequestIssuance, RequestIssuanceRtgs, DistributeToParticipant, RequestRedemption
Limit	SetTierLimit, SetSystemLimit, SetAutoLimitPolicy
KYC	SubmitKycProfile, RefreshKycProfile, GetKycProfile
Pengawasan	GetReconciliationReport, GetMetrics, GetSupervisionEvents, GetTransactionHistory

Penegakan kepatuhan dilakukan langsung pada *chaincode*. Tingkatan dompet ritel diturunkan dari jenis subjek, tingkat uji tuntas, tingkat risiko, dan persetujuan senior pada profil KYC. Operator tidak dapat memilih tingkatan. Dompot institusional tidak menggunakan tingkatan ritel dan tunduk pada batas tingkat sistem. Tabel 4.2 menunjukkan parameter kebijakan purwarupa, sedangkan Gambar 4.38 menunjukkan siklus hidup peserta dari pendaftaran hingga pelepasan keanggotaan.



Gambar 4.38. Siklus hidup peserta pada *chaincode*: dari *pending* (menunggu persetujuan) menjadi *active*, kemudian dapat dibekukan (*frozen*) dan dicairkan kembali. Penolakan menghapus peserta yang masih *pending*, sedangkan *OffboardParticipant* menyimpan status *offboarded*, membekukan dompet terkait, dan mensyaratkan saldo agregat nol.

Pada fungsi *Transfer*, sistem memeriksa status beku dompet, kelayakan KYC pengirim, batas per transaksi, pengeluaran harian, pengeluaran bulanan, penerimaan bulanan, kecukupan saldo pengirim, serta saldo maksimum penerima sebelum transaksi dieksekusi. *Transfer*, pembayaran QRIS, distribusi, dan notifikasi RTGS juga menggunakan referensi bisnis serta receipt yang dapat diverifikasi. *Transfer* memeriksa MSP kustodian dompet pengirim sebelum perubahan saldo.

Tabel 4.2. Batas tingkatan dompet ritel (dalam Rupiah)

Tingkatan	Saldo maks.	Per transaksi	Harian keluar	Bulanan keluar	Bulanan masuk
BASIC	2.000.000	250.000	500.000	5.000.000	20.000.000
STANDARD	20.000.000	2.500.000	10.000.000	40.000.000	40.000.000
MERCHANT	200.000.000	10.000.000	50.000.000	500.000.000	500.000.000

Nilai pada Tabel 4.2 adalah parameter kebijakan purwarupa untuk menguji penurunan tingkatan dan penegakan limit. Nilai tersebut tidak diturunkan dari ketentuan BI, OJK, atau FATF dan tidak membuktikan kepatuhan APU/PPT.

Selain limit per tingkatan, sistem menetapkan batas tingkat sistem berupa pasokan global maksimum Rp10.000.000.000.000, saldo maksimum per peserta Rp1.000.000.000.000, dan nilai maksimum per transaksi Rp100.000.000.000.

4.1.3 Backend API

Lapisan *backend* menyediakan antarmuka REST yang menerjemahkan permintaan HTTP menjadi pemanggilan *chaincode* melalui *Fabric Gateway SDK*. Kontrol akses berbasis peran ditegakkan melalui *middleware* dengan tujuh peran sebagaimana dirangkum pada Tabel 4.3. Pemisahan peran ini memastikan bahwa, misalnya, hanya peran *bank_indonesia* yang dapat melakukan penerbitan, sedangkan peran *supervisor* hanya memiliki akses baca terhadap laporan pengawasan. Untuk pelanggan dan pedagang, backend juga membatasi daftar dompet berdasarkan nama pengguna pada JWT dan menolak transfer apabila *owner_id* dompet pengirim tidak cocok. Pada batas jaringan, *chaincode* memeriksa MSP: mutasi moneter, limit, dan siklus hidup peserta hanya menerima MSP Bank Indonesia, sedangkan MSP pengamat tidak dapat melakukan mutasi.

Tabel 4.3. Peran akses pada backend API

Peran	Kewenangan utama
public	Pemeriksaan kesehatan dan topologi jaringan
authenticated	Pembacaan daftar dompet
kyc_verified	Transfer dan pembacaan saldo pelanggan yang telah disetujui
bank_pjp	Onboarding pelanggan, keputusan KYC, pembuatan dompet, dan pengajuan peserta
bank_indonesia	Penerbitan, distribusi/penebusan likuiditas, siklus hidup peserta, kebijakan limit, dan pengawasan; akses baca KYC
merchant	Transfer dan pembacaan saldo pedagang yang telah disetujui
supervisor	Akses baca KYC, laporan, dan pengawasan

4.1.4 Hasil Pengujian Fungsional

Pengujian fungsional dilakukan untuk memverifikasi setiap alur proses yang dirancang pada BAB III. Hasil pengujian menunjukkan bahwa seluruh alur berjalan sesuai rancangan, yaitu:

1. **Penerbitan dan distribusi berjenjang.** Bank Indonesia menerbitkan Rupiah Digital kepada peserta. Bank validator kemudian mendistribusikan likuiditas ke PJP melalui `DistributeToParticipant`, yang melakukan `Burn` pada pengirim dan `Mint` pada penerima. PJP menyalurkan saldo tersebut kepada pelanggan ritel. Upaya penerbitan langsung kepada PJP ditolak sesuai aturan sehingga PJP hanya memperoleh likuiditas melalui bank.
2. **Penegakan limit.** Transaksi yang melampaui batas per transaksi, keluar harian, keluar bulanan, atau masuk bulanan ditolak, demikian pula transfer yang menyebabkan saldo penerima melampaui batas.
3. **KYC dan pembekuan.** Bank/PJP mengajukan dan memperbarui profil KYC. BASIC diturunkan untuk pelanggan berisiko rendah dengan uji tuntas sederhana. STANDARD diberikan kepada pelanggan dengan uji tuntas standar atau lanjutan, sedangkan MERCHANT diberikan kepada pedagang dengan uji tuntas standar atau lanjutan. Risiko tinggi memerlukan EDD dan persetujuan senior. Dompet yang dibekukan tidak dapat bertransaksi.

4. **Transfer ritel.** Jalur *Transfer* yang sama berhasil digunakan untuk transaksi antarpelanggan dan pelanggan-ke-pedagang.
5. **Pengawasan.** Laporan rekonsiliasi, metrik, dan *event* pengawasan dapat diambil oleh peran berwenang.

Selain pengujian berbasis skenario tersebut, verifikasi fungsional juga diperkuat dengan *unit test* otomatis pada lapisan *backend* dan *chaincode*. Hasil verifikasi menunjukkan bahwa paket uji *backend* lulus seluruh pengujian dengan 57 pengujian pada 6 paket, sedangkan paket uji *chaincode* lulus seluruh pengujian dengan 53 pengujian pada 1 paket. Pada lapisan *backend*, pengujian tersebut memverifikasi pemilihan metode layanan ke *chaincode* beserta urutan argumennya, jalur tulis pengajuan dan pembaruan KYC, validasi persetujuan risiko tinggi, pembentukan *anchor hash*, propagasi kegagalan penyimpanan *off-chain*, serta penanganan layanan untuk pembuatan pelanggan ritel, transfer, penerbitan, penebusan, dan distribusi kepada peserta. Pada lapisan *chaincode*, pengujian memverifikasi evaluasi kebijakan kepatuhan, penurunan tier dompet, reset penghitung transaksi, aksi siklus hidup peserta, validasi siklus hidup KYC, batasan pembuatan dompet, jalur berhasil dan jalur penolakan transfer terkait status KYC, status dompet, saldo, serta limit transaksi, disertai persistensi dan pembacaan kembali limit sistem maupun pembaruan status atau *event* transaksi yang relevan. Temuan tersebut mendukung kebenaran logika layanan *backend* dan aturan bisnis *chaincode* pada purwarupa, namun tidak ditafsirkan sebagai bukti validasi menyeluruh, pembuktian *end-to-end*, atau kesiapan sistem untuk lingkungan produksi berskala penuh.

Jika dirujuk ke SRS pada Tabel 3.3, hasil pada subbab ini memberikan bukti utama untuk FR-01 sampai FR-08 dan NFR-02 sampai NFR-04. FR-01 dan FR-02 didukung oleh keberhasilan skenario pendaftaran peserta, penerbitan, dan distribusi dua jenjang. FR-03, FR-04, FR-05, dan FR-08 didukung oleh *unit test* dan skenario fungsional yang menunjukkan bahwa profil KYC, penurunan tier dompet, penolakan transaksi yang tidak patuh, serta pemisahan data KYC luring berjalan sesuai rancangan. FR-06 dan FR-07 didukung oleh skenario pengawasan dan verifikasi hak akses pada lapisan *backend*. Dengan demikian, hasil pengujian fungsional pada bab ini berperan sebagai bukti bahwa kebutuhan perangkat lunak utama purwarupa telah diimplementasikan dan diverifikasi pada tingkat yang sesuai untuk penelitian prototipe.

4.2 Hasil Evaluasi Kinerja

Profil transfer dijalankan pada host dengan spesifikasi dan perangkat lunak sebagaimana dirangkum pada Tabel 4.4. Nilai memori merupakan kapasitas yang terdeteksi oleh Linux, sedangkan versi Fabric dan CouchDB mengikuti citra yang digunakan oleh jaringan Garuda.

Tabel 4.4. Lingkungan perangkat keras dan perangkat lunak evaluasi kinerja v3

Komponen	Nilai
CPU host	Intel Core i7-7700HQ, 4 inti fisik, 8 thread, 2,80 GHz
Memori host	15,52 GiB (16.665.542.656 byte)
Sistem operasi	Linux kernel 7.1.6-201.fc44.x86_64
Node.js / npm	v22.23.2 / 10.9.8
Docker Engine / Compose	29.7.2 / 5.4.0
Caliper	caliper-cli dan caliper-fabric 0.6.0
Fabric / CouchDB	fabric-peer/orderer 2.5 / couchdb 3.2
Jaringan / kontrak	Garuda / digital-rupiah v3.0, sequence 1

Konfigurasi evaluasi kinerja v3 menggunakan Hyperledger Caliper 0.6 pada jaringan lokal lima organisasi dengan sapuan konfigurasi *worker* (1, 2, dan 4 *worker*). Setiap profil transfer menjalankan pemanasan selama 30 detik pada 10 TPS, lalu dua ronde terukur selama 120 detik pada 30 dan 60 TPS. Populasi total dijaga tetap pada 116 pelanggan, 16 pedagang, dan 52 slot transaksi untuk setiap konfigurasi *worker*. Dua puluh persen pelanggan memperoleh BASIC melalui uji tuntas sederhana, sedangkan 80% memperoleh STANDARD melalui uji tuntas standar; pedagang memperoleh MERCHANT.

Beban terukur hanya memakai jalur yang masih berada dalam lingkup penelitian: 70% transfer antarpelanggan, 25% transfer pelanggan-ke-pedagang, dan 5% pembacaan dompet. Setiap subjek diberi jangkar KYC yang disetujui sebelum ronde dimulai. Dana awal STANDARD sebesar Rp5.000.000 dan BASIC sebesar Rp500.000 menyisakan ruang penerimaan di bawah saldo maksimum. Pengukuran melaporkan jumlah berhasil dan gagal, *throughput*, serta latensi minimum, rata-rata, dan maksimum. Nomor slot global menentukan jenis transaksi secara deterministik. Ke-52 slot diputar selama ronde berbasis durasi, sehingga kunci dapat digunakan kembali setelah satu siklus. Pembagian ini menghindari satu dompet panas sebagai garis dasar, tetapi tidak menghapus seluruh kontensi. Kegagalan kebijakan dan konflik MVCC tetap dihitung dan dilaporkan jika terjadi.

Selain metrik transaksi, Caliper mengaktifkan pemantau sumber daya Docker pada sebelas kontainer jaringan. Tabel 4.5 merangkum nilai CPU dan memori maksimum dari satu kontainer dengan nilai tertinggi pada setiap profil. Kolom lalu lintas dan penulisan disk merupakan jumlah seluruh kontainer, sehingga angka tersebut menggambarkan beban jaringan dan penyimpanan yang dicatat oleh pemantau, bukan pemakaian seluruh host.

Tabel 4.5. Pemakaian sumber daya Docker pada profil transfer v3.0

Profil	Kont.	CPU maks. (%)	Memori maks. (MB)	Masuk (MB)	Keluar (MB)	Disk tulis (MB)
Pemanasan	11	1,09	123	46,1	44,94	119,63
Terukur 30 TPS	11	2,47	137	477,4	464,14	1.704,2
Terukur 60 TPS	11	2,35	174	695,9	670,09	2.038

Pengujian pada jaringan yang dibangun kembali dari keadaan bersih pada 11 Agustus 2026 menghasilkan bukti konformitas v3 untuk dua profil Caliper dan bukti kinerja untuk satu profil transfer. Nilai 2.0.0 pada akar `networkconfig.yaml` merupakan versi skema konfigurasi yang didukung konektor Fabric pada Caliper 0.6, sedangkan versi kontrak 3.0 merujuk pada *chaincode* yang dipasang. Perbedaan kedua nilai tersebut tidak menunjukkan penggunaan *chaincode* lama.

Tabel 4.6. Hasil konformitas Caliper pada *chaincode* v3.0

Profil	Hasil semantik
<i>Authorization</i>	Satu transfer oleh HimbaraBankOrgMSP terhadap dompet berkustodian BankIndonesiaOrgMSP ditolak karena ketidakcocokan MSP kustodian. Saldo pengirim tetap Rp100.000 dan saldo penerima tetap Rp0; <i>infrastructure error</i> =0 dan hasil oracle PASS.
<i>Negative-path</i>	Tujuh dari tujuh pelanggaran ditolak dengan alasan yang tepat: batas per transaksi, saldo tidak cukup, saldo maksimum penerima, dompet pengirim dibekukan, persetujuan tidak diizinkan, risiko tinggi tanpa persetujuan senior, dan KYC kedaluwarsa. <i>Enforcement gap</i> =0, <i>oracle error</i> =0, dan hasil oracle PASS.

Kedua profil konformitas menguji aturan, sedangkan profil transfer mengukur kinerja pada satu *worker*. Hasil profil transfer tersebut dirangkum pada Tabel 4.7.

Tabel 4.7. Hasil profil transfer Caliper v3.0 dengan 1 *worker*

Ronde	Sukses	Gagal	Laju (TPS)	Lat. min. (s)	Lat. avg. (s)	Lat. maks. (s)	Throughput (TPS)
Pemanasan	301	0	10	0,00	0,58	2,12	9,4
Terukur 30 TPS	3178	423	30	0,00	0,88	3,50	29,5
Terukur 60 TPS	2072	5129	60	0,01	2,77	15,23	58,7

Pada beban 30 TPS, tingkat keberhasilan adalah 88,25% dengan *throughput* 29,5 TPS. Pada 60 TPS, tingkat keberhasilan turun menjadi 28,77%, sedangkan *throughput* yang dilaporkan adalah 58,7 TPS. Log Caliper mencatat ketidakcocokan *proposal response payload* pada tahap *endorsement* dan status kode 11 pada tahap *submit*. Karena konektor tidak menyediakan read-write set *peer* untuk kasus ini, status kode 11

tidak diklasifikasikan lebih lanjut sebagai konflik MVCC. Hasil ini berasal dari satu *worker* dan satu *ledger* bersih, sehingga menjadi garis dasar lokal, bukan kapasitas nasional atau bukti reproduktibilitas lintas mesin.

Artefak tabel-tabel historis berikut menggunakan *chaincode* versi 2.0. Angka tersebut dipertahankan sebagai *baseline* pembandingan dan tidak digabungkan dengan hasil v3.0.

Konfigurasi pada Tabel 4.8 merupakan beban ritel yang menyerupai perilaku sistem CBDC ritel pada tingkat purwarupa, bukan klaim lalu lintas produksi nasional. Kesesuaiannya berasal dari tiga aspek. Pertama, transaksi yang diukur adalah transfer antarpelanggan, transfer pelanggan-ke-pedagang, pembacaan dompet, KYC, operasi moneter, kueri pengawasan, dan kebijakan administratif, sesuai fungsi pembayaran umum pada CBDC ritel [6]. Kedua, populasi pengujian memuat pelanggan BASIC dan STANDARD serta pedagang MERCHANT dengan KYC yang disetujui, mengikuti prinsip dompet bertingkat dan limit berbasis identifikasi yang juga muncul pada e-CNY dan eNaira [46, 47]. Ketiga, laju tetap 10, 30, dan 60 TPS pada profil transfer memungkinkan observasi hubungan antara *send rate*, *throughput*, latensi, dan kegagalan transaksi sebagaimana dianjurkan dalam metrik kinerja blockchain [1, 50]. Dengan batasan tersebut, hasil Caliper layak digunakan untuk menilai kelayakan desain dan membangun garis dasar pengujian lanjutan, tetapi tidak untuk mengekstrapolasi kapasitas nasional tanpa data transaksi riil, topologi produksi, dan perangkat keras setara.

Dalam konteks SRS pada Bab III, evaluasi kinerja ini terutama berfungsi sebagai bukti verifikasi untuk NFR-05, yaitu bahwa purwarupa dapat diukur secara kuantitatif dengan metrik *throughput*, latensi, dan tingkat keberhasilan transaksi. Oleh karena itu, angka Caliper pada subbab ini tidak ditafsirkan sebagai pengganti kebutuhan fungsional atau bukti keamanan formal, melainkan sebagai ukuran performa purwarupa pada lingkungan uji yang telah didefinisikan.

Tabel 4.8. Konfigurasi evaluasi transfer ritel

Profil	Worker	Durasi/jumlah Laju		Komposisi
Transfer	1/2/4	30 detik	10 TPS	70% P2P, 25% pelanggan–pedagang, 5% baca
Transfer	1/2/4	120 detik	30 TPS	70% P2P, 25% pelanggan–pedagang, 5% baca
Transfer	1/2/4	120 detik	60 TPS	70% P2P, 25% pelanggan–pedagang, 5% baca
Pengulangan puncak	2	5 × 60 detik	60 TPS	Transfer ritel identik untuk rerata dan <i>confidence interval</i>
Fungsional	2	40–80	5–30 TPS	KYC, moneter, pengawasan, administratif
<i>Negative-path</i>	1	7	5 TPS	Tujuh pelanggaran harus ditolak dengan alasan yang tepat
Agregat <i>overspend</i>	4	200	80 TPS	Permintaan Rp200.000 dari saldo awal Rp50.000

Seluruh angka historis pada subbab ini berasal dari batch 2026-07-27-214918 tanggal 27 Juli 2026. Tujuh profil pada berkas status berakhir PASS dan setiap profil memiliki laporan baru. Manifes batch mencatat commit dasar f4b0910ecc69, status repositori yang masih memuat perubahan, *seed* 20260725, versi alat, dan hash seluruh masukan.

Tabel 4.9. Baseline historis v2: hasil transfer dasar pada ronde terukur

Worker	Beban	Sukses	Gagal	Sukses (%)	Selesai (TPS)	Sukses (TPS)
1	30 TPS	3.601	0	100,00	29,5	29,5
1	60 TPS	5.037	2.164	69,95	58,9	41,2
2	30 TPS	3.602	0	100,00	29,5	29,5
2	60 TPS	5.748	1.454	79,81	58,9	47,0
4	30 TPS	3.604	0	100,00	29,5	29,5
4	60 TPS	5.603	1.601	77,78	58,9	45,8

Pada baseline historis v2, kolom “Selesai” pada Tabel 4.9 adalah *throughput* yang dilaporkan Caliper dan mencakup transaksi sukses serta gagal. Kolom “Sukses (TPS)” dihitung dengan mengalikan nilai tersebut dengan proporsi transaksi sukses. Pada 30 TPS, ketiga konfigurasi mencapai 29,5 TPS dengan tingkat sukses 100%. Pada 60 TPS, *completion throughput* tetap 58,9 TPS, tetapi *successful throughput* hanya 41,2–47,0 TPS. Kegagalan didominasi ketidakcocokan *proposal response payload* saat beberapa *peer* membaca versi status yang berbeda. Satu observasi W1 dan W4 tidak cukup untuk menyimpulkan pengaruh kausal jumlah *worker*.

Tabel 4.10. Baseline historis v2: hasil lima ronde pengulangan beban 60 TPS dengan 2 worker

Ronde	Sukses	Gagal	Sukses (%)	Selesai (TPS)	Sukses (TPS)
1	2.342	1.256	65,09	57,7	37,6
2	1.494	2.105	41,51	57,5	23,9
3	1.885	1.713	52,39	59,2	31,0
4	1.671	1.931	46,39	59,5	27,6
5	2.327	1.275	64,60	57,7	37,3

Pada baseline historis v2, rerata *successful throughput* lima ronde adalah 31,46 TPS dengan interval kepercayaan 95% untuk rerata sebesar $\pm 7,44$ TPS. Rerata tingkat sukses adalah 54,00% dengan interval $\pm 13,20$ poin persentase, sedangkan rerata latensi adalah 1,44 detik dengan interval $\pm 0,43$ detik. Interval ini mengukur ketidakpastian estimasi rerata, bukan pita yang harus memuat seluruh observasi. Rentang *successful throughput* 23,87–37,56 TPS menunjukkan variasi yang besar pada beban 60 TPS. Lima ronde menggunakan ruang nama berbeda pada satu *ledger*, sehingga hasilnya merupakan *repeatability* dalam satu lingkungan, bukan reproduksibilitas lintas mesin.

Tabel 4.11. Baseline historis v2: hasil profil fungsional

Profil	Sukses	Gagal	Throughput (TPS)	Latensi rata-rata (detik)
KYC/onboarding	56	0	3,8	0,85
Operasi moneter	80	0	19,7	0,33
Kueri pengawasan	80	0	30,7	0,04
Kebijakan administratif	40	0	5,2	0,90

Keempat profil fungsional selesai tanpa kegagalan. Angka tersebut tidak dibandingkan langsung karena setiap profil memiliki fungsi, jumlah transaksi, dan pola baca-tulis yang berbeda.

Tabel 4.12. Baseline historis v2: hasil oracle jalur negatif dan pengeluaran agregat

Profil	Hasil semantik
<i>Negative-path</i>	Tujuh dari tujuh transaksi ditolak dengan alasan yang tepat; <i>enforcement gap</i> =0 dan <i>oracle error</i> =0.
Pengeluaran agregat	Empat dari 200 transfer dikomit dan 196 ditolak. Total permintaan Rp200.000 melebihi saldo awal Rp50.000; saldo akhir Rp46.000 dan tetap nonnegatif pada seluruh worker.

Profil *negative-path* memvalidasi alasan penolakan dari detail galat *peer* pada log, bukan dari status generik konektor. Profil pengeluaran agregat menunjukkan bahwa

permintaan bersamaan tidak membuat saldo negatif. Konektor Caliper menghapus diagnostik *peer* dari objek status yang dikembalikan, sehingga 196 penolakan pada ringkasan *workload* tidak diklaim sebagai konflik MVCC tertentu. Bukti ini mendukung invariant saldo pada topologi yang diuji, bukan pembuktian formal pencegahan *double-spend*.

4.3 Perbandingan Hasil Penelitian dengan Hasil Terdahulu

Penelitian ini dibandingkan dengan tinjauan literatur Aryani [8] dan PoC Proyek Garuda [12] karena keduanya membahas kebutuhan CBDC Indonesia dari sisi kebijakan dan pembuktian teknologi. Tabel 4.13 membandingkan jenis artefak, lingkup bukti, dan posisi kontribusi. Jika angka kinerja disebutkan, angka tersebut hanya menjelaskan konfigurasi masing-masing artefak dan tidak dimaksudkan sebagai peringkat langsung antarsistem. Dalam kerangka itu, penelitian ini mengisi ruang purwarupa CBDC ritel berbasis Hyperledger Fabric, sedangkan Aryani berfokus pada kajian literatur dan PoC Proyek Garuda pada Rupiah Digital grosir.

Tabel 4.13. Perbandingan penelitian ini dengan tinjauan literatur Aryani (2023) dan PoC Proyek Garuda (2024).

Dimensi	Aryani (2023)	PoC Garuda (2024)	Penelitian ini
Jenis CBDC	Ritel (analisis konseptual)	Grosir (PoC eksperimental)	Ritel (purwarupa)
Metodologi	Tinjauan literatur kritis komparatif	PoC dengan 55 skenario bisnis dan teknis	Purwarupa fungsional + Caliper
Platform DLT	Tidak dievaluasi (kutipan dari kasus negara)	R3 Corda + Hyperledger Besu	Hyperledger Fabric
Dompot berjenjang	Direkomendasikan (eNaira 5 tingkat, Sand Dollar 2, e-CNY berjenjang)	Tidak ada (grosir saja)	BASIC, STANDARD, MERCHANT (3 tingkat)
KYC	Direkomendasikan bertingkat	Tidak dibahas eksplisit	Luring, jangkar kepatuhan, penurunan berbasis risiko
Metrik kinerja	Kutipan sekunder; tidak diukur langsung	Corda 37 TPS, Besu 39 TPS (Locust)	v3.0: 29,5 TPS pada 30 TPS dan 58,7 TPS pada 60 TPS (1 <i>worker</i>); baseline v2 terpisah
Lingkup pengujian	Tidak ada pengujian	Penerbitan, penebusan, transfer antarlembaga	Transfer antarpelanggan, pelanggan-pedagang, baca dompet
Pembayaran luring	Direkomendasikan untuk r-Digital Rupiah	Tidak dibahas	Tidak diuji dalam purwarupa ini
Tokenisasi	Didukung (konseptual)	UTXO (Corda) / ERC-20 + ERC-1400 (Besu)	Transfer berbasis kunci-nilai pada Fabric

Perbandingan ini terbatas pada posisi kontribusi dan ruang lingkup teknis. Profil v3.0 yang tersedia hanya satu *worker*, sehingga hasil historis v2 tetap dipisahkan dan tidak digunakan untuk menetapkan keunggulan atau kapasitas nasional karena topologi, jumlah peserta, dan skenario beban masih berada pada skala purwarupa. Selisih angka

kinerja terhadap PoC Proyek Garuda (Corda 37 TPS dan Besu 39 TPS) bersumber pada perbedaan platform, *harness*, tujuan pengujian, dan komposisi beban, bukan pada klaim keunggulan langsung Hyperledger Fabric. Variasi tingkat sukses pada lima ronde beban 60 TPS menunjukkan bahwa hasil ini merupakan garis dasar lokal yang dapat diulang, bukan kapasitas maksimum Hyperledger Fabric atau bukti reproduksibilitas lintas mesin [32].

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil rancang bangun dan pengujian, kesimpulan penelitian adalah sebagai berikut.

1. Berdasarkan rumusan masalah pertama mengenai perancangan sistem Rupiah Digital ritel berbasis Hyperledger Fabric, penelitian ini berhasil merancang dan membangun purwarupa dengan arsitektur dua jenjang yang memisahkan peran Bank Indonesia, bank validator, PJP, dan pengguna ritel. Implementasi dijalankan pada Hyperledger Fabric 2.5 dengan lima organisasi, *backend* Go, CouchDB, dan PostgreSQL, sehingga alur penerbitan dan distribusi likuiditas berjenjang dapat direalisasikan sebagai artefak teknis yang dapat diuji.
2. Berdasarkan rumusan masalah kedua mengenai kontrol kepatuhan pada lapisan *chaincode*, penelitian ini menunjukkan bahwa status dan risiko KYC yang dipasok bank/PJP, penurunan tingkatan dompet, limit transaksi, serta batas peran organisasi dapat divalidasi sesuai aturan purwarupa. Data identitas mentah tetap berada di PostgreSQL, sedangkan *chaincode* menerima jangkar kepatuhan. Cakupan tersebut belum membuktikan kepatuhan APU/PPT menyeluruh, persetujuan empat mata, atau privasi transaksi pada kanal bersama. Pada pengujian ulang v3.0, satu transfer dengan MSP pemanggil yang tidak sesuai dengan MSP kustodian ditolak tanpa perubahan saldo. Tujuh dari tujuh kasus *negative-path* juga ditolak dengan alasan yang tepat, dengan *enforcement gap*=0 dan *oracle error*=0. Hasil tersebut berlaku pada kasus dan topologi yang diuji, bukan sebagai pembuktian formal keamanan.
3. Berdasarkan rumusan masalah ketiga mengenai kinerja sistem, profil transfer Caliper v3.0 dengan satu *worker* menghasilkan *throughput* 29,5 TPS pada laju 30 TPS dan 58,7 TPS pada laju 60 TPS. Tingkat keberhasilannya masing-masing 88,25% dan 28,77%. Hasil ini hanya berasal dari satu *ledger* bersih dan satu konfigurasi *worker*, sehingga belum menetapkan kapasitas nasional atau reproduktibilitas lintas mesin. Batch historis v2 tetap dicatat pada Bab IV sebagai *baseline* terpisah dan tidak digunakan untuk menyimpulkan kinerja kontrak v3.

5.2 Saran

1. Mengulang protokol Caliper pada beberapa perangkat keras, topologi jaringan, dan jumlah *worker* yang lebih luas, lalu melaporkan konfigurasi, versi *chaincode*, jumlah sukses/gagal, latensi, *throughput*, konflik MVCC, dan pengulangan setiap skenario bersama hasilnya.

2. Menelusuri ketidakcocokan *proposal response payload* pada beban 60 TPS dan memisahkan pengaruh propagasi status, kebijakan *endorsement*, kontensi kunci, dan jalur submit *gateway*.
3. Memperluas *repeatability run* ke profil KYC, operasi moneter, kueri pengawasan, dan kebijakan administratif agar setiap profil tidak hanya memiliki satu angka *throughput*, tetapi juga simpangan, interval kepercayaan, dan latensi persentil ke-95 serta ke-99.
4. Mengembangkan skenario *hot-wallet contention* dari jejak transaksi yang dianonimkan atau distribusi sintetis yang parameternya dilaporkan eksplisit, lalu mengukur konflik MVCC awal, keberhasilan setelah *retry*, *throughput* akhir, dan pengaruh kebijakan *backoff* terhadap idempotensi transaksi.
5. Memvalidasi angka limit dan distribusi tingkatan bersama Bank Indonesia, bank/PJP, serta ahli APU/PPT. Nilai yang digunakan dalam penelitian ini merupakan parameter purwarupa dan tidak boleh diperlakukan sebagai ketentuan CBDC.
6. Mengintegrasikan penyedia eKYC nyata dan menerapkan kontrol retensi, enkripsi, penghapusan, serta audit akses untuk data identitas di PostgreSQL.
7. Menguji *private data collection* atau teknik kriptografi penjaga privasi untuk mengurangi keterlihatan metadata transaksi tanpa menghilangkan kemampuan pengawasan [23,35,36].
8. Mengembangkan integrasi, interoperabilitas, dan interkoneksi dengan infrastruktur pasar keuangan pada penelitian terpisah setelah kebijakan buku besar ritel tervalidasi [5].
9. Mengembangkan pembayaran tanpa konektivitas sebagai lingkup terpisah yang mencakup perangkat aman, pencegahan *double-spending*, dan rekonsiliasi sesuai Project Polaris [33,34].

DAFTAR PUSTAKA

- [1] Hyperledger Foundation, “Hyperledger caliper: A blockchain performance benchmark framework,” 2024, accessed: Jun. 27, 2026. [Online]. Available: <https://hyperledger.github.io/caliper/>
- [2] T. T. A. Dinh, J. Wang, G. Chen, R. Liu, B. C. Ooi, and K.-L. Tan, “BLOCKBENCH: A framework for analyzing private blockchains,” in *Proceedings of the 2017 ACM International Conference on Management of Data*. ACM, 2017, pp. 1085–1100.
- [3] F. Syarifuddin and T. Bakhtiar, “Monetary policy strategy in the presence of central bank digital currency,” Bank Indonesia, Working Paper WP/09/2021, 2021.
- [4] Hyperledger Foundation, “Couchdb as the state database,” 2024, accessed: Jun. 28, 2026. [Online]. Available: https://hyperledger-fabric.readthedocs.io/en/latest/couchdb_as_state_database.html
- [5] Bank Indonesia, “Proyek garuda: Menavigasi arsitektur digital rupiah — white paper central bank digital currency,” Bank Indonesia, Jakarta, Tech. Rep., 2022.
- [6] R. A. Auer and R. Böhme, “The technology of retail central bank digital currency,” *BIS Quarterly Review*, pp. 85–100, Mar. 2020. [Online]. Available: https://www.bis.org/publ/qtrpdf/r_qt2003j.htm
- [7] D. Urbančok, “Blockchain open-source software comparison,” Master’s thesis, Masaryk University, Faculty of Informatics, Brno, 2019.
- [8] J. N. Aryani, “Designing central bank digital currency in indonesia: A critical literature review,” 2023, tugas Akhir Sarjana, Sekolah Bisnis dan Manajemen, Institut Teknologi Bandung.
- [9] Bank Indonesia, “Blueprint sistem pembayaran indonesia 2025,” Bank Indonesia, Jakarta, Tech. Rep., 2019.
- [10] R. A. Auer, J. Frost, L. Gambacorta, C. Monnet, T. Rice, and H. Shin, “Central bank digital currencies: motives, economic implications and the research frontier,” *SSRN Electronic Journal*, 2022.
- [11] L. Dionysopoulos, M. Marra, and A. Urquhart, “Central bank digital currencies: A critical review,” *International Review of Financial Analysis*, vol. 91, p. 103031, 2024.
- [12] Bank Indonesia, “Proyek garuda: Wholesale digital rupiah cash ledger — proof of concept report,” Bank Indonesia, Jakarta, Tech. Rep., 2024.
- [13] —, “Consultative paper pengembangan rupiah digital: Tahap immediate state,” Bank Indonesia, Jakarta, Tech. Rep., 2023.
- [14] T. Zhang and Z. Huang, “Blockchain and central bank digital currency,” *ICT Express*, vol. 8, no. 2, pp. 264–270, 2022.

- [15] V. Sethaput and S. Innet, “Sok: Blockchain applications in central bank digital currencies (cbdc),” in *2025 7th International Conference on Blockchain Computing and Applications (BCCA)*. IEEE, 2025, pp. 195–202.
- [16] Bhawana and S. Kumar, “Permission blockchain network based central bank digital currency,” in *2021 IEEE 4th International Conference on Computing, Power and Communication Technologies (GUCON)*. IEEE, 2021, pp. 1–6.
- [17] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich *et al.*, “Hyperledger fabric: A distributed operating system for permissioned blockchains,” in *Proceedings of the Thirteenth EuroSys Conference (EuroSys '18)*. Porto, Portugal: ACM, 2018, pp. 1–15.
- [18] J. Barrdear and M. Kumhof, “The macroeconomics of central bank digital currencies,” *Journal of Economic Dynamics and Control*, p. 104148, 2021.
- [19] J. Kiff, J. Alwazir, S. Davidovic, A. Farias, A. Khan, T. Khiaonarong, M. Malaika, H. Monroe, N. Sugimoto, H. Tourpe, and P. X. Zhou, “A survey of research on retail central bank digital currency,” *IMF Working Papers*, 2020.
- [20] S. Allen, S. Čapkun, I. Eyal, G. Fanti, B. Ford, J. Grimmelmann, A. Juels, K. Kostianen, S. Meiklejohn, A. K. Miller, E. Prasad, K. Wüst, and F. Zhang, “Design choices for central bank digital currency: Policy and technical considerations,” *SSRN Electronic Journal*, 2020.
- [21] A. Tsareva and M. Komarov, “Retail central bank digital currency design choices: Guide for policymakers,” *IEEE Access*, vol. 12, pp. 66 129–66 146, 2024.
- [22] M. M. Islam and H. P. In, “A privacy-preserving transparent central bank digital currency system based on consortium blockchain and unspent transaction outputs,” *IEEE Transactions on Services Computing*, vol. 16, no. 4, pp. 2372–2386, 2023.
- [23] N. Pocher and A. Veneris, “Privacy and transparency in cbdc: A regulation-by-design aml/cft scheme,” *IEEE Transactions on Network and Service Management*, vol. 19, pp. 1776–1788, 2022.
- [24] P. Michalopoulos, O. Olowookere, N. Pocher, J. Sedlmeir, A. G. Veneris, and P. Puri, “Privacy and compliance design options in offline central bank digital currencies,” *IEEE Transactions on Network and Service Management*, vol. 22, pp. 3748–3763, 2025.
- [25] J. Hans, S. Khan, and D. Svetinovic, “Blockchain cbdc security threats using stride,” in *2023 Fifth International Conference on Blockchain Computing and Applications (BCCA)*. IEEE, 2023, pp. 522–529.
- [26] H. Sun, H. Mao, X. Bai, Z. Chen, K. Hu, and W. Yu, “Multi-blockchain model for central bank digital currency,” in *2017 18th International Conference on Parallel and Distributed Computing, Applications and Technologies (PDCAT)*. IEEE, 2017, pp. 360–367.

- [27] B. M. Zams, R. Indrastuti, A. G. Pangrsa, N. A. Hasniawati, F. A. Zahra, and I. A. Fauziah, “Designing central bank digital currency for indonesia: The delphi–analytic network process,” *Bulletin of Monetary Economics and Banking*, vol. 23, no. 3, pp. 413–440, 2020.
- [28] F. Syarifuddin, “Optimal central bank digital currency design for emerging economies,” *Journal of Central Banking Law and Institutions*, 2024.
- [29] J. Ree, “Nigeria’s enaira, one year after,” International Monetary Fund, IMF Working Paper WP/23/104, 2023.
- [30] N. Bhensook, T. Rattanakul, W. Synsatayakul, and P. Tohwisessuk, “Public use and distribution of retail cbdc: An evidence from thailand’s retail cbdc pilot program,” *Jahrbücher für Nationalökonomie und Statistik*, vol. 245, pp. 367 – 399, 2025.
- [31] J. Lovejoy, C. Fields, M. Virza, T. Frederick, D. Urness, K. Karwaski, A. Brownworth, and N. Narula, “A high performance payment processing system designed for central bank digital currencies,” *IACR Cryptology ePrint Archive*, vol. 2022, p. 163, 2022. [Online]. Available: <https://eprint.iacr.org/2022/163>
- [32] M. Kuzlu, M. Pipattanasomporn, L. Gurses, and S. Rahman, “Performance analysis of a hyperledger fabric blockchain framework: Throughput, latency and scalability,” in *2019 IEEE International Conference on Blockchain (Blockchain)*. IEEE, 2019, pp. 536–540.
- [33] Bank for International Settlements, “Project polaris part 2: A security and resilience framework for cbdc systems,” BIS Innovation Hub, Tech. Rep., 2023.
- [34] —, “Project polaris part 1: A handbook for offline payments with cbdc,” BIS Innovation Hub, Tech. Rep., 2023.
- [35] G. Soana and T. de Arruda, “Central bank digital currencies and financial integrity: finding a new trade-off between privacy and traceability within a changing financial architecture,” *Journal of Banking Regulation*, vol. 25, pp. 467 – 486, 2024.
- [36] J. E. Said and J. Lánský, “Designing retail central bank digital currencies: A systematic literature review of trade-offs between security, privacy, and financial stability,” *International Journal of Financial Studies*, 2026.
- [37] M. Kakebayashi, G. P. Presto, T. Yuyama, and S. Matsuo, “Policy design of retail central bank digital currencies: Embedding aml/cft compliance,” *SSRN Electronic Journal*, 2023.
- [38] Y.-R. Wang, C.-Q. Ma, and Y.-S. Ren, “A model for cbdc audits based on blockchain technology: Learning from the dcep,” *Research in International Business and Finance*, vol. 63, p. 101781, 2022.
- [39] O. Atangana, L. Khoukhi, M. Barbier, and A. Kokcam, “Block-pad: A blockchain-enabled framework for resilient and flexible cbdc transactions leveraging digital identity,” *Computer Networks*, vol. 275, p. 111805, 2026.

- [40] J. E. Abang, H. Takruri, R. Al-Zaidi, and M. Al-Khalidi, "Latency performance modelling in hyperledger fabric blockchain: Challenges and directions with an iot perspective," *Internet of Things*, vol. 26, p. 101217, 2024.
- [41] A. Lohachab, S. Garg, B. H. Kang, and M. B. Amin, "Performance evaluation of hyperledger fabric-enabled framework for pervasive peer-to-peer energy trading in smart cyber–physical systems," *Future Generation Computer Systems*, vol. 118, pp. 392–416, 2021.
- [42] M. Ajwalia and P. Shah, "Performance comparison of permissioned and permissionless blockchain by varying workload transaction," *BenchCouncil Transactions on Benchmarks, Standards and Evaluations*, vol. 5, no. 4, p. 100251, 2025.
- [43] C. Beer, S. Zingg, K. Kostianen, K. Wüst, V. Capkun, and S. Čapkun, "Payoff: A regulated central bank digital currency with private offline payments," 2024. [Online]. Available: <https://arxiv.org/abs/2408.06956>
- [44] H. Arslanian, "Retail central bank digital currencies," in *The Book of Crypto*. Springer International Publishing, 2022, pp. 203–232.
- [45] M. L. Bech and R. Garratt, "Central bank cryptocurrencies," *BIS Quarterly Review*, pp. 55–70, Sep. 2017. [Online]. Available: https://www.bis.org/publ/qtrpdf/r_qt1709f.htm
- [46] Central Bank of Nigeria, "Design paper for the enaira," Central Bank of Nigeria, Tech. Rep., 2021. [Online]. Available: https://enaira.gov.ng/wp-content/uploads/2023/06/Design-Paper-for-Nigerias-CBDC-02_Oct-2021.pdf
- [47] People's Bank of China, "Progress of research and development of e-cny in china," People's Bank of China, Tech. Rep., 2021. [Online]. Available: <https://www.pbc.gov.cn/en/3688110/3688172/4157443/4293696/2021072014364791207.pdf>
- [48] W. Xiong, X. Yao, and X. Deng, "Research on the performance analysis of mainstream consensus algorithms of consortium blockchains," in *2024 2nd International Conference on Mobile Internet, Cloud Computing and Information Security (MICCIS)*. IEEE, 2024.
- [49] S. Das, M. Mishra, and R. Priyadarshini, "Hyperledger fabric: The enterprise-grade blockchain for modern business applications," in *Fostering Machine Learning and IoT for Blockchain Technology*. Springer Nature Singapore, 2025, pp. 163–188.
- [50] Hyperledger Performance and Scale Working Group, "Hyperledger blockchain performance metrics," Hyperledger Foundation, Tech. Rep., 2018.
- [51] Financial Action Task Force, "Guidance on digital identity," FATF, Paris, France, Tech. Rep., 2020. [Online]. Available: <https://www.fatf-gafi.org/content/dam/fatf-gafi/guidance/Guidance-on-Digital-Identity.pdf>
- [52] Bank Indonesia, "Peraturan bank indonesia nomor 10 tahun 2024 tentang penerapan anti pencucian uang, pencegahan pendanaan terorisme, dan pencegahan pendanaan proliferasi senjata pemusnah massal bagi pihak yang diatur dan diawasi

- oleh bank indonesia,” 2024. [Online]. Available: https://www.bi.go.id/id/publikasi/peraturan/Pages/PBI_102024.aspx
- [53] I. Agur, A. Ari, and G. Dell’Ariccia, “Designing central bank digital currencies,” *Journal of Monetary Economics*, vol. 125, pp. 62–79, 2022. [Online]. Available: <https://doi.org/10.1016/j.jmoneco.2021.05.002>
- [54] Bank Indonesia, “Peraturan anggota dewan gubernur nomor 24/7/padg/2022,” 2022, ketentuan penyelenggaraan uang elektronik; digunakan sebagai proksi kebijakan untuk parameter purwarupa, bukan sebagai dasar hukum limit CBDC. [Online]. Available: https://www.bi.go.id/id/publikasi/peraturan/Pages/PADG_240722.aspx
- [55] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000.
- [56] A. A. A. Donovan and B. W. Kernighan, *The Go Programming Language*. New York: Addison-Wesley Professional, 2015.
- [57] C. Saraf and S. Sabadra, “Blockchain platforms: A compendium,” in *2018 IEEE International Conference on Innovative Research and Development (ICIRD)*. IEEE, 2018, pp. 1–6.
- [58] Central Bank of The Bahamas, “Project sand dollar: A digital fiat currency for the bahamas,” Central Bank of The Bahamas, Nassau, Tech. Rep., 2020. [Online]. Available: <https://www.centralbankbahamas.com/viewPDF/documents/2019-12-25-02-18-11-Project-Sanddollar.pdf>
- [59] Sveriges Riksbank, “E-krona pilot — phase 1 and phase 2 technical reports,” Sveriges Riksbank, Stockholm, Tech. Rep., 2021. [Online]. Available: <https://www.riksbank.se/en-gb/payments--cash/e-krona/technical-solution-for-the-e-krona-pilot/>
- [60] I. Systems. (2024) Nosql databases in financial services: Modernizing banking and insurance data management. [Online]. Available: <https://www.ispirer.com/blog/nosql-databases-financial-services>
- [61] PostgreSQL Global Development Group. (2025) Postgresql 18 documentation: Json types, row security policies, and mvcc concurrency control. PostgreSQL Global Development Group. [Online]. Available: <https://www.postgresql.org/docs/current/>
- [62] LogicMonitor. (2025) Postgresql vs. mysql: A comprehensive comparison. [Online]. Available: <https://www.logicmonitor.com/blog/postgresql-vs-mysql>
- [63] Hyperledger Foundation. (2025) Hyperledger fabric documentation: Private data collections. Linux Foundation. [Online]. Available: <https://hyperledger-fabric.readthedocs.io/en/latest/private-data/private-data.html>

LAMPIRAN

Lampiran ini memuat kode sumber inti sistem Rupiah Digital ritel yang dibangun pada penelitian ini, meliputi fungsi-fungsi utama *chaincode* *digital-rupiah* serta modul konfigurasi dan *workload* pengujian Hyperledger Caliper. Versi lampiran ini diselaraskan dengan suite benchmark yang digunakan pada pembahasan hasil final, sehingga mencakup konfigurasi transfer dasar 1/2/4 *worker*, *transfer-repeat*, *functionality*, *negative-path*, dan agregat *overspend*. Lampiran hanya memuat artefak yang langsung mendukung argumentasi tesis. Berkas hasil run (*log*, *HTML report*, dan *JSON summary*), konfigurasi Docker/Fabric standar, serta lapisan *backend* REST API tidak ditampilkan ulang di sini karena bukan bagian inti dari kontribusi benchmark yang dibahas pada Bab III dan Bab IV. Kode sumber lengkap tetap tersedia pada repositori sistem.

L.1 Source Code Tipe Data dan Struktur *Chaincode*

Bagian ini memuat kode sumber definisi paket, impor, serta tipe data dan struktur utama (berkas `chaincode/digital_rupiah.go`).

```
1 package main
2
3 import (
4     "encoding/json"
5     "fmt"
6     "time"
7
8     "github.com/hyperledger/fabric-contract-api-go/v2/contractapi"
9 )
10
11 type SmartContract struct {
12     contractapi.Contract
13 }
14
15 // --- Enums -----
16
17 type WalletTier string
18
19 const (
20     TierBasic      WalletTier = "BASIC"
21     TierStandard   WalletTier = "STANDARD"
22     TierMerchant   WalletTier = "MERCHANT"
23 )
24
25 type WalletType string
26
27 const (
28     WalletHot   WalletType = "hot"
29     WalletCold  WalletType = "cold"
30 )
31
32 type ParticipantType string
33
34 const (
35     ParticipantValidator ParticipantType = "validator"
36     ParticipantObserver  ParticipantType = "observer"
37     ParticipantPJP        ParticipantType = "pjp"
38 )
39
40 type ParticipantStatus string
41
42 const (
43     ParticipantPending ParticipantStatus = "pending"
44     ParticipantActive   ParticipantStatus = "active"
45     ParticipantFrozen   ParticipantStatus = "frozen"
46 )
47
48 type KycStatus string
49
50 const (
51     KycPending    KycStatus = "pending"
52     KycInReview   KycStatus = "in_review"
53     KycApproved   KycStatus = "approved"
54     KycRejected   KycStatus = "rejected"
```

```

55     KycExpired    KycStatus = "expired"
56     KycSuspended KycStatus = "suspended"
57 )
58
59 type KycRiskLevel string
60
61 const (
62     RiskLow      KycRiskLevel = "low"
63     RiskMedium   KycRiskLevel = "medium"
64     RiskHigh     KycRiskLevel = "high"
65     RiskProhibited KycRiskLevel = "prohibited"
66 )
67
68 type KycSubjectType string
69
70 const (
71     SubjectParticipant    KycSubjectType = "participant"
72     SubjectRetailCustomer KycSubjectType = "retail_customer"
73     SubjectMerchant       KycSubjectType = "merchant"
74 )
75
76 type DueDiligenceLevel string
77
78 const (
79     DueDiligenceSimplified DueDiligenceLevel = "simplified"
80     DueDiligenceStandard   DueDiligenceLevel = "standard"
81     DueDiligenceEnhanced   DueDiligenceLevel = "enhanced"
82 )
83
84 type LimitScope string
85
86 const (
87     ScopeGlobalSupply      LimitScope = "global_supply"
88     ScopePerParticipantBalance LimitScope = "per_participant_balance"
89     ScopePerTxAmount        LimitScope = "per_tx_amount"
90     ScopeMinParticipantBalance LimitScope = "min_participant_balance"
91 )
92
93 type TransactionType string
94
95 const (
96     TxMint      TransactionType = "issuance"
97     TxBurn      TransactionType = "redemption"
98     TxTransfer  TransactionType = "transfer"
99 )
100
101 type TransactionStatus string
102
103 const (
104     TxSettled TransactionStatus = "settled"
105     TxRejected TransactionStatus = "rejected"
106 )
107
108 // --- State Objects -----
109
110 type Participant struct {
111     ParticipantID string    `json:"participant_id"`
112     Name           string    `json:"name"`
113     Domain         string    `json:"domain"`

```

```

114 AccountID          string          `json:"account_id"`
115 ParticipantType    ParticipantType `json:"participant_type"`
116 InitialReserveBalance string        `json:"initial_reserve_balance"`
117 ComplianceStatus   string        `json:"compliance_status"`
118 Status             ParticipantStatus `json:"status"`
119 CreatedAt          string        `json:"created_at"`
120 UpdatedAt          string        `json:"updated_at"`
121 }
122
123 type Wallet struct {
124     WalletID          string          `json:"wallet_id"`
125     OwnerID           string          `json:"owner_id"`
126     ParticipantID     string          `json:"participant_id"`
127     Tier              WalletTier      `json:"tier"`
128     WalletType        WalletType      `json:"wallet_type"`
129     Balance           int64          `json:"balance"`
130     Frozen            bool           `json:"frozen"`
131     DailySpent        int64          `json:"daily_spent"`
132     MonthlySpent      int64          `json:"monthly_spent"`
133     MonthlyReceived   int64          `json:"monthly_received"`
134     LastResetDay      string          `json:"last_reset_day"`
135     LastResetMonth    string          `json:"last_reset_month"`
136     CreatedAt         string          `json:"created_at"`
137     UpdatedAt         string          `json:"updated_at"`
138 }
139
140 type OldWallet struct {
141     OwnerID           string          `json:"owner_id"`
142     Tier              WalletTier      `json:"tier"`
143     Balance           int64          `json:"balance"`
144     Frozen            bool           `json:"frozen"`
145     DailySpent        int64          `json:"daily_spent"`
146     MonthlySpent      int64          `json:"monthly_spent"`
147     LastResetDay      string          `json:"last_reset_day"`
148     CreatedAt         string          `json:"created_at"`
149     UpdatedAt         string          `json:"updated_at"`
150 }
151
152 type TierLimit struct {
153     Tier              WalletTier      `json:"tier"`
154     MaxBalance        int64          `json:"max_balance"`
155     MinBalance        int64          `json:"min_balance"`
156     DailyTxLimit      int64          `json:"daily_tx_limit"`
157     MonthlyTxLimit    int64          `json:"monthly_tx_limit"`
158     MonthlyIncomingLimit int64          `json:"monthly_incoming_limit"`
159     PerTxLimit        int64          `json:"per_tx_limit"`
160 }
161
162 type KycProfile struct {
163     ProfileID         string          `json:"profile_id"`
164     SubjectType       KycSubjectType `json:"subject_type"`
165     SubjectID         string          `json:"subject_id"`
166     DocumentHashes    []string        `json:"document_hashes"`
167     Status            KycStatus      `json:"status"`
168     RiskLevel         KycRiskLevel   `json:"risk_level"`
169     DueDiligenceLevel DueDiligenceLevel `json:"due_diligence_level"`
170     SeniorApproval    bool           `json:"senior_approval"`
171     ExpiresAt         string          `json:"expires_at"`
172     CreatedAt         string          `json:"created_at"`

```

```

173 UpdatedAt      string          `json:"updated_at"`
174 }
175
176 type KycProviderCheck struct {
177     CheckID      string          `json:"check_id"`
178     ProfileID    string          `json:"profile_id"`
179     Status       KycStatus      `json:"status"`
180     RiskLevel    KycRiskLevel   `json:"risk_level"`
181     Checks       map[string]string `json:"checks"`
182     DocumentHashes []string        `json:"document_hashes"`
183     ExpiresAt    string          `json:"expires_at"`
184     Timestamp    string          `json:"timestamp"`
185 }
186
187 type KycAuditEvent struct {
188     EventID      string `json:"event_id"`
189     ProfileID    string `json:"profile_id"`
190     EventType    string `json:"event_type"`
191     OldStatus    string `json:"old_status"`
192     NewStatus    string `json:"new_status"`
193     Description  string `json:"description"`
194     Timestamp    string `json:"timestamp"`
195 }
196
197 type RetailCustomer struct {
198     CustomerID    string `json:"customer_id"`
199     IdentityHash  string `json:"identity_hash"`
200     WalletAccountID string `json:"wallet_account_id"`
201     KycProfileID  string `json:"kyc_profile_id"`
202     CreatedAt     string `json:"created_at"`
203     UpdatedAt     string `json:"updated_at"`
204 }
205
206 type SystemLimit struct {
207     Scope LimitScope `json:"scope"`
208     Value int64          `json:"value"`
209 }
210
211 type AutoLimitPolicy struct {
212     ParticipantID string `json:"participant_id"`
213     AutoRedemption bool  `json:"auto_redemption"`
214     MinBalance     int64 `json:"min_balance"`
215     UpdatedAt      string `json:"updated_at"`
216 }
217
218 type TransactionRecord struct {
219     TxID          string          `json:"tx_id"`
220     TransactionType TransactionType `json:"transaction_type"`
221     SenderID      string          `json:"sender_id"`
222     ReceiverID    string          `json:"receiver_id"`
223     Amount        int64          `json:"amount"`
224     Status        TransactionStatus `json:"status"`
225     RelatedIntentID string        `json:"related_intent_id"`
226     Timestamp     string          `json:"timestamp"`
227 }
228
229 type SupervisionEvent struct {
230     EventID      string `json:"event_id"`
231     EventType    string `json:"event_type"`

```

```

232 EntityType string `json:"entity_type"`
233 EntityID   string `json:"entity_id"`
234 Data       string `json:"data"`
235 Timestamp  string `json:"timestamp"`
236 }
237
238 type ReconciliationReport struct {
239     ReportID      string `json:"report_id"`
240     TotalSupply   int64  `json:"total_supply"`
241     TotalIssued   int64  `json:"total_issued"`
242     TotalBurned   int64  `json:"total_burned"`
243     Wallets       int     `json:"wallets"`
244     Participants  int     `json:"participants"`
245     GeneratedAt   string `json:"generated_at"`
246 }
247
248 type Metrics struct {
249     TotalTransactions int64 `json:"total_transactions"`
250     SettledCount      int64 `json:"settled_count"`
251     RejectedCount     int64 `json:"rejected_count"`
252     ActiveWallets     int64 `json:"active_wallets"`
253     ActiveParticipants int64 `json:"active_participants"`
254     TotalSupply       int64 `json:"total_supply"`
255 }
256
257 // --- InitLedger -----
258
259 // txNow returns the transaction's timestamp as a UTC time. Using the proposal
260 // timestamp (identical across all endorsing peers) instead of time.Now() keeps the
261 // chaincode deterministic, so multi-org endorsement read/write sets always match.
262 func txNow(ctx contractapi.TransactionContextInterface) time.Time {
263     ts, err := ctx.GetStub().GetTxTimestamp()
264     if err != nil || ts == nil {
265         return time.Unix(0, 0).UTC()
266     }
267     return ts.AsTime().UTC()
268 }

```

L.2 Source Code Otorisasi MSP *Chaincode*

Bagian ini memuat fungsi pemeriksaan identitas organisasi untuk membatasi mutasi pada batas kepercayaan *chaincode*.

```

1
2 const (
3     BankIndonesiaMSP = "BankIndonesiaOrgMSP"
4     HimbaraBankMSP   = "HimbaraBankOrgMSP"
5     CommercialBankMSP = "CommercialBankOrgMSP"
6     OJKObserverMSP    = "OJKObserverOrgMSP"
7     PJPMSP            = "PJPOrgMSP"
8
9     schemaStateKey = "schema::digital_rupiah_v3"
10    schemaVersionV3 = "v3"
11 )
12
13 func requireMSP(ctx contractapi.TransactionContextInterface, allowed ...string) error {
14     identity := ctx.GetClientIdentity()
15     if identity == nil {

```

```

16     return fmt.Errorf("access denied: client identity is unavailable")
17 }
18 mspID, err := identity.GetMSPID()
19 if err != nil {
20     return fmt.Errorf("access denied: resolve client MSP: %w", err)
21 }
22 for _, allowedMSP := range allowed {
23     if mspID == allowedMSP {
24         return nil
25     }
26 }
27 return fmt.Errorf("access denied: MSP %s is not authorized", mspID)
28 }
29
30 func requireBankIndonesia(ctx contractapi.TransactionContextInterface) error {
31     return requireMSP(ctx, BankIndonesiaMSP)
32 }

```

L.3 Source Code Inisialisasi *Ledger*

Bagian ini memuat kode sumber fungsi `InitLedger` yang menanamkan batas tingkatan dan batas sistem (berkas `chaincode/digital_rupiah.go`).

```

1 func (s *SmartContract) InitLedger(ctx contractapi.TransactionContextInterface) error {
2     limits := []TierLimit{
3         {Tier: TierBasic, MaxBalance: 1_000_000, MinBalance: 0, DailyTxLimit: 500_000,
4          MonthlyTxLimit: 5_000_000, PerTxLimit: 250_000},
5         {Tier: TierStandard, MaxBalance: 50_000_000, MinBalance: 100_000, DailyTxLimit: 10
6          _000_000, MonthlyTxLimit: 100_000_000, PerTxLimit: 2_500_000},
7         {Tier: TierMerchant, MaxBalance: 200_000_000, MinBalance: 500_000, DailyTxLimit: 50
8          _000_000, MonthlyTxLimit: 500_000_000, PerTxLimit: 10_000_000},
9     }
10
11     for _, limit := range limits {
12         if err := s.putLimit(ctx, limit); err != nil {
13             return err
14         }
15     }
16
17     now := txNow(ctx)
18
19     biWallet := Wallet{
20         OwnerID:      "bank_indonesia",
21         ParticipantID: "bank_indonesia",
22         Tier:          TierMerchant,
23         WalletType:    WalletHot,
24         Balance:       0,
25         LastResetDay:  now.Format("2006-01-02"),
26         CreatedAt:     now.Format(time.RFC3339),
27         UpdatedAt:     now.Format(time.RFC3339),
28     }
29
30     if err := s.putWallet(ctx, "bi_treasury", biWallet); err != nil {
31         return err
32     }
33
34     sysLimits := []SystemLimit{
35         {Scope: ScopeGlobalSupply, Value: 10_000_000_000_000},
36         {Scope: ScopePerParticipantBalance, Value: 1_000_000_000_000},
37         {Scope: ScopePerTxAmount, Value: 100_000_000_000},
38     }
39
40     for _, limit := range sysLimits {
41         if err := s.putSystemLimit(ctx, limit); err != nil {
42             return err
43         }
44     }
45 }

```

```

33     }
34     for _, sl := range sysLimits {
35         if err := s.putSystemLimit(ctx, sl); err != nil {
36             return err
37         }
38     }
39
40     return nil
41 }

```

L.4 Source Code Penerbitan dan Distribusi Likuiditas

Bagian ini memuat kode sumber fungsi Mint, Burn, RequestIssuance, dan DistributeToParticipant (berkas chaincode/digital_rupiah.go).

```

1 func (s *SmartContract) Mint(ctx contractapi.TransactionContextInterface, walletID
   string, amount int64) error {
2     if amount <= 0 {
3         return fmt.Errorf("amount must be positive")
4     }
5     wallet, err := s.getWallet(ctx, walletID)
6     if err != nil {
7         return err
8     }
9     if wallet.Frozen {
10        return fmt.Errorf("wallet %s is frozen", walletID)
11    }
12    limit, err := s.getLimit(ctx, wallet.Tier)
13    if err != nil {
14        return err
15    }
16    newBalance := wallet.Balance + amount
17    if newBalance > limit.MaxBalance {
18        return fmt.Errorf("mint would exceed max balance %d for tier %s", limit.MaxBalance,
19            wallet.Tier)
20    }
21    wallet.Balance = newBalance
22    wallet.UpdatedAt = txNow(ctx).Format(time.RFC3339)
23    if err := s.putWallet(ctx, walletID, wallet); err != nil {
24        return err
25    }
26    _ = s.emitAudit(ctx, "MINT", walletID, "", amount)
27    return s.recordTransaction(ctx, TxMint, walletID, "", amount, TxSettled, "")
28 }
29 func (s *SmartContract) Burn(ctx contractapi.TransactionContextInterface, walletID
   string, amount int64) error {
30     if amount <= 0 {
31         return fmt.Errorf("amount must be positive")
32     }
33     wallet, err := s.getWallet(ctx, walletID)
34     if err != nil {
35         return err
36     }
37     if wallet.Frozen {
38         return fmt.Errorf("wallet %s is frozen", walletID)
39     }
40     if wallet.Balance < amount {

```

```

41     return fmt.Errorf("insufficient balance: have %d, need %d", wallet.Balance, amount)
42 }
43 wallet.Balance -= amount
44 wallet.UpdatedAt = txNow(ctx).Format(time.RFC3339)
45 if err := s.putWallet(ctx, walletID, wallet); err != nil {
46     return err
47 }
48 _ = s.emitAudit(ctx, "BURN", walletID, "", amount)
49 return s.recordTransaction(ctx, TxBurn, walletID, "", amount, TxSettled, "")
50 }
51
52 func (s *SmartContract) RequestIssuance(ctx contractapi.TransactionContextInterface,
    participantID string, amount int64) (interface{}, error) {
53     p, err := s.getParticipant(ctx, participantID)
54     if err != nil {
55         return nil, err
56     }
57     if p.ParticipantType == ParticipantPJP {
58         return nil, fmt.Errorf("PJP participants cannot request direct issuance; receive
    liquidity via DistributeToParticipant from a bank validator")
59     }
60     if p.Status != "active" {
61         return nil, fmt.Errorf("participant %s is not active", participantID)
62     }
63     walletID := fmt.Sprintf("wlt_%s", participantID)
64     return nil, s.Mint(ctx, walletID, amount)
65 }
66
67 func (s *SmartContract) DistributeToParticipant(ctx contractapi.
    TransactionContextInterface, senderParticipantID string, receiverParticipantID
    string, amount int64) error {
68     if amount <= 0 {
69         return fmt.Errorf("amount must be positive")
70     }
71     sender, err := s.getParticipant(ctx, senderParticipantID)
72     if err != nil {
73         return fmt.Errorf("sender participant: %w", err)
74     }
75     if sender.ParticipantType != ParticipantValidator {
76         return fmt.Errorf("sender %s must be a validator (bank); got %s",
    senderParticipantID, sender.ParticipantType)
77     }
78     if sender.Status != ParticipantActive {
79         return fmt.Errorf("sender participant %s is not active", senderParticipantID)
80     }
81     receiver, err := s.getParticipant(ctx, receiverParticipantID)
82     if err != nil {
83         return fmt.Errorf("receiver participant: %w", err)
84     }
85     if receiver.ParticipantType != ParticipantPJP {
86         return fmt.Errorf("receiver %s must be a PJP; got %s", receiverParticipantID,
    receiver.ParticipantType)
87     }
88     if receiver.Status != ParticipantActive {
89         return fmt.Errorf("receiver participant %s is not active", receiverParticipantID)
90     }
91     senderWalletID := fmt.Sprintf("wlt_%s", senderParticipantID)
92     receiverWalletID := fmt.Sprintf("wlt_%s", receiverParticipantID)
93     if err := s.Burn(ctx, senderWalletID, amount); err != nil {

```

```

94     return fmt.Errorf("burn sender: %w", err)
95 }
96 if err := s.Mint(ctx, receiverWalletID, amount); err != nil {
97     return fmt.Errorf("mint receiver: %w", err)
98 }
99 _ = s.emitSupervisionEvent(ctx, "pjp_distribution", "participant",
100     receiverParticipantID, map[string]interface{}{
101         "sender": senderParticipantID,
102         "receiver": receiverParticipantID,
103         "amount": amount,
104     })
105 return nil
106 }

```

L.5 Source Code Transfer dan Penegakan Limit

Bagian ini memuat kode sumber fungsi Transfer dengan penegakan limit per transaksi, harian, dan bulanan (berkas chaincode/digital_rupiah.go).

```

1 func (s *SmartContract) Transfer(ctx contractapi.TransactionContextInterface, senderID
2     string, receiverID string, amount int64) error {
3     if amount <= 0 {
4         return fmt.Errorf("amount must be positive")
5     }
6     if senderID == receiverID {
7         return fmt.Errorf("sender and receiver must differ")
8     }
9     sender, err := s.getWallet(ctx, senderID)
10    if err != nil {
11        return err
12    }
13    if sender.Frozen {
14        return fmt.Errorf("sender wallet %s is frozen", senderID)
15    }
16    receiver, err := s.getWallet(ctx, receiverID)
17    if err != nil {
18        return err
19    }
20    if receiver.Frozen {
21        return fmt.Errorf("receiver wallet %s is frozen", receiverID)
22    }
23    senderLimit, err := s.getLimit(ctx, sender.Tier)
24    if err != nil {
25        return err
26    }
27    if amount > senderLimit.PerTxLimit {
28        return fmt.Errorf("per-transaction limit exceeded: %d for tier %s", senderLimit.
29            PerTxLimit, sender.Tier)
30    }
31    today := txNow(ctx).Format("2006-01-02")
32    s.resetSpendingIfNewDay(&sender, today)
33    newDaily := sender.DailySpent + amount
34    if newDaily > senderLimit.DailyTxLimit {
35        return fmt.Errorf("daily transaction limit exceeded: %d for tier %s", senderLimit.
36            DailyTxLimit, sender.Tier)
37    }
38    newMonthly := sender.MonthlySpent + amount
39    if newMonthly > senderLimit.MonthlyTxLimit {

```

```

37     return fmt.Errorf("monthly transaction limit exceeded: %d for tier %s", senderLimit.
    MonthlyTxLimit, sender.Tier)
38 }
39 if sender.Balance < amount {
40     return fmt.Errorf("insufficient balance: have %d, need %d", sender.Balance, amount)
41 }
42 receiverLimit, err := s.getLimit(ctx, receiver.Tier)
43 if err != nil {
44     return err
45 }
46 if receiver.Balance+amount > receiverLimit.MaxBalance {
47     return fmt.Errorf("transfer would exceed receiver max balance %d for tier %s",
    receiverLimit.MaxBalance, receiver.Tier)
48 }
49 sender.Balance -= amount
50 sender.DailySpent = newDaily
51 sender.MonthlySpent = newMonthly
52 sender.UpdatedAt = txNow(ctx).Format(time.RFC3339)
53 receiver.Balance += amount
54 receiver.UpdatedAt = txNow(ctx).Format(time.RFC3339)
55 if err := s.putWallet(ctx, senderID, sender); err != nil {
56     return err
57 }
58 if err := s.putWallet(ctx, receiverID, receiver); err != nil {
59     return err
60 }
61 if senderLimit.MinBalance > 0 && sender.Balance < senderLimit.MinBalance {
62     policy, err := s.getAutoLimitPolicy(ctx, sender.ParticipantID)
63     if err == nil && policy.AutoRedemption && sender.Balance > 0 {
64         if arErr := s.autoRedeem(ctx, senderID, sender.Balance); arErr != nil {
65             _ = s.emitAudit(ctx, "AUTO_REDEMPTION_FAILED", senderID, "", sender.Balance)
66         }
67     }
68 }
69 _ = s.emitAudit(ctx, "TRANSFER", senderID, receiverID, amount)
70 return s.recordTransaction(ctx, TxTransfer, senderID, receiverID, amount, TxSettled, "
    ")
71 }

```

L.6 Source Code Kepatuhan KYC

Bagian ini memuat kode sumber fungsi SubmitKycProfile dan RefreshKycProfile (berkas chaincode/digital_rupiah.go).

```

1 func (s *SmartContract) SubmitKycProfile(ctx contractapi.TransactionContextInterface,
    profileID string, subjectType string, subjectID string, providerCaseID string,
    documentHashesJSON string) error {
2     exists, err := s.kycProfileExists(ctx, profileID)
3     if err != nil {
4         return err
5     }
6     if exists {
7         return fmt.Errorf("KYC profile %s already exists", profileID)
8     }
9     var hashes []string
10    if err := json.Unmarshal([]byte(documentHashesJSON), &hashes); err != nil {
11        return fmt.Errorf("invalid document_hashes: %w", err)
12    }

```

```

13 now := txNow(ctx)
14 profile := KycProfile{
15     ProfileID:      profileID,
16     SubjectType:    KycSubjectType(subjectType),
17     SubjectID:      subjectID,
18     ProviderCaseID: providerCaseID,
19     DocumentHashes: hashes,
20     Status:         KycPending,
21     RiskLevel:      RiskLow,
22     KycChecks:      map[string]string{},
23     CreatedAt:      now.Format(time.RFC3339),
24     UpdatedAt:      now.Format(time.RFC3339),
25 }
26 if err := s.putKycProfile(ctx, profile); err != nil {
27     return err
28 }
29 return s.emitKycAudit(ctx, profileID, "profile_created", "", "pending", "KYC profile
    submitted")
30 }
31
32 func (s *SmartContract) RefreshKycProfile(ctx contractapi.TransactionContextInterface,
    profileID string, status string, riskLevel string, checksJSON string,
    documentHashesJSON string, rejectionReason string, expiresAt string) error {
33 profile, err := s.getKycProfile(ctx, profileID)
34 if err != nil {
35     return err
36 }
37 oldStatus := string(profile.Status)
38 var checks map[string]string
39 if checksJSON != "" {
40     if err := json.Unmarshal([]byte(checksJSON), &checks); err != nil {
41         return fmt.Errorf("invalid checks: %w", err)
42     }
43 }
44 var hashes []string
45 if documentHashesJSON != "" {
46     if err := json.Unmarshal([]byte(documentHashesJSON), &hashes); err != nil {
47         return fmt.Errorf("invalid document_hashes: %w", err)
48     }
49 }
50 now := txNow(ctx)
51 profile.Status = KycStatus(status)
52 profile.RiskLevel = KycRiskLevel(riskLevel)
53 if checks != nil {
54     profile.KycChecks = checks
55 }
56 if len(hashes) > 0 {
57     profile.DocumentHashes = hashes
58 }
59 profile.RejectionReason = rejectionReason
60 profile.ExpiresAt = expiresAt
61 profile.UpdatedAt = now.Format(time.RFC3339)
62 if err := s.putKycProfile(ctx, profile); err != nil {
63     return err
64 }
65 checkID := fmt.Sprintf("chk_%s_%s", profileID, now.Format("20060102150405"))
66 pc := KycProviderCheck{
67     CheckID:      checkID,
68     ProfileID:    profileID,

```

```

69     Status:           KycStatus(status),
70     RiskLevel:        KycRiskLevel(riskLevel),
71     Checks:           checks,
72     DocumentHashes:   hashes,
73     RejectionReason:  rejectionReason,
74     ExpiresAt:        expiresAt,
75     Timestamp:        now.Format(time.RFC3339),
76 }
77 if err := s.putKycProviderCheck(ctx, pc); err != nil {
78     return err
79 }
80 return s.emitKycAudit(ctx, profileID, "profile_refreshed", oldStatus, status, fmt.
    Sprintf("Provider decision: %s risk=%s", status, riskLevel))
81 }

```

L.7 Source Code Pengawasan dan Rekonsiliasi

Bagian ini memuat kode sumber fungsi laporan rekonsiliasi, metrik, dan emisi *event* pengawasan (berkas chaincode/digital_rupiah.go).

```

1 func (s *SmartContract) GetReconciliationReport(ctx contractapi.
    TransactionContextInterface) (*ReconciliationReport, error) {
2     wallets, err := s.GetAllWallets(ctx)
3     if err != nil {
4         return nil, err
5     }
6     participants, err := s.getAllParticipants(ctx)
7     if err != nil {
8         participants = nil
9     }
10    var totalSupply int64
11    for _, w := range wallets {
12        totalSupply += w.Balance
13    }
14    report := ReconciliationReport{
15        ReportID:        fmt.Sprintf("rec_%s", txNow(ctx).Format("20060102150405")),
16        TotalSupply:      totalSupply,
17        TotalIssued:      0,
18        TotalBurned:      0,
19        Wallets:          len(wallets),
20        Participants:     len(participants),
21        GeneratedAt:      txNow(ctx).Format(time.RFC3339),
22    }
23    txIter, err := ctx.GetStub().GetStateByPartialCompositeKey("tx", []string{})
24    if err == nil {
25        defer txIter.Close()
26        for txIter.HasNext() {
27            kv, err := txIter.Next()
28            if err != nil {
29                continue
30            }
31            var tr TransactionRecord
32            if err := json.Unmarshal(kv.Value, &tr); err != nil {
33                continue
34            }
35            switch tr.TransactionType {
36            case TxMint:
37                report.TotalIssued += tr.Amount

```

```

38         case TxBurn:
39             report.TotalBurned += tr.Amount
40         }
41     }
42 }
43 return &report, nil
44 }
45
46 func (s *SmartContract) GetMetrics(ctx contractapi.TransactionContextInterface) (*
    Metrics, error) {
47     wallets, err := s.GetAllWallets(ctx)
48     if err != nil {
49         return nil, err
50     }
51     participants, err := s.getAllParticipants(ctx)
52     if err != nil {
53         participants = nil
54     }
55     var totalSupply int64
56     activeWallets := int64(0)
57     for _, w := range wallets {
58         totalSupply += w.Balance
59         if !w.Frozen {
60             activeWallets++
61         }
62     }
63     activeParticipants := int64(0)
64     for _, p := range participants {
65         if p.Status == ParticipantActive {
66             activeParticipants++
67         }
68     }
69     m := Metrics{
70         ActiveWallets:    activeWallets,
71         ActiveParticipants: activeParticipants,
72         TotalSupply:      totalSupply,
73     }
74     txIter, err := ctx.GetStub().GetStateByPartialCompositeKey("tx", []string{})
75     if err == nil {
76         defer txIter.Close()
77         for txIter.HasNext() {
78             kv, err := txIter.Next()
79             if err != nil {
80                 continue
81             }
82             var tr TransactionRecord
83             if err := json.Unmarshal(kv.Value, &tr); err != nil {
84                 continue
85             }
86             m.TotalTransactions++
87             switch tr.Status {
88             case TxSettled:
89                 m.SettledCount++
90             case TxRejected:
91                 m.RejectedCount++
92             }
93         }
94     }
95     return &m, nil

```

```

96 }
97
98 func (s *SmartContract) emitSupervisionEvent(ctx contractapi.TransactionContextInterface
    , eventType string, entityType string, entityID string, data interface{}) error {
99     now := txNow(ctx)
100     eventID := fmt.Sprintf("sup_%s_%s", entityID, now.Format("20060102150405"))
101     var dataStr string
102     if data != nil {
103         b, err := json.Marshal(data)
104         if err != nil {
105             return err
106         }
107         dataStr = string(b)
108     }
109     ev := SupervisionEvent{
110         EventID:     eventID,
111         EventType:    eventType,
112         EntityType:   entityType,
113         EntityID:     entityID,
114         Data:         dataStr,
115         Timestamp:    now.Format(time.RFC3339),
116     }
117     key, err := ctx.GetStub().CreateCompositeKey("supervision", []string{entityID, eventID})
118     if err != nil {
119         return err
120     }
121     evData, err := json.Marshal(ev)
122     if err != nil {
123         return err
124     }
125     return ctx.GetStub().PutState(key, evData)
126 }

```

L.8 Source Code Konfigurasi Jaringan Caliper

Bagian ini memuat konfigurasi koneksi Caliper ke jaringan Fabric. Nilai *top-level version* menyatakan versi skema konfigurasi Caliper, sedangkan versi kontrak dicatat terpisah pada entri kanal. Berkas dimuat langsung dari sumber otoritatif agar lampiran tetap sesuai dengan konfigurasi eksekusi.

```

1 ---
2 name: PayNet Garuda Network
3 version: "2.0.0"
4 caliper:
5     blockchain: fabric
6
7 channels:
8     - channelName: mychannel
9       contracts:
10         - id: digital-rupiah
11           version: "3.0"
12
13 organizations:
14     - mspid: BankIndonesiaOrgMSP
15       identities:
16         certificates:

```

```

17     - name: User1
18       clientSignedCert:
19         path: network/organizations/peerOrganizations/bi.paynet/users/User1@bi.
paynet/msp/signcerts/cert.pem
20       clientPrivateKey:
21         path: network/organizations/peerOrganizations/bi.paynet/users/User1@bi.
paynet/msp/keystore/key.pem
22     connectionProfile:
23       path: network/organizations/peerOrganizations/bi.paynet/connection-bi.json
24       discover: true
25
26 - mspid: HimbaraBankOrgMSP
27   identities:
28     certificates:
29       - name: User1
30         clientSignedCert:
31           path: network/organizations/peerOrganizations/himbara.paynet/users/
User1@himbara.paynet/msp/signcerts/cert.pem
32         clientPrivateKey:
33           path: network/organizations/peerOrganizations/himbara.paynet/users/
User1@himbara.paynet/msp/keystore/key.pem
34       connectionProfile:
35         path: network/organizations/peerOrganizations/himbara.paynet/connection-himbara.
json
36       discover: true
37
38 - mspid: CommercialBankOrgMSP
39   identities:
40     certificates:
41       - name: User1
42         clientSignedCert:
43           path: network/organizations/peerOrganizations/commercial.paynet/users/
User1@commercial.paynet/msp/signcerts/cert.pem
44         clientPrivateKey:
45           path: network/organizations/peerOrganizations/commercial.paynet/users/
User1@commercial.paynet/msp/keystore/key.pem
46       connectionProfile:
47         path: network/organizations/peerOrganizations/commercial.paynet/connection-
commercial.json
48       discover: true

```

L.9 Source Code Suite Konfigurasi *Benchmark* Caliper

Bagian ini memuat konfigurasi benchmark yang digunakan pada versi tesis saat ini. Berkas dimuat langsung dari sumber otoritatif agar lampiran tidak menyimpang dari konfigurasi eksekusi.

```

1 test:
2   name: Digital Rupiah Retail Transfer Benchmark - 1 Worker
3   description: Seeded low-contention retail transfer baseline with one Caliper worker
4   workers:
5     type: local
6     number: 1
7   rounds:
8     - label: retail-warmup-w1
9       txDuration: 30
10      rateControl:

```

```

11         type: fixed-rate
12         opts:
13             tps: 10
14     workload:
15         module: benchmark/workloads/retail-transfer.js
16         arguments:
17             scenario: transfer-w1
18             customers: 116
19             merchants: 16
20             transactionSlots: 52
21             fundedRatio: 1
22             maxTransferAmount: 50000
23             seed: 20260725
24
25     - label: retail-sustained-w1
26       txDuration: 120
27       rateControl:
28         type: fixed-rate
29         opts:
30             tps: 30
31       workload:
32         module: benchmark/workloads/retail-transfer.js
33         arguments:
34             scenario: transfer-w1
35             customers: 116
36             merchants: 16
37             transactionSlots: 52
38             fundedRatio: 1
39             maxTransferAmount: 50000
40             seed: 20260725
41
42     - label: retail-peak-w1
43       txDuration: 120
44       rateControl:
45         type: fixed-rate
46         opts:
47             tps: 60
48       workload:
49         module: benchmark/workloads/retail-transfer.js
50         arguments:
51             scenario: transfer-w1
52             customers: 116
53             merchants: 16
54             transactionSlots: 52
55             fundedRatio: 1
56             maxTransferAmount: 50000
57             seed: 20260725
58
59 observer:
60     type: local
61     interval: 1
62
63 monitors:
64     resource:
65         - module: docker
66         options:
67             interval: 1
68             cpuUsageNormalization: true
69         containers:

```

```

70         - /peer0.bi.paynet
71         - /peer0.himbara.paynet
72         - /peer0.commercial.paynet
73         - /peer0.pjp.paynet
74         - /peer0.ojk.paynet
75         - /couchdb0.bi.paynet
76         - /couchdb0.himbara.paynet
77         - /couchdb0.commercial.paynet
78         - /couchdb0.pjp.paynet
79         - /couchdb0.ojk.paynet
80         - /orderer.paynet

```

```

1 test:
2   name: Digital Rupiah Retail Transfer Benchmark - 2 Workers
3   description: Seeded low-contention retail transfer baseline with two Caliper workers
4   workers:
5     type: local
6     number: 2
7   rounds:
8     - label: retail-warmup-w2
9       txDuration: 30
10      rateControl:
11        type: fixed-rate
12        opts:
13          tps: 10
14      workload:
15        module: benchmark/workloads/retail-transfer.js
16        arguments:
17          scenario: transfer-w2
18          customers: 58
19          merchants: 8
20          transactionSlots: 26
21          fundedRatio: 1
22          maxTransferAmount: 50000
23          seed: 20260725
24
25     - label: retail-sustained-w2
26       txDuration: 120
27       rateControl:
28         type: fixed-rate
29         opts:
30           tps: 30
31       workload:
32         module: benchmark/workloads/retail-transfer.js
33         arguments:
34           scenario: transfer-w2
35           customers: 58
36           merchants: 8
37           transactionSlots: 26
38           fundedRatio: 1
39           maxTransferAmount: 50000
40           seed: 20260725
41
42     - label: retail-peak-w2
43       txDuration: 120
44       rateControl:
45         type: fixed-rate
46         opts:
47           tps: 60

```

```

48     workload:
49       module: benchmark/workloads/retail-transfer.js
50       arguments:
51         scenario: transfer-w2
52         customers: 58
53         merchants: 8
54         transactionSlots: 26
55         fundedRatio: 1
56         maxTransferAmount: 50000
57         seed: 20260725
58
59 observer:
60   type: local
61   interval: 1
62
63 monitors:
64   resource:
65     - module: docker
66     options:
67       interval: 1
68       cpuUsageNormalization: true
69     containers:
70       - /peer0.bi.paynet
71       - /peer0.himbara.paynet
72       - /peer0.commercial.paynet
73       - /peer0.pjp.paynet
74       - /peer0.ojk.paynet
75       - /couchdb0.bi.paynet
76       - /couchdb0.himbara.paynet
77       - /couchdb0.commercial.paynet
78       - /couchdb0.pjp.paynet
79       - /couchdb0.ojk.paynet
80       - /orderer.paynet

```

```

1 test:
2   name: Digital Rupiah Retail Transfer Benchmark - 4 Workers
3   description: Seeded low-contention retail transfer baseline with four Caliper workers
4   workers:
5     type: local
6     number: 4
7   rounds:
8     - label: retail-warmup-w4
9       txDuration: 30
10      rateControl:
11        type: fixed-rate
12        opts:
13          tps: 10
14      workload:
15        module: benchmark/workloads/retail-transfer.js
16        arguments:
17          scenario: transfer-w4
18          customers: 29
19          merchants: 4
20          transactionSlots: 13
21          fundedRatio: 1
22          maxTransferAmount: 50000
23          seed: 20260725
24
25     - label: retail-sustained-w4

```

```

26     txDuration: 120
27     rateControl:
28       type: fixed-rate
29       opts:
30         tps: 30
31     workload:
32       module: benchmark/workloads/retail-transfer.js
33       arguments:
34         scenario: transfer-w4
35         customers: 29
36         merchants: 4
37         transactionSlots: 13
38         fundedRatio: 1
39         maxTransferAmount: 50000
40         seed: 20260725
41
42   - label: retail-peak-w4
43     txDuration: 120
44     rateControl:
45       type: fixed-rate
46       opts:
47         tps: 60
48     workload:
49       module: benchmark/workloads/retail-transfer.js
50       arguments:
51         scenario: transfer-w4
52         customers: 29
53         merchants: 4
54         transactionSlots: 13
55         fundedRatio: 1
56         maxTransferAmount: 50000
57         seed: 20260725
58
59 observer:
60   type: local
61   interval: 1
62
63 monitors:
64   resource:
65     - module: docker
66     options:
67       interval: 1
68       cpuUsageNormalization: true
69       containers:
70         - /peer0.bi.paynet
71         - /peer0.himbara.paynet
72         - /peer0.commercial.paynet
73         - /peer0.pjp.paynet
74         - /peer0.ojk.paynet
75         - /couchdb0.bi.paynet
76         - /couchdb0.himbara.paynet
77         - /couchdb0.commercial.paynet
78         - /couchdb0.pjp.paynet
79         - /couchdb0.ojk.paynet
80         - /orderer.paynet

```

```

1 test:
2   name: Digital Rupiah Retail Transfer - Repetition (mean +/- CI)
3   description: Five seeded 60-second peak rounds of the low-contention transfer workload

```

```

    for mean and confidence-interval estimation
4  workers:
5    type: local
6    number: 2
7  rounds:
8    - label: retail-peak-rep-1
9      txDuration: 60
10     rateControl:
11       type: fixed-rate
12       opts:
13         tps: 60
14     workload: &repeat_workload
15     module: benchmark/workloads/retail-transfer.js
16     arguments:
17       scenario: transfer-repeat
18       customers: 58
19       merchants: 8
20       transactionSlots: 26
21       fundedRatio: 1
22       maxTransferAmount: 50000
23       seed: 20260725
24     - label: retail-peak-rep-2
25       txDuration: 60
26       rateControl:
27         type: fixed-rate
28         opts:
29           tps: 60
30       workload: *repeat_workload
31     - label: retail-peak-rep-3
32       txDuration: 60
33       rateControl:
34         type: fixed-rate
35         opts:
36           tps: 60
37       workload: *repeat_workload
38     - label: retail-peak-rep-4
39       txDuration: 60
40       rateControl:
41         type: fixed-rate
42         opts:
43           tps: 60
44       workload: *repeat_workload
45     - label: retail-peak-rep-5
46       txDuration: 60
47       rateControl:
48         type: fixed-rate
49         opts:
50           tps: 60
51       workload: *repeat_workload
52
53 observer:
54   type: local
55   interval: 1
56
57 monitors:
58   resource:
59     - module: docker
60     options:
61       interval: 1

```

```

62     cpuUsageNormalization: true
63     containers:
64       - /peer0.bi.paynet
65       - /peer0.himbara.paynet
66       - /peer0.commercial.paynet
67       - /peer0.pjp.paynet
68       - /peer0.ojk.paynet
69       - /couchdb0.bi.paynet
70       - /couchdb0.himbara.paynet
71       - /couchdb0.commercial.paynet
72       - /couchdb0.pjp.paynet
73       - /couchdb0.ojk.paynet
74       - /orderer.paynet

```

```

1 test:
2   name: Digital Rupiah Functional Benchmark Profiles
3   description: Separate benchmark profiles for onboarding, monetary operations,
4     supervision reads, and administrative policy writes
5   workers:
6     type: local
7     number: 2
8   rounds:
9     - label: kyc-onboarding
10      txNumber: 40
11      rateControl:
12        type: fixed-rate
13        opts:
14          tps: 5
15      workload:
16        module: benchmark/workloads/retail-onboarding.js
17        arguments:
18          scenario: kyc-onboarding
19          tier: STANDARD
20    - label: monetary-ops
21      txNumber: 80
22      rateControl:
23        type: fixed-rate
24        opts:
25          tps: 20
26      workload:
27        module: benchmark/workloads/monetary-ops.js
28        arguments:
29          scenario: monetary-ops
30          wallets: 50
31          transactionSlots: 40
32          initialStandardBalance: 1000000
33          initialBasicBalance: 100000
34
35    - label: supervision-reads
36      txNumber: 80
37      rateControl:
38        type: fixed-rate
39        opts:
40          tps: 30
41      workload:
42        module: benchmark/workloads/supervision-reads.js
43        arguments:
44          scenario: supervision-reads

```

```

45     customers: 50
46     merchants: 10
47     transactionSlots: 40
48     seedTransfers: 5
49
50   - label: admin-policy
51     txNumber: 40
52     rateControl:
53       type: fixed-rate
54       opts:
55         tps: 5
56     workload:
57       module: benchmark/workloads/admin-policy.js
58       arguments:
59         scenario: admin-policy
60
61 observer:
62   type: local
63   interval: 1
64
65 monitors:
66   resource:
67     - module: docker
68     options:
69       interval: 1
70       cpuUsageNormalization: true
71       containers:
72         - /peer0.bi.paynet
73         - /peer0.himbara.paynet
74         - /peer0.commercial.paynet
75         - /peer0.pjp.paynet
76         - /peer0.ojk.paynet
77         - /couchdb0.bi.paynet
78         - /couchdb0.himbara.paynet
79         - /couchdb0.commercial.paynet
80         - /couchdb0.pjp.paynet
81         - /couchdb0.ojk.paynet
82         - /orderer.paynet

```

```

1 test:
2   name: Digital Rupiah Negative-Path Conformance
3   description: Every submission violates one policy rule and must be rejected; a zero-
4     gap run proves enforcement
5   workers:
6     type: local
7     number: 1
8   rounds:
9     - label: negative-path
10       txNumber: 7
11       rateControl:
12         type: fixed-rate
13         opts:
14           tps: 5
15       workload:
16         module: benchmark/workloads/negative-path.js
17         arguments:
18           scenario: negative-path
19           seed: 20260725

```

```

20 observer:
21   type: local
22   interval: 1
23
24 monitors:
25   resource:
26     - module: docker
27     options:
28       interval: 1
29       cpuUsageNormalization: true
30       containers:
31         - /peer0.bi.paynet
32         - /peer0.himbara.paynet
33         - /peer0.commercial.paynet
34         - /peer0.pjp.paynet
35         - /peer0.ojk.paynet
36         - /couchdb0.bi.paynet
37         - /couchdb0.himbara.paynet
38         - /couchdb0.commercial.paynet
39         - /couchdb0.pjp.paynet
40         - /couchdb0.ojk.paynet
41         - /orderer.paynet

```

```

1 test:
2   name: Digital Rupiah Aggregate Overspend Contention
3   description: Concurrent transfers request more value than a shared sender owns; the
4     final non-negative balance and classified rejections test the conservation invariant
5   workers:
6     type: local
7     number: 4
8   rounds:
9     - label: aggregate-overspend-contention
10     txNumber: 200
11     rateControl:
12       type: fixed-rate
13       opts:
14         tps: 80
15     workload:
16       module: benchmark/workloads/adversarial-double-spend.js
17       arguments:
18         scenario: overspend-contention
19         receivers: 20
20         transferAmount: 1000
21         senderFunding: 50000
22         seed: 20260725
23 observer:
24   type: local
25   interval: 1
26
27 monitors:
28   resource:
29     - module: docker
30     options:
31       interval: 1
32       cpuUsageNormalization: true
33       containers:
34         - /peer0.bi.paynet
35         - /peer0.himbara.paynet

```

```

36         - /peer0.commercial.paynet
37         - /peer0.pjp.paynet
38         - /peer0.ojk.paynet
39         - /couchdb0.bi.paynet
40         - /couchdb0.himbara.paynet
41         - /couchdb0.commercial.paynet
42         - /couchdb0.pjp.paynet
43         - /couchdb0.ojk.paynet
44         - /orderer.paynet

```

L.10 Source Code Skrip Pembangkit Aset Jaringan Caliper

Bagian ini memuat kode sumber skrip penyiapan aset jaringan untuk Caliper (berkas benchmark/gen-network-assets.sh).

```

1  #!/bin/bash
2  # Generates Caliper connection profiles + stable identity files for the garuda network.
3  # Idempotent; safe to re-run after the network is up. Run from repo root.
4  set -euo pipefail
5
6  SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
7  PROJECT_DIR="$(dirname "$SCRIPT_DIR")"
8  ORG_ROOT="$PROJECT_DIR/network/organizations/peerOrganizations"
9
10 # org | OrgName | MSPID | peerPort
11 ORGS=(
12     "bi|BankIndonesiaOrg|BankIndonesiaOrgMSP|7051"
13     "himbara|HimbaraBankOrg|HimbaraBankOrgMSP|9051"
14     "commercial|CommercialBankOrg|CommercialBankOrgMSP|11051"
15 )
16
17 for entry in "${ORGS[@]}; do
18     IFS='|' read -r ORG ORGNAME MSPID PORT <<< "$entry"
19     DOMAIN="$ORG.paynet"
20     ORG_DIR="$ORG_ROOT/$DOMAIN"
21     PEER="peer0.$DOMAIN"
22     TLS_CA="$ORG_DIR/peers/$PEER/tls/ca.crt"
23     USER_MSP="$ORG_DIR/users/User1@$DOMAIN/msp"
24
25     # Stable identity filenames (cryptogen emits hashed keystore names).
26     cp "$USER_MSP/signcerts/User1@$DOMAIN-cert.pem" "$USER_MSP/signcerts/cert.pem"
27     cp "$USER_MSP/keystore/*_sk" "$USER_MSP/keystore/key.pem"
28
29     cat > "$ORG_DIR/connection-$ORG.json" <<EOF
30 {
31     "name": "$ORG-paynet",
32     "version": "1.0.0",
33     "client": { "organization": "$ORGNAME" },
34     "channels": {
35         "mychannel": { "peers": { "$PEER": {} } }
36     },
37     "organizations": {
38         "$ORGNAME": { "mspid": "$MSPID", "peers": ["$PEER"] }
39     },
40     "peers": {
41         "$PEER": {
42             "url": "grpc://localhost:$PORT",
43             "tlsCACerts": { "path": "$TLS_CA" },

```

```

44     "grpcOptions": {
45       "ssl-target-name-override": "$PEER",
46       "hostnameOverride": "$PEER"
47     }
48   }
49 }
50 }
51 EOF
52 echo "Generated connection-$ORG.json + stable identity for $MSPID"
53 done

```

L.11 Source Code *Workload* Transfer Ritel

Bagian ini memuat kode sumber modul beban transfer antarpelanggan, transfer pelanggan-ke-pedagang, dan pembacaan dompet (berkas `benchmark/workloads/retail-transfer.js`).

```

1  'use strict';
2
3  const { RetailWorkloadBase, CONTRACT } = require('./retail-base');
4
5  /**
6   * Transfer-only retail CBDC workload:
7   *   70% customer-to-customer transfer
8   *   25% customer-to-merchant transfer
9   *   5% wallet read
10  *
11  * Customer population is 20% BASIC and 80% STANDARD, while the measured
12  * senders are STANDARD wallets so duration-based rounds do not fail because a
13  * BASIC wallet reaches its deliberately small balance or transaction limit.
14  * Merchant wallets are policy-derived MERCHANT wallets. All subjects receive
15  * approved KYC anchors during deterministic seeding before measured traffic.
16  */
17  class RetailTransferWorkload extends RetailWorkloadBase {
18    async initializeWorkloadModule(workerIndex, totalWorkers, roundIndex, roundArguments
19    , sutAdapter, sutContext) {
20      await super.initializeWorkloadModule(workerIndex, totalWorkers, roundIndex,
21      roundArguments, sutAdapter, sutContext);
22      this.workerIndex = workerIndex;
23      await this.seedPopulation({
24        numCustomers: this.arg('customers', 200),
25        numMerchants: this.arg('merchants', 20),
26        fundedRatio: this.arg('fundedRatio', 1),
27        fundStandard: this.arg('fundStandard', 5000000),
28        fundBasic: this.arg('fundBasic', 500000),
29      });
30      this.configureMeasuredTraffic(this.arg('transactionSlots', 0), totalWorkers);
31    }
32
33    operationKind(slot) {
34      const totalSlots = this.transactionSlots * this.totalWorkers;
35      const globalSlot = (slot * this.totalWorkers) + this.workerIndex;
36      const phase = Math.floor((globalSlot * 100) / totalSlots);
37      if (phase < 70) return 'customer';
38      if (phase < 95) return 'merchant';
39      return 'read';
40    }
41  }

```

```

39
40 configureMeasuredTraffic(transactionSlots, totalWorkers) {
41     this.transactionSlots = transactionSlots;
42     this.totalWorkers = totalWorkers;
43
44     const senders = this.standardCustomers.slice(0, transactionSlots);
45     const receivers = this.standardCustomers.slice(transactionSlots);
46     const required = Array.from({ length: transactionSlots }, (_, slot) => this.
operationKind(slot));
47     const requiredReceivers = required.filter(kind => kind === 'customer').length;
48     const requiredMerchants = required.filter(kind => kind === 'merchant').length;
49
50     if (senders.length < transactionSlots || receivers.length < requiredReceivers ||
this.merchants.length < requiredMerchants) {
51         throw new Error(
52             `insufficient contention-free population: need ${transactionSlots}
senders, ` +
53             `${requiredReceivers} STANDARD receivers, and ${requiredMerchants}
merchants`,
54             );
55     }
56
57     let receiverIndex = 0;
58     let merchantIndex = 0;
59     this.transactionPlans = required.map((kind, slot) => {
60         if (kind === 'customer') {
61             return { kind, sender: senders[slot], receiver: receivers[receiverIndex
62 ++] };
63         }
64         if (kind === 'merchant') {
65             return { kind, sender: senders[slot], receiver: this.merchants[
66 merchantIndex++] };
67         }
68         return { kind, customer: senders[slot] };
69     });
70
71     planTransaction(slot) {
72         const plan = this.transactionPlans && this.transactionPlans[slot % this.
transactionPlans.length];
73         if (!plan) throw new Error(`transaction slot ${slot} exceeds configured rotating
population`);
74         return plan;
75     }
76
77     async submitTransaction() {
78         const plan = this.planTransaction(this.txIndex++);
79         const readOnly = plan.kind === 'read';
80         const contractFunction = readOnly ? 'GetWallet' : 'Transfer';
81         const contractArguments = readOnly
82             ? [plan.customer.walletId]
83             : [plan.sender.walletId, plan.receiver.walletId, this.retailAmount(plan.
sender.perTxCap), `bench_${this.ns()}_${this.txIndex}`];
84
85         // Measured failures belong in Caliper's metrics; only setup uses fail-closed
submit().
86         return this.sutAdapter.sendRequests({
87             ...CONTRACT,
88             contractFunction,

```

```

88         contractArguments: contractArguments.map(String),
89         readOnly,
90     });
91 }
92 }
93
94 function createWorkloadModule() {
95     return new RetailTransferWorkload();
96 }
97
98 module.exports.createWorkloadModule = createWorkloadModule;
99 module.exports.RetailTransferWorkload = RetailTransferWorkload;

```

L.12 Source Code *Workload* Beban Dasar Ritel

Bagian ini memuat kode sumber modul inisialisasi populasi dan beban dasar (berkas `benchmark/workloads/retail-base.js`).

```

1  'use strict';
2
3  const { WorkloadModuleBase } = require('@hyperledger/caliper-core');
4
5  const CONTRACT = { contractId: 'digital-rupiah', contractVersion: '3.0' };
6
7  const ACTORS = Object.freeze({
8      bi: { invokerMspId: 'BankIndonesiaOrgMSP', invokerIdentity: 'User1' },
9      himbara: { invokerMspId: 'HimbaraBankOrgMSP', invokerIdentity: 'User1' },
10     commercial: { invokerMspId: 'CommercialBankOrgMSP', invokerIdentity: 'User1' },
11 });
12
13 function actorRequest(request, actor = 'bi') {
14     const identity = ACTORS[actor];
15     if (!identity) throw new Error('unknown Caliper actor: ${actor}');
16     return { ...request, ...identity };
17 }
18
19 // Tier per-transaction caps (must match chaincode InitLedger).
20 const PER_TX_BASIC = 250000;
21 const PER_TX_STANDARD = 2500000;
22
23 function transactionStatus(tx) {
24     if (!tx) return '';
25     const value = typeof tx.GetStatus === 'function' ? tx.GetStatus() : tx.status;
26     return String(value || '').toLowerCase();
27 }
28
29 function transactionDiagnostic(tx) {
30     if (!tx) return 'missing transaction result';
31     const parts = [];
32     for (const key of ['error', 'message', 'status']) {
33         if (tx[key]) parts.push(String(tx[key]));
34     }
35     if (typeof tx.GetResult === 'function') {
36         try {
37             const result = tx.GetResult();
38             if (result) parts.push(Buffer.isBuffer(result) ? result.toString('utf8') :
String(result));
39         } catch (error) {

```

```

40         parts.push(String(error));
41     }
42 }
43 return parts.join(' | ') || 'transaction failed without diagnostic';
44 }
45
46 function transactionPayload(tx) {
47     if (!tx) return '';
48     let value = tx.result;
49     if (typeof tx.GetResult === 'function') {
50         value = tx.GetResult();
51     }
52     if (Buffer.isBuffer(value)) return value.toString('utf8');
53     if (ArrayBuffer.isView(value)) {
54         return Buffer.from(value.buffer, value.byteOffset, value.byteLength).toString('utf8');
55     }
56     return typeof value === 'string' ? value : JSON.stringify(value || '');
57 }
58
59 function transactionResults(response) {
60     return Array.isArray(response) ? response : [response];
61 }
62
63 function assertSuccessfulResponse(fn, response) {
64     const results = transactionResults(response);
65     if (results.length === 0) {
66         throw new Error(`setup ${fn} returned no transaction result`);
67     }
68     for (const tx of results) {
69         const status = transactionStatus(tx);
70         if (status !== 'success') {
71             throw new Error(`setup ${fn} failed: ${transactionDiagnostic(tx)}`);
72         }
73     }
74 }
75
76 /**
77  * Shared base for the retail CBDC workloads.
78  *
79  * Seeds a worker- and round-partitioned population (customers and merchants) so
80  * that load spreads across many distinct keys instead of a single hot key-pair.
81  * Provides realistic Indonesian-retail amount sampling and selection helpers.
82  *
83  * Population IDs are namespaced per run and worker; transaction references remain
84  * round-specific so setup is reusable across Caliper rounds.
85  */
86 class RetailWorkloadBase extends WorkloadModuleBase {
87     constructor() {
88         super();
89         this.customers = []; // { id, walletId, tier, perTxCap, funded }
90         this.standardCustomers = []; // subset of customers on the STANDARD tier
91         this.merchants = []; // { id, walletId }
92         this.txIndex = 0;
93         this.randomState = 1;
94     }
95
96     async initializeWorkloadModule(workerIndex, totalWorkers, roundIndex, roundArguments
, sutAdapter, sutContext) {

```

```

97     await super.initializeWorkloadModule(workerIndex, totalWorkers, roundIndex,
roundArguments, sutAdapter, sutContext);
98     this.workerIndex = workerIndex;
99     this.roundIndex = roundIndex;
100    this.totalWorkers = totalWorkers;
101    const configuredSeed = Number(this.arg('seed', 20260725));
102    this.randomState = (configuredSeed + (workerIndex + 1) * 1009 + (roundIndex + 1)
* 9176) >>> 0;
103  }
104
105  arg(name, def) {
106    const v = this.roundArguments ? this.roundArguments[name] : undefined;
107    return v === undefined ? def : v;
108  }
109
110  // Namespace: per-worker+round by default. A scenario prefix lets multiple
111  // benchmark profiles run on the same clean ledger without cross-profile key
112  // collisions while still keeping each round isolated.
113  ns() {
114    const scenario = this.arg('scenario', null);
115    const run = process.env.BENCHMARK_STAMP || '';
116    const prefix = scenario ? `${scenario}_` : '';
117    return `${run ? `${run}_` : ''}${prefix}w${this.workerIndex}_r${this.roundIndex
}`;
118  }
119
120  populationNs() {
121    const scenario = this.arg('scenario', null);
122    const run = process.env.BENCHMARK_STAMP || '';
123    const prefix = scenario ? `${scenario}_` : '';
124    return `${run ? `${run}_` : ''}${prefix}w${this.workerIndex}`;
125  }
126
127  customerId(i) { return `c_${this.populationNs()}_${i}`; }
128  merchantId(i) { return `m_${this.populationNs()}_${i}`; }
129  walletId(ownerId) { return `wlt_${ownerId}`; }
130  isBasic(i) { return i % 5 === 0; } // ~20% BASIC, rest STANDARD
131
132  random() {
133    this.randomState = (1664525 * this.randomState + 1013904223) >>> 0;
134    return this.randomState / 0x100000000;
135  }
136
137  randInt(min, max) { return Math.floor(this.random() * (max - min + 1)) + min; }
138  pick(arr) { return arr[Math.floor(this.random() * arr.length)]; }
139
140  // Run async tasks with bounded concurrency. Seeding is otherwise sequential and,
141  // with
142  // a 2s orderer BatchTimeout, far too slow; concurrency lets blocks cut on
143  // MaxMessageCount.
144  async runPool(tasks, concurrency = 8) {
145    let next = 0;
146    const workers = Array.from({ length: Math.min(concurrency, tasks.length) },
147    async () => {
148      while (next < tasks.length) {
149        const i = next++;
150        await tasks[i]();
151      }
152    });

```

```

150     await Promise.all(workers);
151 }
152
153 async submit(fn, args, readOnly = false, actor = 'bi', retrySetupConflicts = false)
154 {
155     if (fn === 'Transfer' && args.length === 3) args = [...args, `bench_${this.ns()}
156     _${this.txIndex++}`];
157     const attempts = retrySetupConflicts ? 5 : 1;
158     for (let attempt = 0; attempt < attempts; attempt++) {
159         const response = await this.sutAdapter.sendRequests(actorRequest({
160             ...CONTRACT,
161             contractFunction: fn,
162             contractArguments: args.map(String),
163             readOnly,
164             }, actor));
165         try {
166             assertSuccessfulResponse(fn, response);
167             return response;
168         } catch (error) {
169             if (attempt + 1 === attempts) throw error;
170             await new Promise(resolve => setTimeout(resolve, 250 * (attempt + 1) +
171             this.workerIndex * 113));
172         }
173     }
174 }
175
176 // Small-value-heavy distribution typical of retail payments, clamped to cap.
177 retailAmount(cap) {
178     const r = this.random();
179     let a;
180     if (r < 0.70) a = this.randInt(5000, 100000);
181     else if (r < 0.92) a = this.randInt(100000, 500000);
182     else if (r < 0.99) a = this.randInt(500000, 2000000);
183     else a = this.randInt(2000000, 2500000);
184     return Math.min(a, cap, this.arg('maxTransferAmount', cap));
185 }
186
187 pickTwoDistinct() {
188     if (this.customers.length < 2) return null;
189     const a = this.randInt(0, this.customers.length - 1);
190     let b = this.randInt(0, this.customers.length - 1);
191     if (b === a) b = (b + 1) % this.customers.length;
192     return [this.customers[a], this.customers[b]];
193 }
194
195 // Pick a STANDARD-tier receiver distinct from senderId. Routing most P2P receipts
196 to
197 // STANDARD wallets avoids expected BASIC balance-cap rejections dominating results.
198 pickStandardReceiver(senderId) {
199     if (this.standardCustomers.length === 0) return null;
200     for (let attempt = 0; attempt < 5; attempt++) {
201         const c = this.pick(this.standardCustomers);
202         if (c.id !== senderId) return c;
203     }
204     return null;
205 }
206
207 /**
208  * Seed merchants and customers (tier mix + funding).

```

```

205     * 'fundedRatio' of customers are funded normally; the remainder are underfunded.
206     */
207     async seedPopulation({ numCustomers, numMerchants, fundedRatio, fundStandard,
208       fundBasic, seed = this.roundIndex === 0 }) {
209       this.customers = [];
210       this.standardCustomers = [];
211       this.merchants = [];
212       const seedConcurrency = this.arg('seedConcurrency', Math.max(1, Math.ceil(10 /
213         this.totalWorkers)));
214       const merchantTasks = [];
215       for (let i = 0; i < numMerchants; i++) {
216         const id = this.merchantId(i);
217         const walletId = this.walletId(id);
218         this.merchants.push({ id, walletId });
219         if (seed) merchantTasks.push(async () => {
220           const profileId = `kyc_${id}`;
221           const hashes = JSON.stringify(['sha256:${id}']);
222           await this.submit('SubmitKycProfile', [profileId, 'merchant', id, hashes
223             ], false, 'bi', true);
224           await this.submit('RefreshKycProfile', [profileId, 'approved', 'low', '
225             standard', false, hashes, '2099-12-31T23:59:59Z'], false, 'bi', true);
226           await this.submit('CreateWallet', [walletId, id, 'MERCHANT'], false, 'bi
227             ', true);
228         });
229       }
230       await this.runPool(merchantTasks, seedConcurrency);
231
232       const customerTasks = [];
233       const mintTasks = [];
234       for (let i = 0; i < numCustomers; i++) {
235         const id = this.customerId(i);
236         const walletId = this.walletId(id);
237         const basic = this.isBasic(i);
238         const tier = basic ? 'BASIC' : 'STANDARD';
239         const perTxCap = basic ? PER_TX_BASIC : PER_TX_STANDARD;
240         const funded = (i / numCustomers) < fundedRatio;
241         const fund = funded ? (basic ? fundBasic : fundStandard) : this.randInt
242           (1000, 20000);
243         const cust = { id, walletId, tier, perTxCap, funded };
244         this.customers.push(cust);
245         if (!basic) this.standardCustomers.push(cust);
246         // Per-customer steps are ordered; different customers run concurrently.
247         if (seed) customerTasks.push(async () => {
248           const profileId = `kyc_${id}`;
249           const hashes = JSON.stringify(['sha256:${id}']);
250           const diligence = basic ? 'simplified' : 'standard';
251           await this.submit('CreateRetailCustomer', [id, `sha256:identity:${id}`,
252             walletId, profileId], false, 'bi', true);
253           await this.submit('SubmitKycProfile', [profileId, 'retail_customer', id,
254             hashes], false, 'bi', true);
255           await this.submit('RefreshKycProfile', [profileId, 'approved', 'low',
256             diligence, false, hashes, '2099-12-31T23:59:59Z'], false, 'bi', true);
257           await this.submit('CreateWallet', [walletId, id, tier], false, 'bi',
258             true);
259         });
260         if (seed && fund > 0) mintTasks.push(async () => {
261           await this.submit('Mint', [walletId, fund], false, 'bi', true);
262         });
263       }

```

```

254     await this.runPool(customerTasks, seedConcurrency);
255     // Mint enforces global supply by scanning the wallet range; concurrent setup
Mints
256     // create Fabric phantom-read conflicts (validation code 12). Keep only funding
serial.
257     await this.runPool(mintTasks, 1);
258
259   }
260 }
261
262 module.exports = {
263   RetailWorkloadBase,
264   CONTRACT,
265   ACTORS,
266   actorRequest,
267   PER_TX_BASIC,
268   PER_TX_STANDARD,
269   assertSuccessfulResponse,
270   transactionDiagnostic,
271   transactionPayload,
272   transactionResults,
273   transactionStatus,
274 };

```

L.13 Source Code *Workload Negative-Path*

Bagian ini memuat kode sumber modul pengujian konformansi kebijakan yang menargetkan penolakan transaksi tidak sah sebagai keluaran yang diharapkan (berkas `benchmark/workloads/negative-path.js`).

```

1 'use strict';
2
3 const {
4   RetailWorkloadBase,
5   CONTRACT,
6   PER_TX_BASIC,
7   transactionResults,
8   transactionStatus,
9   transactionDiagnostic,
10 } = require('./retail-base');
11
12 const INFRASTRUCTURE_FAILURE = /proposalresponsepayloads do not match|channel has been
    shut down|failed to connect|deadline exceeded|unavailable|timeout|endorsement policy
    failure/i;
13
14 /**
15  * Negative-path conformance workload.
16  *
17  * Every transaction submitted here violates exactly one chaincode policy rule
18  * and MUST be rejected. The workload records "enforced" (correctly rejected) vs
19  * "gap" (unexpectedly accepted). A run with zero gaps means every rule rejected
20  * as required; a gap is an enforcement defect and is surfaced as a Caliper
21  * transaction failure.
22  *
23  * Note on Caliper accounting: on-chain rejection is the intended outcome here,
24  * so Caliper will count the rejected submissions as failed transactions. Read
25  * the per-worker "[negative-path]" line logged at cleanup for the conformance

```

```

26 * verdict (enforced vs gap), not the raw success/failure ratio.
27 *
28 * Rules exercised: per-transaction cap, insufficient balance, receiver
29 * maximum-balance cap, frozen sender, expired KYC, prohibited-risk approval,
30 * and high-risk approval without senior sign-off.
31 *
32 * This is a scaffold: it is designed to run against the deployed chaincode and
33 * report a conformance verdict. Verify each fixture's expected rejection reason
34 * against the current chaincode before citing the numbers.
35 */
36 class NegativePathWorkload extends RetailWorkloadBase {
37     constructor() {
38         super();
39         this.rejectedStatuses = 0;
40         this.gaps = 0;
41         this.infrastructureErrors = 0;
42         this.reasonGaps = 0;
43         this.deferredReasons = 0;
44         this.cases = [];
45     }
46
47     async initializeWorkloadModule(workerIndex, totalWorkers, roundIndex, roundArguments
, sutAdapter, sutContext) {
48         await super.initializeWorkloadModule(workerIndex, totalWorkers, roundIndex,
roundArguments, sutAdapter, sutContext);
49         this.workerIndex = workerIndex;
50         this.totalWorkers = totalWorkers;
51         this.roundIndex = roundIndex;
52         await this.seedFixtures();
53         this.buildCases();
54     }
55
56     id(suffix) { return `neg_${this.ns()}_${suffix}`; }
57
58     // Create an approved retail customer + wallet in a caller-chosen state so
59     // each fixture sits in exactly the condition one negative case needs.
60     async createCustomer({ suffix, tier, risk = 'low', diligence, senior = false,
expiresAt = '2099-12-31T23:59:59Z', fund = 0 }) {
61         const id = this.id(suffix);
62         const walletId = this.walletId(id);
63         const profileId = `kyc_${id}`;
64         const hashes = JSON.stringify(['sha256:${id}']);
65         const dd = diligence || (tier === 'BASIC' ? 'simplified' : 'standard');
66         await this.submit('CreateRetailCustomer', [id, `sha256:identity:${id}`, walletId
, profileId]);
67         await this.submit('SubmitKycProfile', [profileId, 'retail_customer', id, hashes
]);
68         await this.submit('RefreshKycProfile', [profileId, 'approved', risk, dd, senior,
hashes, expiresAt]);
69         await this.submit('CreateWallet', [walletId, id, tier]);
70         if (fund > 0) await this.submit('Mint', [walletId, fund]);
71         return { id, walletId, tier };
72     }
73
74     async seedFixtures() {
75         // Mint scans the global wallet range; concurrent fixture Mints produce
76         // Fabric phantom-read conflicts. Keep setup deterministic and serial.
77         const fixtures = [
78             { suffix: 'basic_sender', tier: 'BASIC', fund: 500000 },

```

```

79         { suffix: 'std_receiver', tier: 'STANDARD', fund: 0 },
80         { suffix: 'poor_sender', tier: 'STANDARD', fund: 1000 },
81         { suffix: 'std_sender', tier: 'STANDARD', fund: 5000000 },
82         // Receiver near its BASIC max-balance cap (2,000,000).
83         { suffix: 'full_receiver', tier: 'BASIC', fund: 1900000 },
84         { suffix: 'frozen_sender', tier: 'STANDARD', fund: 5000000 },
85         { suffix: 'expired_sender', tier: 'STANDARD', fund: 5000000 },
86     ];
87     for (const fixture of fixtures) {
88         const customer = await this.createCustomer(fixture);
89         if (fixture.suffix === 'basic_sender') this.basicSender = customer;
90         else if (fixture.suffix === 'std_receiver') this.stdReceiver = customer;
91         else if (fixture.suffix === 'poor_sender') this.poorSender = customer;
92         else if (fixture.suffix === 'std_sender') this.stdSender = customer;
93         else if (fixture.suffix === 'full_receiver') this.fullReceiver = customer;
94         else if (fixture.suffix === 'frozen_sender') this.frozenSender = customer;
95         else this.expiredSender = customer;
96     }
97     await this.submit('FreezeWallet', [this.frozenSender.walletId]);
98     const expiredProfileId = `kyc_${this.expiredSender.id}`;
99     const expiredHashes = JSON.stringify(['sha256:${this.expiredSender.id}']);
100    await this.submit('RefreshKycProfile', [expiredProfileId, 'approved', 'low', '
standard', false, expiredHashes, '2000-01-01T00:00:00Z']);
101
102    await this.seedProfile(this.id('prohibited'));
103    await this.seedProfile(this.id('highrisk'));
104 }
105
106 buildCases() {
107     const cases = [
108         { label: 'per-tx-cap', expected: /per-transaction limit exceeded/i, fn: '
Transfer', args: () => [this.basicSender.walletId, this.stdReceiver.walletId,
PER_TX_BASIC + 50000, this.id('per-tx-cap')] },
109         { label: 'insufficient-balance', expected: /insufficient balance/i, fn: '
Transfer', args: () => [this.poorSender.walletId, this.stdReceiver.walletId, 500000,
this.id('insufficient-balance')] },
110         { label: 'receiver-max-balance', expected: /exceed receiver max balance/i,
fn: 'Transfer', args: () => [this.stdSender.walletId, this.fullReceiver.walletId,
200000, this.id('receiver-max-balance')] },
111         { label: 'frozen-sender', expected: /sender wallet .* is frozen/i, fn: '
Transfer', args: () => [this.frozenSender.walletId, this.stdReceiver.walletId,
50000, this.id('frozen-sender')] },
112         { label: 'prohibited-approval', fn: 'RefreshKycProfile',
expected: /prohibited-risk subject cannot be approved/i,
113         args: () => [`kyc_${this.id('prohibited')}`, 'approved', 'prohibited', '
enhanced', true, JSON.stringify(['sha256:x']), '2099-12-31T23:59:59Z'] },
114         { label: 'high-risk-no-senior', fn: 'RefreshKycProfile',
expected: /high-risk approval requires enhanced due diligence and senior
115         approval/i,
args: () => [`kyc_${this.id('highrisk')}`, 'approved', 'high', 'enhanced
', false, JSON.stringify(['sha256:x']), '2099-12-31T23:59:59Z'] },
116         { label: 'expired-kyc', expected: /KYC profile expired/i, fn: 'Transfer',
args: () => [this.expiredSender.walletId, this.stdReceiver.walletId, 50000, this.id
('expired-kyc')] },
117     ];
118     this.cases = cases;
119 }
120
121 async seedProfile(subjectId) {
122
123

```

```

124         await this.submit('SubmitKycProfile', [
125             'kyc_${subjectId}',
126             'retail_customer',
127             subjectId,
128             JSON.stringify(['sha256:x']),
129         ]);
130     }
131
132     async expectReject(label, fn, args, expected) {
133         const request = { ...CONTRACT, contractFunction: fn, contractArguments: args.map(
134             (String), readOnly: false );
135         let response;
136         try {
137             response = await this.sutAdapter.sendRequests(request);
138         } catch (err) {
139             this.infrastructureErrors += 1;
140             throw new Error(`[negative-path] ORACLE ERROR: connector threw before
141 returning a status for "${label}": ${err}`);
142         }
143         const results = transactionResults(response);
144         const diagnostics = results.map(transactionDiagnostic);
145         const diagnosticText = diagnostics.join('\n');
146         if (INFRASTRUCTURE_FAILURE.test(diagnosticText)) {
147             this.infrastructureErrors += 1;
148             throw new Error(`[negative-path] ORACLE ERROR: infrastructure failure for "${
149 {label}: ${diagnosticText}`);
150         }
151         const failed = results.filter(tx => transactionStatus(tx) === 'failed');
152         if (failed.length === results.length && failed.length > 0) {
153             if (!expected.test(diagnosticText)) {
154                 if (diagnostics.every(message => message === '[object Object]')) {
155                     this.deferredReasons += 1;
156                 } else {
157                     this.reasonGaps += 1;
158                     throw new Error(`[negative-path] ORACLE ERROR: expected ${expected}
159 for "${label}", got ${diagnosticText}`);
160                 }
161             }
162             this.rejectedStatuses += 1;
163             return response;
164         }
165
166         this.gaps += 1;
167         const msg = `[negative-path] ENFORCEMENT GAP: "${label}" committed but a policy
168 rule should reject it`;
169         console.error(msg);
170         throw new Error(msg);
171     }
172
173     async submitTransaction() {
174         const c = this.cases[this.txIndex++ % this.cases.length];
175         return this.expectReject(c.label, c.fn, c.args(), c.expected);
176     }
177
178     async cleanupWorkloadModule() {
179         const rejectedAll = this.gaps === 0 && this.reasonGaps === 0 && this.
180 infrastructureErrors === 0 && this.rejectedStatuses === this.cases.length;
181         console.log(JSON.stringify({

```

```

177         event: 'negative-path-oracle',
178         worker: this.workerIndex,
179         rejected: this.rejectedStatuses,
180         expected: this.cases.length,
181         gaps: this.gaps,
182         reasonGaps: this.reasonGaps,
183         deferredReasons: this.deferredReasons,
184         infrastructureErrors: this.infrastructureErrors,
185         verdict: rejectedAll ? (this.deferredReasons === 0 ? 'PASS' : 'PENDING') : '
FAIL',
186     }));
187     console.log(`[negative-path] worker ${this.workerIndex}: rejected_status=${this.
rejectedStatuses}/${this.cases.length}, gaps=${this.gaps}`);
188 }
189 }
190
191 function createWorkloadModule() {
192     return new NegativePathWorkload();
193 }
194
195 module.exports.createWorkloadModule = createWorkloadModule;

```

L.14 Source Code *Workload Aggregate Overspend*

Bagian ini memuat kode sumber modul pengujian dengan satu pengirim bersama, permintaan agregat yang melebihi saldo, klasifikasi alasan penolakan, dan pemeriksaan saldo akhir (berkas `benchmark/workloads/adversarial-double-spend.js`).

```

1 'use strict';
2
3 const {
4     RetailWorkloadBase,
5     CONTRACT,
6     transactionDiagnostic,
7     transactionPayload,
8     transactionResults,
9     transactionStatus,
10 } = require('./retail-base');
11
12 /**
13  * Adversarial aggregate-overspend / hot-key workload.
14  *
15  * All workers issue transfers from ONE shared sender wallet, so concurrent
16  * transfers read the same balance version. The aggregate requested value is
17  * deliberately greater than the sender's funded balance. Rejections are
18  * classified from their actual diagnostics instead of being assumed to be MVCC.
19  *
20  * Requires >= 2 Caliper workers and an overlapping rate to create contention; a
21  * single worker serialises submissions and will not conflict.
22  *
23  * Reports committed and rejected transactions by reason plus the final balance.
24  * Each worker verifies the non-negative-balance invariant; the suite runner
25  * verifies that at least one worker observed a rejection.
26  */
27 class AdversarialDoubleSpendWorkload extends RetailWorkloadBase {
28     constructor() {

```

```

29     super();
30     this.committed = 0;
31     this.rejected = 0;
32     this.rejectionReasons = {
33         mvcc: 0,
34         insufficientBalance: 0,
35         endorsement: 0,
36         other: 0,
37     };
38 }
39
40 async initializeWorkloadModule(workerIndex, totalWorkers, roundIndex, roundArguments
, sutAdapter, sutContext) {
41     await super.initializeWorkloadModule(workerIndex, totalWorkers, roundIndex,
roundArguments, sutAdapter, sutContext);
42     this.workerIndex = workerIndex;
43     this.totalWorkers = totalWorkers;
44     this.roundIndex = roundIndex;
45     this.transferAmount = this.arg('transferAmount', 1000);
46     this.senderFunding = this.arg('senderFunding', 50000);
47     await this.seedSharedSender();
48     await this.seedReceivers(this.arg('receivers', 20));
49 }
50
51 // Shared across workers: namespaced by round only, NOT by worker.
52 sharedSenderId() {
53     const s = this.arg('scenario', null);
54     return `${s ? s + '_' : ''}dblsend_sender_r${this.roundIndex}`;
55 }
56
57 async seedSharedSender() {
58     const id = this.sharedSenderId();
59     const walletId = this.walletId(id);
60     const profileId = `kyc_${id}`;
61     const hashes = JSON.stringify(['sha256:${id}']);
62     if (this.workerIndex === 0) {
63         await this.submit('CreateRetailCustomer', [id, `sha256:identity:${id}`,
walletId, profileId]);
64         await this.submit('SubmitKycProfile', [profileId, 'retail_customer', id,
hashes]);
65         await this.submit('RefreshKycProfile', [profileId, 'approved', 'low', '
standard', false, hashes, '2099-12-31T23:59:59Z']);
66         await this.submit('CreateWallet', [walletId, id, 'STANDARD']);
67         await this.submit('Mint', [walletId, this.senderFunding]);
68     } else {
69         await this.waitForWallet(walletId);
70     }
71     this.senderWallet = walletId;
72 }
73
74 async waitForWallet(walletId) {
75     let lastError;
76     for (let attempt = 0; attempt < 120; attempt++) {
77         try {
78             await this.submit('GetWallet', [walletId], true);
79             return;
80         } catch (error) {
81             lastError = error;
82             await new Promise(resolve => setTimeout(resolve, 500));

```

```

83         }
84     }
85     throw new Error(`shared sender was not seeded: ${lastError}`);
86 }
87
88 async seedReceivers(n) {
89     this.receivers = [];
90     const tasks = [];
91     for (let i = 0; i < n; i++) {
92         const rid = `dbl_rcv_${this.ns()}_${i}`;
93         const walletId = this.walletId(rid);
94         this.receivers.push(walletId);
95         const profileId = `kyc_${rid}`;
96         const hashes = JSON.stringify(['sha256:${rid}']);
97         tasks.push(async () => {
98             await this.submit('CreateRetailCustomer', [rid, `sha256:identity:${rid}`], walletId, profileId);
99             await this.submit('SubmitKycProfile', [profileId, 'retail_customer', rid, hashes]);
100             await this.submit('RefreshKycProfile', [profileId, 'approved', 'low', 'standard', false, hashes, '2099-12-31T23:59:59Z']);
101             await this.submit('CreateWallet', [walletId, rid, 'STANDARD']);
102         });
103     }
104     await this.runPool(tasks, this.arg('seedConcurrency', 2));
105 }
106
107 async submitTransaction() {
108     const receiver = this.receivers[this.txIndex++ % this.receivers.length];
109     const request = {
110         ...CONTRACT,
111         contractFunction: 'Transfer',
112         contractArguments: [this.senderWallet, receiver, String(this.transferAmount), `overspend_${this.ns()}_${this.txIndex}`],
113         readOnly: false,
114     };
115     try {
116         const response = await this.sutAdapter.sendRequests(request);
117         for (const tx of transactionResults(response)) {
118             if (transactionStatus(tx) === 'success') {
119                 this.committed += 1;
120             } else {
121                 this.recordRejection(transactionDiagnostic(tx));
122             }
123         }
124     } catch (error) {
125         this.recordRejection(String(error));
126     }
127 }
128
129 recordRejection(diagnostic) {
130     this.rejected += 1;
131     if (/MVCC_READ_CONFLICT/i.test(diagnostic)) {
132         this.rejectionReasons.mvcc += 1;
133     } else if (/insufficient balance/i.test(diagnostic)) {
134         this.rejectionReasons.insufficientBalance += 1;
135     } else if (/endorsement/i.test(diagnostic)) {
136         this.rejectionReasons.endorsement += 1;
137     } else {

```

```

138         this.rejectionReasons.other += 1;
139     }
140 }
141
142 async readFinalBalance() {
143     const response = await this.sutAdapter.sendRequests({
144         ...CONTRACT,
145         contractFunction: 'GetWallet',
146         contractArguments: [this.senderWallet],
147         readOnly: true,
148     });
149     const tx = transactionResults(response)[0];
150     if (transactionStatus(tx) !== 'success') {
151         throw new Error('final wallet read failed: ${transactionDiagnostic(tx)}');
152     }
153     const wallet = JSON.parse(transactionPayload(tx));
154     return Number(wallet.balance);
155 }
156
157 async cleanupWorkloadModule() {
158     const finalBalance = await this.readFinalBalance();
159     const invariant = Number.isFinite(finalBalance) && finalBalance >= 0;
160     console.log(
161         `[overspend-contention] worker ${this.workerIndex}: committed=${this.
162 committed}, rejected=${this.rejected}, `
163         + `mvcc=${this.rejectionReasons.mvcc}, insufficient_balance=${this.
164 rejectionReasons.insufficientBalance}, `
165         + `endorsement=${this.rejectionReasons.endorsement}, other=${this.
166 rejectionReasons.other}, `
167         + `final_balance=${finalBalance}, invariant=${invariant ? 'PASS' : 'FAIL'}`,
168     );
169     if (!invariant) {
170         throw new Error('aggregate-overspend invariant failed');
171     }
172 }
173
174 function createWorkloadModule() {
175     return new AdversarialDoubleSpendWorkload();
176 }
177
178 module.exports.createWorkloadModule = createWorkloadModule;

```